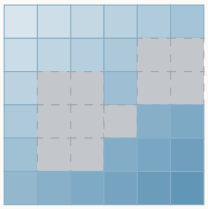


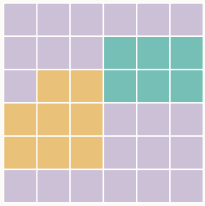
(T, H, W, C)

UNE VIDÉO



$x \longrightarrow E_\theta(x)$

ON EN CACHE



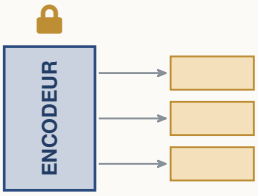
$\|\hat{z} - z\|_1$

ON DEVINE L'IDÉE

V-JEPA 2.1 pas à pas

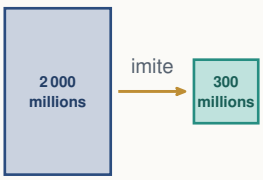
Comprendre, coder et déployer un modèle vidéo auto-supervisé, en partant vraiment de zéro

Aucun prérequis : on part vraiment de zéro.



une sonde par tâche

ENCODEUR GELÉ



presque la même qualité

DISTILLATION

Doctor Mokira

2026

Édition 2026. Document composé avec LaTeX (pdfTeX), polices Palatino (texte et mathématiques), TeX Gyre Heros (titres) et DejaVu Sans Mono (code). Figures et schémas réalisés avec TikZ et pgfplots, coloration syntaxique assurée par minted/Pygments, encadrés par tcolorbox, bibliographie par biblatex.

Ce livre explique de bout en bout l'article *V-JEPA 2.1 : Unlocking Dense Features in Video Self-Supervised Learning*, publié par l'équipe FAIR de Meta et l'Université de Saragosse. Il est écrit pour être lu sans aucun prérequis : on part de zéro, on explique avec des mots avant d'écrire des symboles, et on n'avance jamais sans avoir d'abord dit pourquoi. Il ne s'arrête pas à la compréhension : il conduit le lecteur jusqu'au code qui tourne, puis jusqu'au modèle mis en service.

Avant-propos

« Un article de recherche ne devrait effrayer personne. »

Ce livre raconte un seul article scientifique, du premier mot jusqu'à la dernière ligne de code : *V-JEPA 2.1 : Unlocking Dense Features in Video Self-Supervised Learning* [2], écrit par l'équipe FAIR de Meta et l'Université de Saragosse. Cet article présente un programme qui apprend à comprendre des vidéos tout seul, sans qu'un humain lui explique jamais ce qu'il regarde. Il fait suite à un premier modèle, V-JEPA 2 [1], dont nous parlerons beaucoup : c'est en corrigeant ses défauts que V-JEPA 2.1 est né.

Un article de recherche est écrit par des spécialistes, pour des spécialistes. Il va vite. Il emploie des mots que personne ne vous a jamais définis. Il suppose que vous connaissez déjà dix autres articles. Résultat : beaucoup de gens curieux, capables, motivés, s'arrêtent à la première page.

Ici, on ne suppose rien du tout. Chaque idée est d'abord racontée avec des mots de tous les jours, puis dessinée, puis seulement écrite en langage mathématique, puis enfin mise en pratique avec du code que vous pouvez faire tourner vous-même.

> **En clair**
>
> Notre promesse : si vous savez compter et si vous savez lire, vous pouvez comprendre comment fonctionne V-JEPA 2.1. Le reste, nous le construisons ensemble, dans l'ordre, sans jamais sauter une marche.

Pour qui est ce livre

Pour toute personne qui veut trois choses à la fois :

1. **comprendre** ce que fait V-JEPA 2.1, et pourquoi il fait mieux que le modèle qui existait avant lui ;

2. **écrire elle-même** le modèle et le programme qui l'entraîne ;
3. **mettre le modèle au travail** pour de vrai, tous les jours, au service d'une application.

Aucun diplôme n'est demandé. Vous n'avez pas besoin de savoir ce qu'est un réseau de neurones, ni un transformeur, ni même une image numérique : tout cela est expliqué à partir de rien, dans les quatre premiers chapitres.

Comment ce livre est fabriqué

Huit règles ont guidé chaque page, et elles ne sont pas des intentions : ce sont des engagements que vous pouvez vérifier.

1. Aucun mot technique n'apparaît sans avoir été défini juste avant.
2. On commence toujours par les prérequis, jamais par la nouveauté.
3. Un dessin pour chaque idée qui bouge, qui circule ou qui se compare.
4. Un exemple après chaque notion, avec des chiffres quand c'est possible.
5. Des mots simples, ceux de tous les jours.
6. Le texte vous parle et vous accompagne.
7. Une phrase de transition à la fin de chaque section, qui annonce la suite.
8. Des phrases entières, comme dans un vrai livre.

Le texte utilise quelques encadrés qui reviennent souvent, faciles à reconnaître à leur pictogramme :

- **En clair** — l'idée expliquée avec des mots de tous les jours.
- **L'intuition** — l'image mentale à garder en tête.
- **À quoi ça sert en IA ?** — où cette notion sert vraiment dans V-JEPA 2.1.
- **Sur des données réelles** — la même idée, appliquée aux chiffres réels de l'article.
- **Ce qu'il faut savoir avant** — les seuls prérequis de la section.
- **Piège à éviter** — l'erreur classique que tout le monde fait.
- **À retenir** — l'essentiel, si vous ne deviez garder qu'une chose.

- 🚀 **Pour aller plus loin** — un pont vers un sujet plus avancé, que vous pouvez sauter en première lecture.

Chaque chapitre se termine par des exercices, tous corrigés. Ils ne servent pas à vous noter : ils servent à vous montrer, à vous-même, que vous avez compris.

Un mot d'honnêteté

L'article explique comment *entraîner* le modèle et comment l'*évaluer*. Il ne parle presque pas de sa *mise en service* : pas de serveur, pas de temps de réponse, pas de surveillance. La dernière partie de ce livre sera donc de deux couleurs, et nous vous le dirons franchement à chaque fois :

- **Ce que l'article dit** — la taille des modèles, la résolution des images, la vitesse de calcul. Ce sont des résultats mesurés.
- **Ce que nous ajoutons** — le serveur, la mesure du temps de réponse, la surveillance. Ce sont des conseils de métier, utiles et éprouvés, mais qui ne viennent pas de l'article.

Nous dirons aussi, aussi clairement que ses réussites, ce que ce modèle ne sait pas faire, et les tâches sur lesquelles d'autres modèles restent meilleurs que lui. Un livre qui ne raconte que les victoires ne vous aide pas à choisir.

Bonne lecture.

Table des matières

Avant-propos iii

Table des matières vii

I Les fondations : partir vraiment de zéro 1

1 Une image, une vidéo, vues par un ordinateur 3
- 1.1 Un ordinateur ne voit pas des objets, il voit des nombres . . . 4
- 1.2 La résolution : la finesse de la mosaïque 6
- 1.3 La couleur, c'est trois grilles au lieu d'une 7
- 1.4 Ranger les nombres : le tenseur et sa forme 9
- 1.5 Une vidéo, c'est une pile d'images 11
- 1.6 Ce que tout cela pèse vraiment 14
- 1.7 Passons au code 16
- Exercices 18

2 Ce que veut dire « apprendre » pour une machine 21
- 2.1 Une machine qui apprend, c'est une machine à boutons 22
- 2.2 Se tromper, et mesurer de combien 25
- 2.3 De quel côté tourner le bouton ? La pente 28
- 2.4 La descente de gradient, calculée à la main 30
- 2.5 La taille des pas : le taux d'apprentissage 34
- 2.6 Le rythme de l'entraînement : le lot et l'époque 36
- 2.7 Apprendre par cœur n'est pas apprendre 39
- 2.8 Passons au code 41
- Exercices 45

3 Apprendre sans étiquettes : l'auto-supervision 49
- 3.1 La bonne réponse écrite à côté : l'étiquette 50
- 3.2 Le mur : ce que coûte vraiment une étiquette 51
- 3.3 L'idée qui change tout : cacher un morceau 53
- 3.4 La tâche prétexte : un jeu que l'on jette après 55
- 3.5 Ce que l'on garde vraiment : la représentation 57

3.6 Comment savoir si une représentation est bonne 59
3.7 À quelle échelle : les chiffres de V-JEPA 2.1 62
3.8 Passons au code 64
Exercices 72

4 Le transformeur de vision, l'œil de la machine 75
4.1 Pourquoi on ne peut pas regarder les pixels un par un 76
4.2 Le patch : découper l'image en petits carrés 78
4.3 Du carré au jeton : le plongement 80
4.4 Le tubelet : découper aussi dans le temps 83
4.5 Dire où l'on se trouve : l'encodage de position 86
4.6 L'attention : chaque jeton écoute les autres 89
4.7 Plusieurs têtes valent mieux qu'une 94
4.8 Le bloc de transformeur, ouvert en coupe 96
4.9 Empiler les blocs : la profondeur et les quatre tailles 101
4.10 Ce que l'attention coûte vraiment 104
4.11 Passons au code 105
Exercices 110

II Comprendre JEPA et V-JEPA 2, l'ancêtre direct 115

III Le problème que V-JEPA 2.1 vient résoudre 117

IV V-JEPA 2.1, ingrédient par ingrédient 119

V Les résultats, lus avec honnêteté 121

VI Construire le modèle et le programme d'entraînement 123

VII De l'entraînement à la production 125

Bibliographie 127

Première partie

Les fondations : partir vraiment de zéro

CHAPITRE 1

Une image, une vidéo, vues par un ordinateur

Avant de parler d'intelligence artificielle, de vidéos cachées et de modèles à deux milliards de réglages, mettons-nous d'accord sur une chose toute simple : qu'est-ce qu'une image, pour une machine ? Ce chapitre pose le décor, tranquillement, sans rien supposer.

> **Ce qu'il faut savoir avant**
>
> Rien du tout. Vraiment rien. Il faut savoir compter, savoir qu'une multiplication sert à répéter une addition, et c'est tout. Chaque mot nouveau sera expliqué au moment où il arrive.

Vous regardez une photo de chat. Vous voyez un chat. C'est immédiat, c'est gratuit, vous n'avez rien eu à faire. Un ordinateur, lui, ne voit pas de chat. Il ne voit même pas d'image. Il voit une très longue liste de nombres, et rien d'autre.

Tout ce livre part de là. Si nous voulons qu'une machine finisse par comprendre une vidéo, il faut d'abord savoir sous quelle forme la vidéo arrive chez elle. C'est exactement ce que nous allons faire dans ce chapitre : ouvrir le capot et regarder les nombres.

1.1 Un ordinateur ne voit pas des objets, il voit des nombres

En clair

Une image, pour un ordinateur, c'est une grille de petits carrés. Chaque carré porte un nombre, et ce nombre dit une seule chose : à quel point ce petit carré est clair ou sombre.

Pensez à une mosaïque, comme celles qu'on voit sur les murs des vieilles maisons. De loin, vous voyez un dessin. De près, vous voyez des centaines de petits carreaux collés les uns aux autres, chacun d'une seule couleur. Une image numérique, c'est exactement cela : une mosaïque de petits carrés, tous de la même taille, rangés en lignes et en colonnes.

Définition 1.1 (Le pixel)

Un *pixel* est un de ces petits carrés. C'est le plus petit morceau d'une image : on ne peut pas le couper en deux. Le mot vient de l'anglais *picture element*, c'est-à-dire « élément d'image ». Chaque pixel porte un nombre qui décrit sa couleur.

Commençons par le cas le plus simple : une image en *niveaux de gris*, c'est-à-dire une image sans couleur, comme une vieille photo en noir et blanc. Dans ce cas, un seul nombre suffit par pixel. On se met d'accord une fois pour toutes sur ce que ce nombre veut dire :

- le nombre **0** veut dire « ce carré est complètement noir » ;
- le nombre **255** veut dire « ce carré est complètement blanc » ;
- tous les nombres entre les deux sont des gris, de plus en plus clairs à mesure qu'on monte.

La figure 1.1 montre les deux faces de la même image : à gauche ce que la machine a réellement en mémoire, à droite ce que nous voyons apparaître à l'écran.

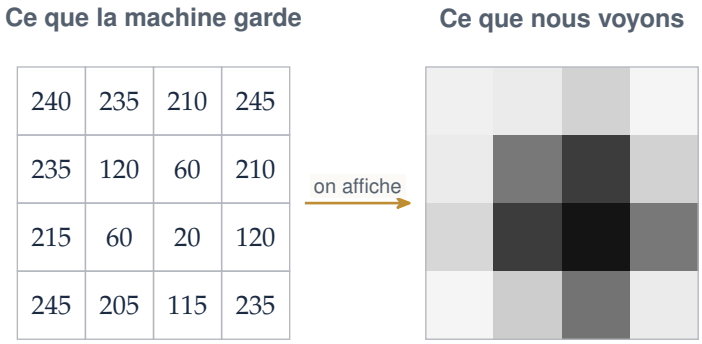


FIGURE 1.1. Les deux visages d'une même image. À gauche, les seize nombres que l'ordinateur garde en mémoire. À droite, le dessin obtenu en peignant chaque carré : petit nombre, carré sombre ; grand nombre, carré clair. Il n'y a rien d'autre dans une image que ces nombres.

Exemple 1.2 (Lire trois pixels)

Dans la grille de gauche de la figure 1.1, le pixel tout en haut à gauche vaut 240 : il est presque blanc. Le pixel du milieu vaut 20 : il est presque noir. Un pixel qui vaudrait 128, soit à peu près la moitié de 255, serait un gris moyen, ni clair ni foncé.

L'intuition

Voyez chaque pixel comme un petit interrupteur à gradateur, du genre de ceux qui règlent la lumière d'un salon. Tourné à fond dans un sens, la lampe est éteinte : c'est 0. Tourné à fond dans l'autre, la lampe éclaire au maximum : c'est 255. Une image, c'est un mur couvert de milliers de ces petites lampes, chacune réglée à sa valeur.

Piège à éviter

Pourquoi 255 et pas 100, ou 1000 ? C'est une simple convention, née de la façon dont les ordinateurs rangent les nombres : avec huit cases de mémoire, on peut écrire exactement 256 valeurs différentes, de 0 à 255. Retenez surtout le sens de lecture : **plus le nombre est grand, plus le pixel est clair**. Beaucoup de débutants inversent, dessinent des images

en négatif, et cherchent l'erreur pendant une heure.

Nous savons maintenant ce que contient une case de la mosaïque. La question suivante vient toute seule : combien y a-t-il de cases ?

1.2 La résolution : la finesse de la mosaïque

Définition 1.3 (La résolution)

La *résolution* d'une image, c'est le nombre de pixels qu'elle contient, donné sous la forme « largeur × hauteur ». Une image de résolution 256×256 possède 256 colonnes de pixels et 256 lignes de pixels.

Plus la mosaïque a de carreaux, plus le dessin est fin. Avec très peu de carreaux, un rond ressemble à un escalier ; avec beaucoup, il redevient un rond. La figure 1.2 montre le même disque dessiné deux fois, la première fois grossièrement, la seconde finement.

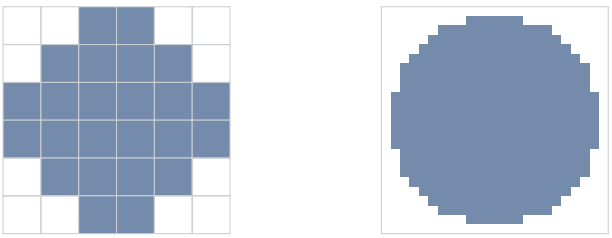


FIGURE 1.2. Le même disque, dessiné avec deux finesses différentes. À gauche, la mosaïque est grossière et les bords deviennent des marches d'escalier. À droite, les carreaux sont seize fois plus nombreux et l'œil retrouve un vrai rond. Rien n'a changé dans le dessin : seule la finesse de la grille a changé.

Le nombre total de pixels s'obtient par une simple multiplication : on multiplie la largeur par la hauteur, comme on calculerait le nombre de carreaux d'un sol.

Exemple 1.4 (Compter les pixels d'une vraie image)

Une image de résolution 256×256, qui est la taille utilisée pendant la plus grande partie de l'entraînement de V-JEPA 2.1, contient

$$256 \times 256 = 65\,536 \text{ pixels.}$$

Soit soixante-cinq mille petits carrés. Et ce n'est encore qu'une image en gris : nous n'avons même pas parlé de couleur.

À retenir

Deux images peuvent montrer exactement la même chose et ne pas avoir la même résolution. Quand la résolution monte, le dessin gagne en finesse, mais le nombre de nombres à traiter grimpe très vite : il grimpe comme le *carré* de la taille. Doubler la largeur et la hauteur, c'est multiplier le travail par quatre. Cette remarque, qui paraît anodine ici, expliquera plus tard pourquoi la dernière phase d'entraînement de V-JEPA 2.1 est si courte.

Nos images sont encore grises. Ajoutons la couleur : vous allez voir que cela ne change presque rien à l'histoire.

1.3 La couleur, c'est trois grilles au lieu d'une

Un écran ne sait fabriquer que trois couleurs de lumière : du rouge, du vert et du bleu. Toutes les autres couleurs sont obtenues en mélangeant ces trois-là, en dosant chacune plus ou moins fort. Du rouge à fond plus du vert à fond donne du jaune. Les trois à fond donnent du blanc. Les trois éteintes donnent du noir.

Un pixel en couleur a donc besoin non pas d'un nombre, mais de trois : un pour le rouge, un pour le vert, un pour le bleu. Chacun se lit comme avant, de 0 (éteint) à 255 (à fond).

Définition 1.5 (Le canal)

Un *canal* est une grille complète de nombres qui ne s'occupe que d'une seule couleur de base. Une image en couleur possède trois canaux superposés : le canal rouge, le canal vert et le canal bleu. On les appelle souvent « les canaux R, V, B », ou « RGB » à l'anglaise.

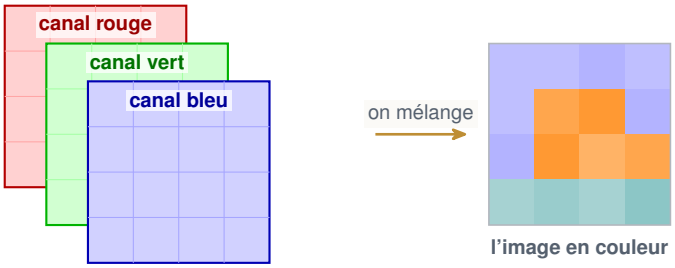


FIGURE 1.3. Une image en couleur, c'est trois mosaïques posées les unes sur les autres. La première dit combien il y a de rouge dans chaque carré, la deuxième combien de vert, la troisième combien de bleu. En regardant les trois ensemble, l'œil reconstitue la couleur.

Exemple 1.6 (Quelques couleurs, en trois nombres)

Voici comment se lisent quelques couleurs courantes, dans l'ordre rouge, vert, bleu :

Couleur	Rouge	Vert	Bleu
Noir	0	0	0
Blanc	255	255	255
Rouge vif	255	0	0
Jaune	255	255	0
Gris moyen	128	128	128

Vous remarquez que le gris s'obtient en donnant la même valeur aux trois canaux. C'est logique : aucune couleur ne tire la couverture à elle.

Comptons maintenant, pour de bon, le nombre de cases d'une image en couleur.

Exemple 1.7 (Le poids d'une image de 256×256)

Notre image possède 65 536 pixels, et chaque pixel réclame trois nombres. Au total :

$$256 \times 256 \times 3 = 196\,608 \text{ nombres.}$$

Presque deux cent mille nombres pour une seule petite image carrée, plus petite que la moitié d'un écran de téléphone. Gardez ce chiffre en tête : il va grossir très vite.

Nous avons beaucoup de nombres sur les bras. Il est temps de les ranger proprement, et de leur donner un nom.

1.4 Ranger les nombres : le tenseur et sa forme

Pour parler de tous ces nombres sans se perdre, les informaticiens utilisent un mot unique, qui a l'air savant mais qui ne cache rien de compliqué : le *tenseur*. Approchons-le par étapes, en montant une marche à la fois.

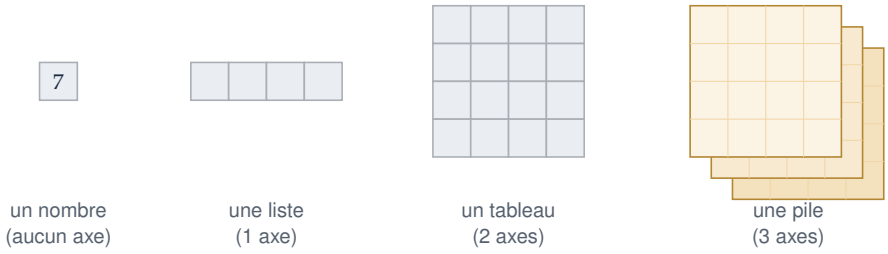


FIGURE 1.4. La même idée, en quatre marches. Un nombre tout seul ; une liste de nombres ; un tableau de nombres ; puis une pile de tableaux. À chaque marche, on ajoute une direction pour se déplacer, c'est-à-dire un axe. Le mot « tenseur » désigne tout cela d'un coup, quel que soit le nombre d'axes.

Définition 1.8 (Le tenseur)

Un *tenseur* est un tableau de nombres qui peut avoir autant de directions que l'on veut. Un nombre seul est un tenseur sans direction ; une liste est un tenseur à une direction ; un tableau à lignes et colonnes est un tenseur à deux directions ; une pile de tableaux est un tenseur à trois

directions, et ainsi de suite. Chaque direction s'appelle un *axe*.

Pour décrire un tenseur, il suffit de dire combien de cases il compte le long de chaque axe. Cette liste de tailles porte un nom, et c'est le mot le plus utile de tout le chapitre.

Notation 1.9 (La forme d'un tenseur)

La *forme* d'un tenseur est la liste du nombre de cases sur chaque axe. On l'écrit entre parenthèses. Pour une image en couleur, on écrit

$$(\text{hauteur},\ \text{largeur},\ \text{canaux}),$$

et notre image de tout à l'heure a donc la forme $(256, 256, 3)$.

Exemple 1.10 (Lire une forme)

Un tenseur de forme $(64, 384, 384, 3)$ se lit de gauche à droite comme une phrase : « soixante-quatre choses, chacune haute de 384 cases, large de 384 cases, et épaisse de 3 canaux ». Nous verrons dans un instant que ces soixante-quatre choses sont les images d'une vidéo.

Piège à éviter

L'ordre des axes n'est pas le même partout, et c'est la source d'erreur numéro un des débutants. Deux conventions se partagent le monde :

- $(\text{hauteur},\ \text{largeur},\ \text{canaux})$ — l'ordre des outils d'affichage et de photo ;
- $(\text{canaux},\ \text{hauteur},\ \text{largeur})$ — l'ordre des bibliothèques d'apprentissage comme PyTorch.

Les nombres sont exactement les mêmes, seule leur façon d'être rangés change. Mais si vous vous trompez, votre programme croira qu'une image de 256×256 en couleur est une image de 3 pixels de haut sur 256 de large avec 256 canaux, et vous obtiendrez une bouillie. Prenez l'habitude, dès aujourd'hui, d'écrire la forme attendue en commentaire au-dessus de chaque ligne de code qui manipule une image.

Nous savons ranger une image. Passons à ce qui nous intéresse vraiment dans ce livre : la vidéo.

1.5 Une vidéo, c'est une pile d'images

Une vidéo n'est rien d'autre qu'une suite d'images fixes, montrées les unes après les autres, très vite. Si elles défilent assez vite, notre œil ne voit plus des images séparées : il voit du mouvement. C'est le principe du petit carnet à feuilleter, celui où l'on dessine un bonhomme dans le coin de chaque page et où l'on fait défiler les pages avec le pouce.

Définition 1.11 (L'image d'une vidéo)

Chacune des images fixes qui composent une vidéo s'appelle une *image*, ou une *trame* ; en anglais on dit *frame*, et ce mot revient tellement souvent dans les articles et dans le code qu'il faut le connaître. Une vidéo est donc une pile d'images empilées non pas dans l'espace, mais dans le temps.

Définition 1.12 (Les images par seconde)

Le nombre d'*images par seconde* dit combien d'images sont montrées pendant une seconde. On l'abrège souvent en *fps*, de l'anglais *frames per second*. Un film de cinéma tourne à 24 images par seconde ; les vidéos de téléphone, souvent à 30 ou 60.

Puisque la vidéo est une pile d'images, sa forme s'obtient en ajoutant un axe devant celle d'une image : l'axe du temps.

Notation 1.13 (La forme d'une vidéo)

Un extrait de vidéo s'écrit sous la forme

$$(\text{temps}, \text{hauteur}, \text{largeur}, \text{canaux}),$$

où le premier nombre est le nombre d'images de l'extrait. On appelle souvent un tel extrait un *clip* : c'est un petit morceau découpé dans une vidéo plus longue, exactement comme on découpe un morceau dans une bobine de film.

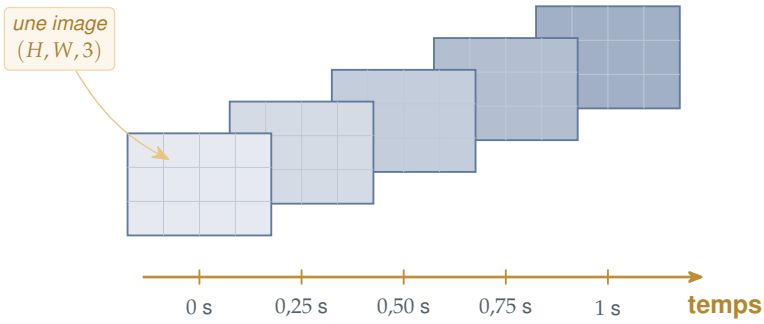


FIGURE 1.5. Une vidéo vue comme une pile de feuilles. Chaque feuille est une image complète, avec ses trois canaux de couleur. La flèche du bas est le temps : elle avance d'un cran à chaque nouvelle feuille. Ici, quatre images par seconde, donc un quart de seconde entre deux feuilles.

La durée d'un clip se calcule sans effort. Si vous avez un certain nombre d'images, et que vous savez combien d'images passent en une seconde, alors la division vous donne les secondes :

$$\text{durée en secondes} = \frac{\text{nombre d'images}}{\text{images par seconde}}.$$

Exemple 1.14 (La durée d'un clip d'entraînement)

Pendant la plus grande partie de son entraînement, V-JEPA 2.1 regarde des clips de 16 images prises à raison de 4 images par seconde. Ces clips durent donc

$$\frac{16}{4} = 4 \text{ secondes}.$$

Quatre secondes, c'est court, mais c'est bien assez pour voir un geste commencer et se terminer : une main qui saisit une tasse, une porte qui s'ouvre. Et c'est justement ce que le modèle doit apprendre à comprendre.

Sur des données réelles

L'article de V-JEPA 2.1 utilise deux réglages, l'un après l'autre. Pendant la longue phase d'entraînement principale, les clips font 16 images en 256×256. Pendant la courte phase finale, appelée phase de refroidis-

sement, on monte à 64 images en 384×384. Toujours à quatre images par seconde, ces clips-là durent

$$\frac{64}{4} = 16 \text{ secondes.}$$

Le modèle regarde donc, à la fin, des extraits quatre fois plus longs et beaucoup plus fins. Nous verrons au chapitre consacré au refroidissement pourquoi cet ordre — d'abord grossier et court, ensuite fin et long — n'est pas un caprice.

Comptons enfin les nombres contenus dans un clip, en multipliant tout simplement les quatre tailles entre elles.

Exemple 1.15 (Le nombre de cases d'un clip)

Pour un clip de 16 images en 256×256 couleur :

$$16 \times 256 \times 256 \times 3 = 3\,145\,728 \text{ nombres.}$$

Plus de trois millions. Et pour un clip de la phase de refroidissement, 64 images en 384×384 :

$$64 \times 384 \times 384 \times 3 = 28\,311\,552 \text{ nombres,}$$

soit environ vingt-huit millions, c'est-à-dire neuf fois plus que le précédent.

À retenir

Une image est un tenseur à trois axes, une vidéo un tenseur à quatre axes. Rien de plus mystérieux : on a seulement ajouté le temps devant. Toutes les formes que vous croiserez dans ce livre se lisent avec cette seule règle.

Trois millions de nombres, cela reste abstrait. Regardons ce que cela représente vraiment dans la mémoire d'une machine.

1.6 Ce que tout cela pèse vraiment

Un ordinateur range ses nombres dans des cases de mémoire. L'unité de mesure de cette mémoire est l'*octet*.

Définition 1.16 (L'octet)

Un *octet* est un petit paquet de mémoire, celui qui suffit tout juste à écrire un nombre entier compris entre 0 et 255. C'est aussi l'unité dans laquelle on mesure la taille des fichiers. Mille octets font environ un kilo-octet, un million d'octets font environ un méga-octet (noté Mo), un milliard d'octets font environ un giga-octet (noté Go).

Nos pixels vont de 0 à 255 : ils tiennent donc chacun dans un seul octet. Mais dès qu'un modèle se met à calculer, il a besoin de nombres à virgule — parce qu'une moyenne, une multiplication ou une petite correction tombent rarement sur un compte rond. Ces nombres à virgule s'appellent des *nombres flottants*, et le format le plus courant en occupe quatre octets chacun, soit quatre fois plus.

Exemple 1.17 (Du fichier à la mémoire de calcul)

Notre clip de 3 145 728 nombres pèse :

- 3 145 728 octets, soit environ **3 Mo**, tant que les pixels restent des entiers de 0 à 255 ;
- $3\,145\,728 \times 4 = 12\,582\,912$ octets, soit environ **12,6 Mo**, une fois convertis en nombres à virgule pour le calcul.

Or un modèle ne regarde jamais un seul clip à la fois : il en regarde tout un paquet d'un coup, pour aller plus vite. Ce paquet s'appelle un *lot*, et sa taille est un réglage du programme d'entraînement.

Exemple 1.18 (Le poids d'un lot entier)

V-JEPA 2.1 traite 128 clips à la fois. Pour la phase principale, cela donne

$$128 \times 12\,582\,912 \approx 1\,610\,000\,000 \text{ octets},$$

soit environ **1,6 Go** — rien que pour les vidéos d'entrée, avant même le

moindre calcul. Pour la phase de refroidissement, où chaque clip pèse neuf fois plus, on arrive à environ **14,5 Go**. Voilà, en une ligne de calcul, pourquoi cette phase-là ne dure que très peu de temps : elle coûte cher.

À quoi ça sert en IA ?

Ce chapitre a l'air de ne parler que de pixels, mais il pose déjà les trois problèmes que tout le reste du livre s'emploie à résoudre.

- **Il y a beaucoup trop de nombres.** Aucun modèle ne peut traiter trois millions de valeurs une par une. Il faudra donc d'abord les regrouper en petits carrés de 16×16 pixels : c'est ce que nous ferons au chapitre consacré au transformeur de vision.
- **La finesse coûte cher.** Chaque fois que la résolution monte, la facture explose. C'est pour cela que l'entraînement commence en basse résolution et ne monte qu'à la toute fin.
- **Lire une vidéo prend du temps.** Une vidéo est stockée compressée ; il faut la décompresser image par image avant de pouvoir en faire quoi que ce soit. En production — c'est-à-dire quand le modèle travaille pour de vrai, tous les jours, au service d'une application — ce travail-là devient souvent plus lent que le modèle lui-même. Nous y reviendrons longuement dans la partie consacrée à la mise en service.

Pour aller plus loin

Dans le même temps où il regarde 128 clips, V-JEPA 2.1 regarde aussi 2 304 images fixes. En reprenant nos calculs, ces images pèsent $2\,304 \times 196\,608 \times 4 \approx 1{,}8$ Go, c'est-à-dire à peu près autant que les 1,6 Go de vidéo. Ces deux nombres, 128 et 2 304, n'ont donc pas été tirés au hasard : ils ont été choisis ensemble pour que les deux moitiés du travail coûtent à peu près pareil. Nous y reviendrons au chapitre sur les données.

Assez de calculs sur le papier. Vérifions tout cela sur une vraie machine.

1.7 Passons au code

Nous allons manipuler nos images et nos clips avec Python, le langage utilisé par presque tous les travaux d'intelligence artificielle, et avec `numpy`, la bibliothèque qui sait manipuler les tenseurs. Vous n'avez pas besoin de comprendre chaque ligne dès maintenant : lisez d'abord les commentaires, et comparez les nombres affichés avec ceux que nous venons de calculer à la main.

PYTHON • Fabriquer une image et lire sa forme

```python
import numpy as np

# Une image en couleur de 256 x 256 pixels, entierement noire.
# La forme se lit : (hauteur, largeur, canaux).
image = np.zeros((256, 256, 3), dtype=np.uint8)

print(image.shape)   # -> (256, 256, 3)
print(image.size)    # -> 196608   le nombre total de cases
print(image.dtype)   # -> uint8    un entier de 0 a 255
print(image.nbytes)  # -> 196608   un octet par case

# Peignons le pixel de la ligne 10, colonne 20, en rouge vif.
image[10, 20] = [255, 0, 0]
print(image[10, 20])  # -> [255   0   0]
```

Le mot `uint8` se lit « entier non signé sur huit bits » : c'est exactement le type de nombre qui va de 0 à 255, celui dont nous avons parlé. Passons maintenant à la vidéo.

PYTHON • Un clip, sa forme, sa durée et son poids

```python
import numpy as np

# Un clip de 16 images de 256 x 256 en couleur.
# La forme se lit : (temps, hauteur, largeur, canaux).
clip = np.zeros((16, 256, 256, 3), dtype=np.uint8)

print(clip.shape)   # -> (16, 256, 256, 3)
print(clip.size)    # -> 3145728   les trois millions annonces

images_par_seconde = 4.0
```

```
duree = clip.shape[0] / images_par_seconde
print(duree, "secondes")            # -> 4.0 secondes

# Le meme clip converti en nombres a virgule, ramenes entre 0 et 1.
clip_flottant = clip.astype(np.float32) / 255.0
print(clip_flottant.nbytes / 1e6, "Mo")   # -> 12.582912 Mo
```

Enfin, voici le changement d'ordre des axes dont nous parlions plus haut. C'est une opération que vous ferez des centaines de fois, alors autant la voir tout de suite.

PYTHON • Passer de l'ordre « affichage » à l'ordre « PyTorch »

```python
import numpy as np
import torch

clip = np.zeros((16, 256, 256, 3), dtype=np.uint8)   # (T, H, W, C)

x = torch.from_numpy(clip).float() / 255.0
x = x.permute(3, 0, 1, 2)                            # (C, T, H, W)

print(x.shape)   # -> torch.Size([3, 16, 256, 256])

# Aucun nombre n'a ete perdu : seul l'ordre des axes a change.
print(x.numel())  # -> 3145728
```

Piège à éviter

Remarquez la dernière ligne : le nombre de cases n'a pas bougé. Changer l'ordre des axes ne crée et ne détruit aucune donnée, cela réorganise seulement la façon de les parcourir. Si jamais ce nombre change après une telle opération, c'est que vous avez commis une erreur ailleurs — et cette vérification en une ligne vous fera gagner des heures.

À retenir

Une image est une grille de nombres ; la couleur ajoute une troisième dimension ; la vidéo en ajoute une quatrième, le temps. Tout le reste de ce livre travaille sur ces tenseurs-là, et sur rien d'autre. Quand vous serez perdu dans un chapitre plus difficile, revenez à cette question toute simple : « quelle est la forme de ce que je manipule ? » Elle remet presque toujours les idées en place.

Nous savons désormais sous quelle forme une image et une vidéo arrivent dans une machine. Mais nous n'avons encore rien appris à personne : nos tenseurs restent inertes. La question suivante est donc la bonne : qu'est-ce que cela veut dire, au juste, qu'une machine « apprend » ? C'est tout le sujet du chapitre suivant.

Exercices

Exercice 1 — Compter les cases

Combien de nombres contient une image en couleur de 512×512 pixels ? C'est la taille des images utilisées pendant la phase de refroidissement de V-JEPA 2.1.

Solution

On multiplie la hauteur, la largeur et le nombre de canaux : $512 \times 512 \times 3 = 262\,144 \times 3 = 786\,432$ nombres. C'est quatre fois plus qu'une image de 256×256, ce qui est normal puisque nous avons doublé les deux côtés.

Exercice 2 — La durée d'un clip

Un clip contient 32 images, prises à raison de 8 images par seconde. Combien de temps dure-t-il ? Et combien de temps durerait-il si les mêmes 32 images étaient prises à 4 images par seconde ?

Solution

Dans le premier cas, $32/8 = 4$ secondes. Dans le second, $32/4 = 8$ secondes. Le nombre d'images est le même, mais elles sont prélevées plus loin les unes des autres : le clip couvre donc une tranche de temps deux fois plus longue. C'est un réglage important, car il décide de ce que le modèle a le droit de voir.

Exercice 3 — Le poids d'un lot

Un lot contient 64 clips de 16 images en 256×256 couleur, convertis en nombres à virgule de quatre octets. Combien de méga-octets occupe ce lot ?

Solution

Un clip contient $16 \times 256 \times 256 \times 3 = 3\,145\,728$ nombres, donc $3\,145\,728 \times 4 = 12\,582\,912$ octets, soit environ 12,6 Mo. Pour 64 clips : $64 \times 12{,}6 \approx 806$ Mo, c'est-à-dire un peu plus de huit cents méga-octets. Ce lot fait exactement la moitié de celui de l'article, qui en contient 128, et l'on retrouve bien les 1,6 Go calculés plus haut en doublant ce résultat.

Exercice 4 — Lire une forme

Un collègue vous tend un tenseur de forme $(3, 64, 384, 384)$ et vous dit que c'est une vidéo. Que représente chaque nombre ? Quelle est la convention utilisée, et quelle serait la forme du même tenseur dans l'autre convention ?

Solution

Le 3 ne peut être que le nombre de canaux de couleur : aucune vidéo ne fait trois pixels de haut. Il s'agit donc de l'ordre (canaux, temps, hauteur, largeur), celui de PyTorch : une vidéo en couleur de 64 images en 384×384. Dans la convention d'affichage, le même tenseur s'écrirait $(64, 384, 384, 3)$. Les nombres qu'il contient sont exactement les mêmes ; seul leur rangement change.

CHAPITRE 2

Ce que veut dire « apprendre » pour une machine

Une image, nous venons de le voir, n'est rien d'autre qu'un grand tas de nombres. Mais un tas de nombres ne fait rien tout seul. Dans ce chapitre, nous allons voir comment une machine part d'un réglage pris au hasard, se trompe, remarque qu'elle s'est trompée, et se corrige toute seule. C'est cela, et rien d'autre, qu'on appelle « apprendre ».

Ce qu'il faut savoir avant

Le chapitre 1, et seulement lui. Il faut savoir qu'une image est une grille de nombres, et savoir ce qu'est la forme d'un tenseur. Pour le reste, il suffit de savoir additionner, multiplier, et diviser. Nous n'utiliserons aucune formule que nous n'aurons pas d'abord dessinée.

Quand on dit qu'une machine « apprend », on imagine souvent quelque chose de mystérieux, presque magique. La réalité est beaucoup plus simple, et beaucoup plus rassurante : une machine qui apprend, c'est une machine qui essaie, qui se trompe, qui regarde de combien elle s'est trompée, et qui recommence un tout petit peu mieux. Des milliers de fois. Des millions de fois.

Tout ce chapitre raconte cette boucle-là. Nous allons la suivre pas à pas sur un exemple minuscule, que nous calculerons entièrement à la main, avant de vérifier sur une vraie machine que nos calculs de papier tombent juste.

2.1 Une machine qui apprend, c'est une machine à boutons

En clair

Imaginez un appareil avec quelques boutons à tourner. Vous lui posez une question, il donne une réponse. Si la réponse est mauvaise, vous tournez un peu les boutons et vous redemandez. Apprendre, pour une machine, c'est exactement cela : trouver la bonne position des boutons.

Vous avez sûrement déjà réglé la température d'une douche. Il y a un robinet chaud et un robinet froid. Vous les tournez un peu, vous mettez la main sous l'eau, c'est trop froid, alors vous tournez encore. Au bout de quelques essais, l'eau est parfaite. Personne ne vous a donné la position exacte des robinets : vous l'avez trouvée en essayant et en corrigeant.

Une machine qui apprend fait la même chose, sauf qu'elle a beaucoup plus de robinets, et qu'elle ne trempe pas la main dans l'eau : elle calcule.

Définition 2.1 (Le modèle)

Un *modèle* est une machine à calculer qui prend quelque chose en entrée et rend une réponse en sortie. Ce qu'elle rend ne dépend pas seulement de l'entrée : cela dépend aussi de la position de ses boutons. Le même modèle, avec des boutons réglés autrement, donnera une autre réponse à la même question.

Définition 2.2 (Le paramètre, ou poids)

Un *paramètre* est un de ces boutons. C'est simplement un nombre, rangé dans la mémoire de la machine, que l'on a le droit de changer. On l'appelle aussi un *poids*, parce qu'il décide de l'importance que la machine donne à telle ou telle information. Régler un modèle, c'est choisir la valeur de tous ses paramètres.

La figure 2.1 montre cette idée dans son entier : une entrée qui arrive, des boutons qu'on peut tourner, une réponse qui sort.

Prenons tout de suite un exemple, et gardons-le jusqu'à la fin du chapitre.

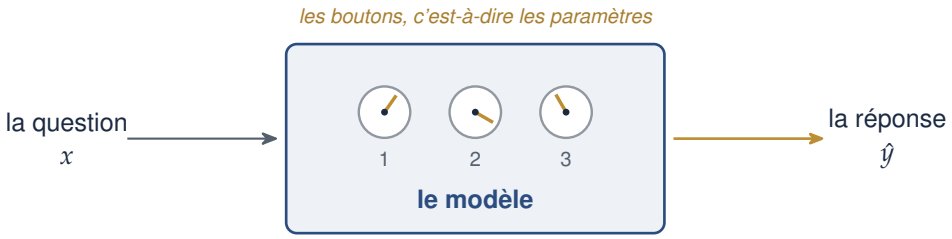


FIGURE 2.1. Un modèle, c'est une boîte à calculer munie de boutons. On lui donne une entrée par la gauche, elle rend une réponse par la droite. Entre les deux, ce sont les boutons — les paramètres — qui décident du résultat. Tourner les boutons change la réponse sans rien changer à la question.

Exemple 2.3 (La machine à bonbons)

Dans le hall d'une école, il y a une machine à bonbons. On y glisse des pièces, et elle rend des bonbons. Trois enfants ont essayé et ont noté ce qu'ils ont obtenu :

Enfant	Pièces glissées	Bonbons reçus
Léa	1	2
Sami	2	5
Nina	3	8

Nous aimerions une machine à calculer qui devine le nombre de bonbons à partir du nombre de pièces. Appelons x le nombre de pièces et $\hat{y}$ le nombre de bonbons deviné. Le modèle le plus simple du monde s'écrit

$$\hat{y} = a \times x + b.$$

Il a exactement deux boutons : a et b. Le bouton a dit combien de bonbons on gagne par pièce ; le bouton b ajoute ou enlève un nombre fixe de bonbons, quel que soit le nombre de pièces.

L'intuition

Le bouton a, c'est l'inclinaison de la droite : plus il est grand, plus la machine est généreuse à chaque pièce. Le bouton b, c'est le cadeau de

départ : il décale toute la réponse vers le haut ou vers le bas, même quand on n'a mis aucune pièce.

Vous avez peut-être déjà trouvé la règle de tête : trois bonbons par pièce, moins un bonbon que la machine garde toujours pour elle. Autrement dit $a = 3$ et $b = -1$. Vérifions sur les trois enfants : $3 \times 1 - 1 = 2$ pour Léa, $3 \times 2 - 1 = 5$ pour Sami, $3 \times 3 - 1 = 8$ pour Nina. Tout tombe juste.

Piège à éviter

Nous connaissons la bonne réponse, mais **la machine, elle, ne la connaît pas**. C'est tout l'intérêt de l'exercice : nous allons la laisser partir de boutons réglés à zéro, complètement à côté de la plaque, et la regarder retrouver toute seule le 3 et le -1. Connaître la réponse à l'avance nous sert seulement à vérifier qu'elle ne se trompe pas de chemin.

Définition 2.4 (La prédiction)

Une *prédiction* est la réponse que le modèle donne avec les boutons tels qu'ils sont réglés en ce moment. On la note $\hat{y}$, avec un petit chapeau, pour bien la distinguer de la vraie valeur y que l'on a observée dans la réalité.

Exemple 2.5 (Une première prédiction, très mauvaise)

Réglons les deux boutons à zéro : $a = 0$ et $b = 0$. Pour Léa, qui a mis une pièce :

$$\hat{y} = 0 \times 1 + 0 = 0.$$

La machine annonce zéro bonbon, alors que Léa en a reçu deux. Elle se trompe donc de deux bonbons. Pour Sami elle annonce encore zéro, alors qu'il en a reçu cinq. Et pour Nina, zéro contre huit. Notre modèle est pour l'instant parfaitement nul, et c'est normal : nous n'avons rien réglé du tout.

Nous avons une machine, des boutons, et des réponses fausses. Il nous manque quelque chose d'essentiel : un moyen de dire *à quel point* elles sont

fausses.

2.2 Se tromper, et mesurer de combien

En clair

Pour se corriger, il ne suffit pas de savoir qu'on s'est trompé. Il faut un nombre qui dise si l'on s'est trompé beaucoup ou un tout petit peu. Ce nombre s'appelle l'erreur.

Reprenons Léa. La machine a dit 0, la réalité était 2. L'écart entre les deux vaut $0 - 2 = -2$. Nous pourrions nous arrêter là et additionner les écarts des trois enfants. Mais un problème apparaît aussitôt.

Piège à éviter

Additionner les écarts tels quels ne marche pas. Imaginez une machine qui se trompe de $+5$ bonbons pour Léa et de -5 bonbons pour Sami. En additionnant, on obtient $5 + (-5) = 0$: on croirait que la machine est parfaite, alors qu'elle s'est lourdement trompée deux fois ! Les erreurs se sont annulées entre elles.

Il faut donc empêcher les écarts de se compenser. La solution retenue presque partout est de **mettre chaque écart au carré**, c'est-à-dire de le multiplier par lui-même. Un carré est toujours positif : $(+5)^2 = 25$ et $(-5)^2 = 25$ aussi. Plus rien ne s'annule.

L'intuition

Mettre au carré a un deuxième avantage, qui est même le principal : cela punit très fort les grosses erreurs. Se tromper de 1 coûte 1. Se tromper de 2 ne coûte pas 2 mais 4. Se tromper de 10 coûte 100. La machine va donc s'occuper en priorité de ses plus grosses bêtises, ce qui est exactement ce qu'on veut.

Définition 2.6 (L'erreur, ou perte)

L'*erreur* est un seul nombre qui résume à quel point le modèle se trompe sur l'ensemble des exemples. On l'appelle aussi la *perte*, et on la note souvent $\mathcal{L}$, de l'anglais *loss*. On la calcule en mettant chaque écart au carré, puis en prenant la moyenne :

$$\mathcal{L} = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2,$$

où N est le nombre d'exemples, $\hat{y}_i$ la réponse du modèle sur l'exemple numéro i, et y_i la vraie valeur. Une erreur de 0 veut dire que le modèle tombe juste partout. Plus le nombre est grand, plus le modèle est mauvais.

Notation 2.7 (Lire le signe somme)

Le grand symbole $\sum$ se lit « somme de ». L'écriture $\sum_{i=1}^{3} z_i$ veut simplement dire $z_1 + z_2 + z_3$: on fait prendre à i toutes les valeurs de 1 à 3, et on additionne tout. C'est une façon courte d'écrire une addition longue, rien de plus. Si l'addition ne comporte que trois termes, vous avez parfaitement le droit de les écrire à la main : c'est ce que nous ferons.

Exemple 2.8 (L'erreur de notre machine réglée à zéro)

Avec $a = 0$ et $b = 0$, les trois prédictions valent 0. Calculons les écarts, puis leurs carrés :

Enfant	x	y	$\hat{y}$	écart $\hat{y} - y$	carré
Léa	1	2	0	-2	4
Sami	2	5	0	-5	25
Nina	3	8	0	-8	64

La somme des carrés vaut $4 + 25 + 64 = 93$. Comme il y a trois exemples, la moyenne vaut

$$\mathcal{L} = \frac{93}{3} = 31.$$

Retenez ce nombre : **l'erreur de départ vaut exactement** 31. Nous allons passer le reste du chapitre à la faire descendre.

La figure 2.2 donne à voir ce que ces écarts représentent : ce sont des distances verticales entre les vrais points et la droite que dessine le modèle.

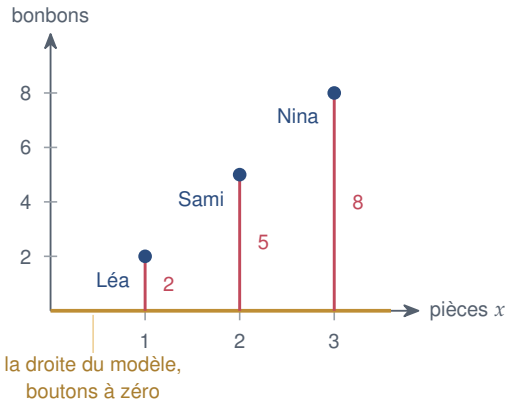


FIGURE 2.2. Les trois enfants sont les points bleus. La droite en trait plein est la réponse du modèle. Ici, ses deux boutons sont encore à zéro : elle est couchée sur l'axe et annonce zéro bonbon à tout le monde. Les traits rouges verticaux sont les écarts, c'est-à-dire de combien la machine s'est trompée sur chaque enfant : 2, 5 et 8 bonbons. Régler les boutons, ce sera faire pivoter et monter cette droite jusqu'à ce que les traits rouges soient les plus courts possible.

À retenir

Toute la suite du livre repose sur ce couple d'idées : un modèle donne une réponse, et une erreur mesure de combien cette réponse est fausse. Apprendre, ce sera toujours et uniquement : **faire descendre l'erreur en changeant les paramètres**.

Nous savons mesurer notre bêtise. Reste la question qui compte vraiment : dans quel sens faut-il tourner les boutons pour que ce nombre baisse ?

2.3 De quel côté tourner le bouton ? La pente

En clair

Pour savoir dans quel sens tourner un bouton, il suffit de le pousser un tout petit peu et de regarder ce que devient l'erreur. Si elle monte, on tournait du mauvais côté. Si elle descend, on continue.

C'est le jeu du « tu chauffes, tu refroidis ». On cache un objet dans une pièce, l'enfant fait un pas, et on lui dit s'il se rapproche ou s'il s'éloigne. Avec cette seule information, répétée assez souvent, il finit toujours par trouver.

Faisons-le pour de vrai sur notre machine à bonbons. Nos boutons sont à $a = 0$ et $b = 0$, et l'erreur vaut 31. Poussons le bouton a d'un tout petit peu, disons de 0,01, sans toucher à b.

Exemple 2.9 (Pousser un bouton et regarder)

Avec $a = 0{,}01$ et $b = 0$, les trois prédictions deviennent 0,01, 0,02 et 0,03. Les écarts valent donc $-1{,}99$, $-4{,}98$ et $-7{,}97$. Leurs carrés valent 3,9601, 24,8004 et 63,5209, dont la somme fait 92,2814. La nouvelle erreur vaut

$$\mathcal{L} = \frac{92{,}2814}{3} \approx 30{,}7605.$$

Elle est passée de 31 à 30,7605 : elle a **baissé** de 0,2395. Nous poussions donc dans le bon sens. Pour savoir à quelle vitesse elle baisse, on divise la baisse par la taille de la poussée :

$$\frac{-0{,}2395}{0{,}01} = -23{,}95.$$

Ce nombre, $-23{,}95$, dit deux choses à la fois. Son *signe* négatif dit : « en augmentant a, l'erreur diminue ». Sa *taille*, presque 24, dit : « et elle diminue vite ».

Définition 2.10 (La pente)

La *pente* par rapport à un bouton est le nombre qui dit de combien l'erreur change quand on pousse ce bouton d'une unité. Une pente

négative veut dire que l'erreur descend quand on augmente le bouton. Une pente positive veut dire qu'elle monte. Une pente proche de zéro veut dire que ce bouton ne change presque rien.

Définition 2.11 (Le gradient)

Le *gradient* est la liste de toutes les pentes, une par bouton. S'il y a deux boutons, le gradient contient deux nombres ; s'il y en a deux milliards, il en contient deux milliards. Le gradient est donc une sorte de boussole : il indique, pour chaque bouton à la fois, dans quel sens tourner.

Pousser chaque bouton à la main pour mesurer sa pente serait épouvantable : un modèle comme V-JEPA 2.1 a deux milliards de boutons, il faudrait deux milliards de petites poussées à chaque étape. Heureusement, les mathématiques savent calculer ces pentes directement, d'un coup, par une formule. Pour notre droite, les deux formules sont les suivantes :

Notation 2.12 (Les deux pentes de notre modèle)

En notant $e_i = \hat{y}_i - y_i$ l'écart sur l'exemple i :

$$\text{pente selon } a \;=\; \frac{2}{N}\sum_i e_i \times x_i, \qquad \text{pente selon } b \;=\; \frac{2}{N}\sum_i e_i.$$

Ces deux formules ont l'air de tomber du ciel. Elles ne sont pourtant pas mystérieuses, et il vaut la peine de comprendre ce qu'elles racontent.

L'intuition

Regardez la pente selon b : c'est simplement la moyenne des écarts, multipliée par deux. C'est logique, car le bouton b déplace toute la droite vers le haut ou vers le bas, de la même quantité pour tout le monde. Si en moyenne la machine annonce trop de bonbons, il faut baisser b.

La pente selon a est la même chose, mais chaque écart y est d'abord multiplié par x_i, le nombre de pièces. C'est l'effet du levier. Le bouton a fait pivoter la droite autour du point de départ : un enfant placé loin,

comme Nina avec ses trois pièces, monte ou descend trois fois plus qu'un enfant placé près, comme Léa. Son écart pèse donc trois fois plus lourd dans la décision.

Exemple 2.13 (La formule donne-t-elle le même résultat que la poussée ?)

Vérifions. Avec $a = 0$ et $b = 0$, les écarts valent -2, -5 et -8, et les pièces valent 1, 2 et 3. La pente selon a vaut donc

$$\frac{2}{3}\big[(-2)\times 1 + (-5)\times 2 + (-8)\times 3\big] = \frac{2}{3}\big[-2-10-24\big] = \frac{2}{3}\times(-36) = -24.$$

La formule annonce -24, et notre petite poussée avait mesuré $-23{,}95$. Les deux méthodes s'accordent. Le minuscule écart vient de ce que notre poussée de 0,01 n'était pas infiniment petite ; en poussant encore moins fort, on collerait encore mieux au -24.
Calculons aussi la pente selon b :

$$\frac{2}{3}\big[(-2)+(-5)+(-8)\big] = \frac{2}{3}\times(-15) = -10.$$

Les deux pentes sont négatives. La boussole nous dit donc : augmente a, et augmente b aussi. Voyons maintenant comment transformer ce conseil en mouvement.

2.4 La descente de gradient, calculée à la main

En clair

La règle tient en une phrase : on tourne chaque bouton dans le sens opposé à sa pente, d'une petite quantité. Puis on recommence.

Pourquoi le sens *opposé* ? Parce qu'une pente négative signifie « en augmentant, l'erreur descend ». Prendre l'opposé du négatif donne du positif : on augmente. Et une pente positive signifie « en augmentant, l'erreur monte » ; son opposé nous fait diminuer. Dans les deux cas, on va vers le bas. C'est

la même règle qu'une bille lâchée sur une colline : elle part toujours dans le sens contraire de la montée.

Définition 2.14 (La descente de gradient)

La *descente de gradient* est la méthode qui consiste à répéter l'opération suivante : calculer l'erreur, calculer le gradient, puis déplacer chaque paramètre d'un petit pas dans le sens opposé à sa pente. Pour un paramètre p quelconque, la mise à jour s'écrit

$$p_{\text{nouveau}} = p_{\text{ancien}} - \eta \times \text{pente selon } p,$$

où η est un petit nombre positif choisi par nous, qui règle la longueur du pas.

La figure 2.3 montre cette idée sous sa forme la plus parlante : l'erreur dessine une vallée, et le modèle est une bille qui la descend.

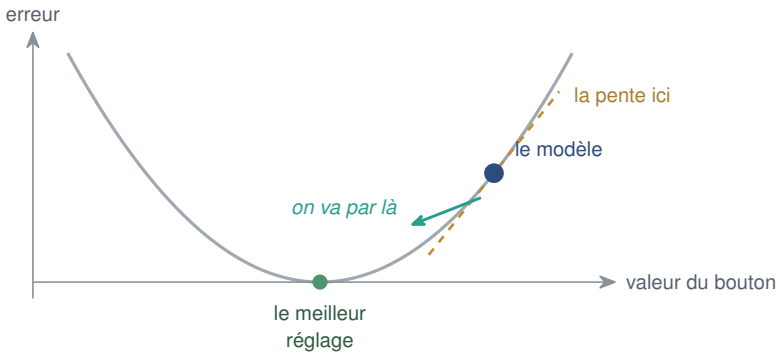


FIGURE 2.3. L'erreur vue comme une vallée. En allant vers la droite ou vers la gauche, on change la valeur d'un bouton ; la hauteur de la courbe donne l'erreur correspondante. La bille est notre modèle. La droite en pointillés est la pente à l'endroit où se trouve la bille : elle monte vers la droite, donc la pente est positive, donc il faut aller à gauche. Le meilleur réglage est tout en bas de la vallée, là où la courbe est plate.

Nous avons tout ce qu'il faut. Choisissons une longueur de pas de 0,1 et faisons deux étapes complètes, en écrivant chaque calcul.

Exemple 2.15 (Première étape, tout en détail)

Nous partons de $a = 0$ et $b = 0$. Nous avons déjà tout calculé : l'erreur vaut 31, la pente selon a vaut -24, la pente selon b vaut -10. Appliquons la règle avec un pas de 0,1 :

$$a_{\text{nouveau}} = 0 - 0{,}1 \times (-24) = 0 + 2{,}4 = 2{,}4,$$

$$b_{\text{nouveau}} = 0 - 0{,}1 \times (-10) = 0 + 1 = 1.$$

Nos deux boutons sont maintenant réglés à $a = 2{,}4$ et $b = 1$.

Exemple 2.16 (A-t-on vraiment progressé ?)

Vérifions en recalculant l'erreur avec les nouveaux boutons. Les trois prédictions valent :

$$2{,}4 \times 1 + 1 = 3{,}4; \quad 2{,}4 \times 2 + 1 = 5{,}8; \quad 2{,}4 \times 3 + 1 = 8{,}2.$$

Les vraies valeurs étaient 2, 5 et 8. Les écarts valent donc 1,4, 0,8 et 0,2, et leurs carrés 1,96, 0,64 et 0,04. La nouvelle erreur vaut

$$\mathcal{L} = \frac{1{,}96 + 0{,}64 + 0{,}04}{3} = \frac{2{,}64}{3} = 0{,}88.$$

Nous sommes passés de 31 à 0,88 en une seule étape. L'erreur a été divisée par plus de trente-cinq. La machine, qui annonçait zéro bonbon à tout le monde, en annonce maintenant trois et demi à Léa et huit à Nina : elle a déjà presque compris.

Exemple 2.17 (Deuxième étape)

Recommençons exactement le même travail depuis $a = 2{,}4$ et $b = 1$. Les écarts, que nous venons de calculer, valent 1,4, 0,8 et 0,2. Ils sont maintenant positifs : la machine annonce trop de bonbons.
Pente selon a :

$$\frac{2}{3}\big[1{,}4 \times 1 + 0{,}8 \times 2 + 0{,}2 \times 3\big] = \frac{2}{3}\big[1{,}4 + 1{,}6 + 0{,}6\big] = \frac{2}{3} \times 3{,}6 = 2{,}4.$$

Pente selon b :

$$\frac{2}{3}\big[1{,}4 + 0{,}8 + 0{,}2\big] = \frac{2}{3} \times 2{,}4 = 1{,}6.$$

Les deux pentes sont cette fois positives : il faut donc *baisser* les deux boutons.

$$a_{\text{nouveau}} = 2{,}4 - 0{,}1 \times 2{,}4 = 2{,}4 - 0{,}24 = 2{,}16,$$

$$b_{\text{nouveau}} = 1 - 0{,}1 \times 1{,}6 = 1 - 0{,}16 = 0{,}84.$$

Et la nouvelle erreur vaut 0,496, contre 0,88 juste avant : elle a encore baissé.

Récapitulons les deux étapes dans un seul tableau, que vous pourrez comparer tout à l'heure à la sortie du programme.

Étape	a	b	**erreur**	**pente selon** a	**pente selon** b
0	0	0	31	-24	-10
1	2,4	1	0,88	2,4	1,6
2	2,16	0,84	0,496	$-0{,}48$	0,32

Piège à éviter

Regardez la colonne du bouton a : il monte à 2,4, puis *redescend* à 2,16, alors que la bonne réponse est 3. On dirait qu'il s'éloigne ! Ne vous inquiétez pas, et surtout ne jugez jamais un entraînement en surveillant un paramètre isolé. Les boutons bougent ensemble, ils se gênent et se rattrapent l'un l'autre. Le seul chiffre qui compte est l'erreur, et elle, elle descend sans arrêt : 31, puis 0,88, puis 0,496. Au bout de deux cents étapes, a vaut 2,993 et b vaut $-0{,}985$: la machine a bien retrouvé le 3 et le -1 que nous cherchions.

À quoi ça sert en IA ?

Ce que nous venons de faire à la main sur deux boutons est *exactement* ce que fait l'entraînement de V-JEPA 2.1, sans un seul changement

d'idée. Les seules différences sont des différences de taille : deux milliards de boutons au lieu de deux, une erreur calculée sur des morceaux de vidéo au lieu de trois enfants, et cent trente-cinq mille étapes au lieu de deux. La boucle, elle, est identique : je prédis, je mesure mon erreur, je calcule mes pentes, je fais un petit pas, je recommence.

Il nous reste à parler du seul réglage que nous avons choisi arbitrairement, et qui est pourtant le plus délicat de tous : la longueur du pas.

2.5 La taille des pas : le taux d'apprentissage

Définition 2.18 (Le taux d'apprentissage)

Le *taux d'apprentissage*, noté η, est le petit nombre par lequel on multiplie la pente avant de bouger un paramètre. Il décide de la longueur des pas. C'est nous qui le choisissons : la machine ne l'apprend pas.

L'intuition

Vous descendez une colline dans le brouillard. Vous ne voyez que le sol sous vos pieds, donc vous ne connaissez que la pente à l'endroit où vous êtes. Avec des pas minuscules, vous descendrez, mais la nuit tombera avant que vous arriviez. Avec des pas de sept lieues, vous sauterez par-dessus la vallée et vous vous retrouverez plus haut qu'avant, sur l'autre versant. Il existe une bonne taille de pas entre les deux, et la trouver fait partie du métier.

La figure 2.4 met les trois situations côte à côte.

Ce ne sont pas des dessins pour faire joli : refaisons une seule étape de notre machine à bonbons avec trois pas différents, et regardons ce que devient l'erreur.

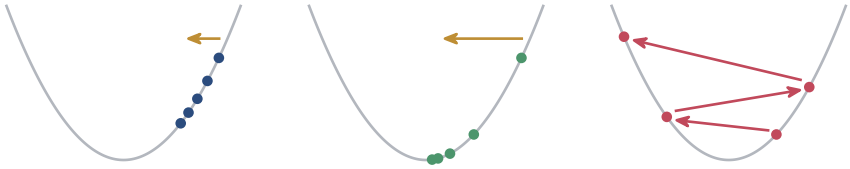


FIGURE 2.4. Trois façons de descendre la même vallée. À gauche, les pas sont trop courts : la bille avance à peine et il faudra une éternité. Au milieu, les pas sont bien choisis : la bille rejoint le fond en quelques sauts. À droite, les pas sont trop longs : la bille saute par-dessus le fond, remonte plus haut de l'autre côté, et s'échappe de la vallée au lieu d'y descendre.

Exemple 2.19 (Trois pas, trois destins)

Nous partons toujours de $a = 0$, $b = 0$, avec une erreur de 31 et des pentes de -24 et -10. Une seule étape suffit à tout révéler.

Pas η	nouveau a	nouveau b	erreur après une étape
0,001	0,024	0,01	30,33
0,1	2,4	1	**0,88**
1	24	10	3 103

Avec un pas de 0,001, l'erreur passe de 31 à 30,33 : elle a bougé d'à peine deux pour cent. À ce rythme, il faudrait des milliers d'étapes. Avec un pas de 0,1, elle tombe à 0,88 : c'est le bon réglage. Avec un pas de 1, elle grimpe à 3 103, soit cent fois pire qu'au départ ! Et si l'on s'obstine, l'étape suivante donne une erreur de 316 063. La machine ne se contente pas d'échouer : elle s'envole.

Piège à éviter

Un taux d'apprentissage trop grand est la panne numéro un des débutants, et elle est facile à reconnaître : au lieu de descendre doucement, l'erreur grossit d'étape en étape, jusqu'à afficher des nombres absurdes, voire le mot `nan`, qui veut dire « ce n'est plus un nombre du tout ». Le

réflexe correct, dans ce cas, est presque toujours le même : diviser le taux d'apprentissage par dix et relancer.

Sur des données réelles

V-JEPA 2.1 ne garde pas le même pas du début à la fin. Pendant les douze mille premières étapes, il l'augmente doucement de 10^{-4} à $5{,}25 \times 10^{-4}$: on appelle cela une montée en douceur, et cela évite de tout casser au tout début, quand les boutons sont encore réglés n'importe comment. Ensuite il le laisse constant très longtemps. Enfin, sur les douze mille dernières étapes, il le fait redescendre jusqu'à 10^{-6}, c'est-à-dire presque zéro : les pas deviennent minuscules et le modèle se pose tout doucement au fond de la vallée, au lieu de continuer à sautiller autour. Cette dernière phase porte un nom que nous retrouverons souvent : la phase de refroidissement.

Nous savons faire un pas, et le faire de la bonne longueur. Il reste à savoir sur combien de données on le calcule, et combien de fois on recommence.

2.6 Le rythme de l'entraînement : le lot et l'époque

Jusqu'ici, nous avions trois enfants, et nous avons calculé l'erreur sur les trois à chaque étape. Avec trois exemples, c'est facile. Avec cent quarante-deux millions d'images, c'est impossible : regarder toute la collection avant de bouger un seul bouton prendrait des jours pour une seule étape.

En clair

La solution est simple et un peu paresseuse : au lieu de regarder toutes les données, on en prend une petite poignée au hasard, on calcule l'erreur sur cette poignée-là, et on fait quand même un pas. La direction est un peu approximative, mais on avance beaucoup plus souvent — et au bout du compte, on va bien plus vite.

Définition 2.20 (Le lot)

Un *lot* est la petite poignée d'exemples que l'on regarde ensemble pour calculer une erreur et faire un pas. En anglais on dit *batch*, et ce mot revient partout dans le code. Le nombre d'exemples dans la poignée s'appelle la *taille de lot*.

Définition 2.21 (Le pas, ou itération)

Un *pas* — on dit aussi une *itération* — est un tour complet de la boucle : on prend un lot, on prédit, on mesure l'erreur, on calcule les pentes, on bouge les paramètres. Un pas, un lot.

Piège à éviter

Le mot « pas » sert malheureusement à deux choses, et il faut les séparer dans votre tête une bonne fois pour toutes. *Faire un pas*, c'est faire un tour de boucle : c'est ce que nous venons de définir, et cela se compte (135 000 pas pour V-JEPA 2.1). *La longueur du pas*, c'est le taux d'apprentissage η de la section précédente : c'est de combien on bouge les boutons pendant ce tour, et cela se règle. On peut donc faire beaucoup de pas très courts, ou peu de pas très longs. Quand un texte parle de « pas », demandez-vous toujours s'il compte des tours ou s'il mesure une longueur.

Définition 2.22 (L'époque)

Une *époque* est le temps qu'il faut pour que le modèle ait vu une fois chacune des données disponibles. Si la collection contient mille images et que les lots en contiennent cent, alors une époque vaut dix pas.

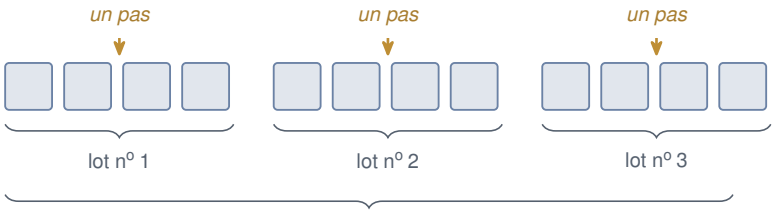


FIGURE 2.5. Toutes les données de la collection, découpées en lots. Le modèle en prend un, fait un pas, en prend un autre, refait un pas. Quand il a épuisé tous les lots, il a vu chaque donnée une fois : cela s'appelle une époque, et il peut alors recommencer un nouveau tour.

Exemple 2.23 (Compter des époques sur un petit jeu)

Vous disposez de 1 000 photos et vous choisissez des lots de 100 photos. Chaque pas consomme 100 photos, il faut donc

$$\frac{1\,000}{100} = 10 \text{ pas pour faire une époque.}$$

Si vous entraînez pendant 50 époques, votre programme aura effectué $50 \times 10 = 500$ pas au total, et chaque photo aura été vue 50 fois.

Sur des données réelles

Faisons le même calcul pour V-JEPA 2.1, et le résultat va sans doute vous surprendre. Le modèle fait 135 000 pas dans sa phase principale, et chaque lot contient 2 304 images. Il a donc regardé

$$135\,000 \times 2\,304 = 311\,040\,000 \text{ images en tout,}$$

soit trois cent onze millions. Or sa collection d'images en compte 142 millions. Le nombre d'époques vaut donc

$$\frac{311\,040\,000}{142\,000\,000} \approx 2{,}2.$$

Chaque image n'est vue qu'un peu plus de deux fois dans toute la vie du modèle. Du côté des vidéos, c'est encore plus frappant : 135 000 pas

avec 128 clips par lot font 17 280 000 clips, pour une collection de 19 millions de vidéos. Le modèle ne voit donc même pas une fois chaque vidéo !
C'est pour cela que les articles de recherche modernes ne parlent presque plus d'époques, mais comptent en pas. Quand les données sont quasi inépuisables, repasser deux fois sur la même image n'a plus grand intérêt : mieux vaut en voir une nouvelle.

Pour aller plus loin

Prendre un lot au hasard plutôt que toutes les données rend le gradient un peu bruité : la direction indiquée n'est pas tout à fait la bonne, elle tremble. On appelle cela la descente de gradient *stochastique*, un mot savant qui veut simplement dire « au hasard ». Ce tremblement, contre toute attente, aide souvent : il empêche la bille de rester coincée dans un petit creux sans intérêt et lui fait trouver de meilleures vallées.

Nous savons entraîner. Il reste un dernier danger, et c'est le plus sournois de tous : une machine peut très bien faire baisser son erreur tout en n'ayant strictement rien compris.

2.7 Apprendre par cœur n'est pas apprendre

En clair

Un élève peut apprendre par cœur les réponses des exercices du livre sans comprendre la leçon. Tant qu'on l'interroge sur ces exercices-là, il a tout juste. Le jour où on lui pose une question nouvelle, il s'effondre. Une machine peut faire exactement pareil.

Revenons à nos trois enfants. Notre droite n'a que deux boutons : elle est trop simple pour tricher, elle est bien obligée de trouver une règle générale. Mais donnons à un modèle beaucoup, beaucoup de boutons, et il pourra faire passer une courbe tordue exactement par chacun des trois points. Son erreur tombera à zéro. Et pourtant, si l'on glisse quatre pièces dans la machine, ce modèle-là annoncera n'importe quoi.

Définition 2.24 (Le sur-apprentissage)

Le *sur-apprentissage* est ce qui arrive quand un modèle apprend par cœur les exemples qu'on lui a montrés, y compris leurs petits défauts, au lieu d'apprendre la règle générale qui se cache derrière. Il est alors excellent sur ce qu'il a déjà vu, et mauvais sur tout le reste.

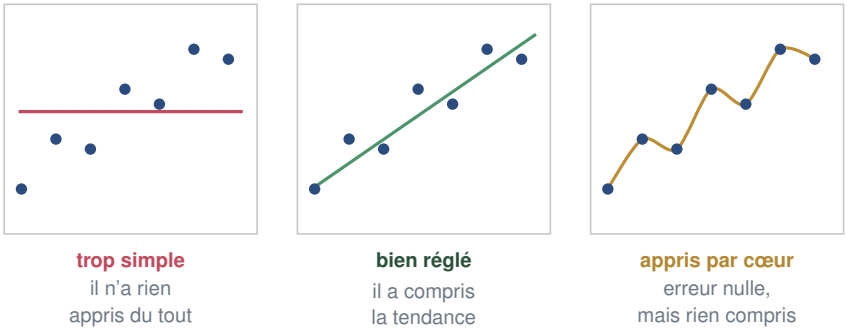


FIGURE 2.6. Les mêmes points, expliqués de trois façons. À gauche, le modèle est trop simple : il ne suit même pas la tendance, il passe à côté de tout le monde. Au milieu, il a saisi la règle générale : il ne touche pas tous les points, et c'est très bien ainsi. À droite, il passe exactement par chaque point, son erreur est nulle — mais entre les points il fait n'importe quoi, et sur un point nouveau il se trompera lourdement.

Comment se protéger de ce piège ? En trichant le moins possible : on cache au modèle une partie des données, et on ne les lui montre jamais pendant l'entraînement. Elles serviront uniquement à le juger.

Définition 2.25 (Jeu d'entraînement et jeu de validation)

Le *jeu d'entraînement* rassemble les données que le modèle a le droit de voir pour régler ses boutons. Le *jeu de validation*, parfois appelé jeu de test, rassemble des données qu'il ne voit jamais pendant l'entraînement, et qui servent uniquement à mesurer honnêtement ce qu'il vaut. C'est la différence entre les exercices du cahier et l'examen de fin d'année.

Piège à éviter

La faute la plus grave, dans tout l'apprentissage automatique, est de

régler quoi que ce soit en regardant le jeu de validation. Si vous essayez trente taux d'apprentissage et que vous gardez celui qui donne le meilleur score de validation, alors ce score n'est plus honnête : vous avez, sans le vouloir, appris par cœur votre examen. Les vrais résultats se mesurent toujours sur des données que rien ni personne n'a servi à choisir.

À quoi ça sert en IA ?

L'article de V-JEPA 2.1 prend cette précaution très au sérieux, et cela se voit dans ses annexes. Pour le suivi d'objets dans les vidéos, par exemple, les auteurs cherchent les meilleurs réglages sur la partie entraînement du jeu DAVIS, puis appliquent tels quels ces réglages à tous les jeux d'évaluation, sans plus jamais y toucher. C'est un détail d'annexe, et pourtant c'est ce détail qui rend leurs chiffres croyables.

Nous avons maintenant tout le vocabulaire. Le moment est venu de vérifier que nos calculs de papier tiennent devant une vraie machine.

2.8 Passons au code

Nous allons refaire, en Python, les deux étapes que nous venons de calculer à la main. Nous utiliserons `torch`, la bibliothèque PyTorch, qui sait faire deux choses très utiles : manipuler des tenseurs, et surtout calculer les pentes toute seule. Nous n'aurons donc plus jamais à écrire les formules du gradient nous-mêmes.

Commençons par le plus important : vérifier que la petite poussée du début du chapitre donne bien ce que nous avions trouvé.

PYTHON • Mesurer une pente en poussant un bouton

```python
import torch

# Les trois enfants : pieces glissees et bonbons recus.
x = torch.tensor([1.0, 2.0, 3.0])
y = torch.tensor([2.0, 5.0, 8.0])

```

```
def erreur(a, b):
    prediction = a * x + b
    return ((prediction - y) ** 2).mean().item()

avant = erreur(0.00, 0.0)   # les boutons a zero
apres = erreur(0.01, 0.0)   # on pousse le bouton a de 0,01

print(f"erreur avant la poussee : {avant:.4f}")   # -> 31.0000
print(f"erreur apres la poussee : {apres:.4f}")   # -> 30.7605
print(f"pente mesuree           : {(apres - avant) / 0.01:.2f}")
↪  # -> -23.95
```

Les trois nombres affichés sont exactement ceux de notre exemple : 31, 30,7605 et −23,95. Le calcul à la main et la machine sont d'accord.

Voici maintenant la boucle d'entraînement complète. Elle tient en une quinzaine de lignes, et c'est littéralement la même boucle que celle qui entraîne V-JEPA 2.1 : seules les tailles changent.

</> PYTHON • Deux étapes de descente de gradient

```
import torch

x = torch.tensor([1.0, 2.0, 3.0])
y = torch.tensor([2.0, 5.0, 8.0])

# Nos deux boutons. « requires_grad » dit a PyTorch :
# surveille ce nombre, je te demanderai sa pente.
a = torch.tensor(0.0, requires_grad=True)
b = torch.tensor(0.0, requires_grad=True)
pas = 0.1

# 1. je prends mes donnees : ici, les trois enfants d'un seul
↪  coup.
for etape in range(2):
    prediction = a * x + b                      # 2. je calcule
    ↪  ma prediction
    erreur = ((prediction - y) ** 2).mean()     # 3. je mesure
    ↪  mon erreur
    erreur.backward()                           # 4. je calcule
    ↪  les pentes

```

```
    print(f"etape {etape} : a = {a.item():.2f}, b =
    ↪ {b.item():.2f}, "
          f"erreur = {erreur.item():.4f}, "
          f"pente_a = {a.grad.item():.2f}, pente_b =
          ↪ {b.grad.item():.2f}")

    with torch.no_grad():                          # 5. je bouge
    ↪ les boutons
        a -= pas * a.grad
        b -= pas * b.grad

    a.grad.zero_()          # puis j'efface les pentes pour le
    ↪ tour suivant
    b.grad.zero_()

print(f"apres 2 etapes : a = {a.item():.2f}, b = {b.item():.2f}")
```

Ce programme affiche exactement ceci :

TEXT • Ce que le programme écrit à l'écran

```
etape 0 : a = 0.00, b = 0.00, erreur = 31.0000, pente_a = -24.00,
↪ pente_b = -10.00
etape 1 : a = 2.40, b = 1.00, erreur = 0.8800, pente_a = 2.40,
↪ pente_b = 1.60
apres 2 etapes : a = 2.16, b = 0.84
```

Comparez ligne à ligne avec le tableau de la section précédente : le 31, le -24, le -10, le 2,4, le 0,88, le 2,16 et le 0,84 y sont tous. Rien n'a été caché, et la machine n'a rien fait de plus que ce que nous avons fait au crayon.

Piège à éviter

La ligne `a.grad.zero_()` paraît anodine, et pourtant l'oublier est l'une des erreurs les plus courantes chez les débutants. PyTorch *additionne* les pentes qu'il calcule au lieu de les remplacer. Si vous n'effacez pas, l'étape suivante utilisera la somme de deux pentes, puis de trois, et vos pas grandiront tout seuls jusqu'à faire exploser l'entraînement. Le symptôme est le même que celui d'un taux d'apprentissage trop grand, ce qui rend la panne difficile à diagnostiquer.

Laissons enfin tourner la même boucle bien plus longtemps, pour voir la machine retrouver la règle des bonbons.

PYTHON • Deux cents étapes, et la règle apparaît

```python
import torch

x = torch.tensor([1.0, 2.0, 3.0])
y = torch.tensor([2.0, 5.0, 8.0])
a = torch.tensor(0.0, requires_grad=True)
b = torch.tensor(0.0, requires_grad=True)

for _ in range(200):
    erreur = ((a * x + b - y) ** 2).mean()
    erreur.backward()
    with torch.no_grad():
        a -= 0.1 * a.grad
        b -= 0.1 * b.grad
    a.grad.zero_()
    b.grad.zero_()

print(f"a = {a.item():.3f}   b = {b.item():.3f}")   # -> a =  2.993   b = -0.985
print(f"erreur = {erreur.item():.6f}")              # -> erreur =  0.000034
```

Le bouton a vaut 2,993 et le bouton b vaut $-0{,}985$. La machine a retrouvé, toute seule, la règle que nous avions devinée de tête au début du chapitre : trois bonbons par pièce, moins un pour la machine. Personne ne lui a jamais soufflé la réponse. Elle a seulement descendu la vallée, deux cents fois de suite.

À retenir

Retenez cette boucle en cinq gestes, car c'est la seule qu'il y ait dans tout l'apprentissage automatique, du plus petit exemple au plus gros modèle du monde :

1. je prends un lot de données ;
2. je calcule ma prédiction ;
3. je mesure mon erreur ;

4. je calcule les pentes ;
5. je bouge chaque bouton d'un petit pas dans le sens opposé à sa pente.

Puis je recommence. Cent trente-cinq mille fois, dans le cas de V-JEPA 2.1.

Il manque pourtant quelque chose d'énorme à tout ce que nous venons de faire, et vous l'avez peut-être déjà remarqué. À chaque étape, nous avions besoin de connaître la bonne réponse : les deux bonbons de Léa, les cinq de Sami, les huit de Nina. Sans elles, impossible de calculer le moindre écart, donc impossible de calculer la moindre erreur, donc impossible d'apprendre quoi que ce soit.

Mais qui va écrire la bonne réponse à côté d'un milliard de vidéos ? Personne. Aucune équipe au monde, aucun budget, aucune quantité de patience ne suffirait. C'est précisément le mur contre lequel s'arrête tout ce que nous venons d'apprendre — et c'est le mur que le chapitre suivant va traverser.

Exercices

Exercice 1 — Calculer une erreur

Un modèle prédit $\hat{y} = 4$, $\hat{y} = 7$ et $\hat{y} = 9$, alors que les vraies valeurs sont 3, 9 et 9. Quelle est l'erreur, au sens de la moyenne des carrés des écarts ?

Solution

Les écarts valent $4 - 3 = 1$, puis $7 - 9 = -2$, puis $9 - 9 = 0$. Leurs carrés valent 1, 4 et 0, dont la somme fait 5. Comme il y a trois exemples,

$$\mathcal{L} = \frac{5}{3} \approx 1{,}67.$$

Remarquez que le deuxième écart, bien que négatif, contribue positivement à l'erreur : c'est tout l'intérêt du carré.

Exercice 2 — Faire un pas

Un paramètre vaut actuellement $p = 5$. On calcule sa pente et l'on trouve $+3$. Le taux d'apprentissage vaut 0,2. Que vaut le paramètre après la mise à jour ? A-t-il augmenté ou diminué, et pourquoi était-ce le bon sens ?

Solution

On applique la règle $p_{\text{nouveau}} = p_{\text{ancien}} - \eta \times \text{pente}$, soit

$$p_{\text{nouveau}} = 5 - 0{,}2 \times 3 = 5 - 0{,}6 = 4{,}4.$$

Le paramètre a diminué. C'était le bon sens, car une pente positive signifie « si j'augmente ce bouton, l'erreur monte ». Pour faire descendre l'erreur, il fallait donc baisser le bouton.

Exercice 3 — Compter des époques

Un jeu de données contient 50 000 images et l'on choisit des lots de 250 images. Combien de pas faut-il pour une époque ? Et si l'on entraîne pendant 30 000 pas, combien d'époques cela représente-t-il ?

Solution

Une époque demande $50\,000/250 = 200$ pas. Avec 30 000 pas au total, on effectue

$$\frac{30\,000}{200} = 150 \text{ époques},$$

et chaque image aura donc été vue 150 fois. C'est un régime très différent de celui de V-JEPA 2.1, qui ne dépasse pas deux époques et demie : quand les données sont peu nombreuses, on repasse dessus souvent, et le risque de sur-apprentissage augmente d'autant.

Exercice 4 — Diagnostiquer une panne

Vous lancez un entraînement et vous voyez défiler ces erreurs : 18,4 puis 91,7 puis 2 340 puis `nan`. Que se passe-t-il très probablement, et quel est

le premier réglage à corriger ?

Solution

L'erreur ne descend pas, elle explose : le modèle saute par-dessus le fond de la vallée et remonte de plus en plus haut à chaque étape, jusqu'à sortir des nombres représentables par la machine, ce qui produit le `nan`. C'est la signature d'un taux d'apprentissage trop grand. Le premier geste est de le diviser par dix et de relancer ; si le problème persiste, on divise encore par dix. Attention toutefois : oublier d'effacer les pentes avec `zero_()` produit exactement le même symptôme, et vaut la peine d'être vérifié dans la foulée.

Exercice 5 — Le piège de l'examen

Vous disposez d'un jeu d'entraînement et d'un jeu de validation. Vous essayez cinquante taux d'apprentissage différents et vous annoncez, comme résultat final de votre travail, le meilleur score obtenu sur le jeu de validation. Pourquoi ce chiffre est-il trompeur, et que fallait-il faire ?

Solution

En choisissant le réglage qui donne le meilleur score de validation, vous avez utilisé le jeu de validation pour prendre une décision : il a donc servi à l'entraînement, au sens large. Parmi cinquante essais, le meilleur score est en partie dû à la chance, et il sera plus flatteur que la réalité. La bonne pratique consiste à garder un troisième jeu de données, mis de côté dès le départ, que l'on ne regarde qu'une seule et unique fois, tout à la fin, pour annoncer le résultat.

CHAPITRE 3

Apprendre sans étiquettes : l'auto-supervision

Au chapitre précédent, la machine avait toujours la bonne réponse écrite à côté de la question. Nous allons voir maintenant que, pour les vidéos, cette bonne réponse n'existe pas et n'existera jamais. Nous allons donc apprendre à la machine un nouveau jeu : cacher elle-même un morceau de ce qu'elle regarde, et essayer de le deviner. Personne ne lui souffle rien. Et pourtant, elle apprend.

> **Ce qu'il faut savoir avant**
>
> Les chapitres 1 et 2. Il faut savoir qu'une image est une grille de nombres et connaître sa forme, et il faut avoir compris la boucle en cinq gestes : je prédis, je mesure mon erreur, je calcule les pentes, je bouge les boutons, je recommence. Nous n'aurons besoin de rien d'autre. Aucun mot nouveau ne sera employé sans être expliqué juste avant.

Reprenons la machine à bonbons du chapitre précédent. Pour qu'elle apprenne, il nous fallait trois choses : les questions (le nombre de pièces), les réponses (le nombre de bonbons), et la boucle qui corrige les boutons. Sans les réponses, la deuxième étape de la boucle s'arrête net : impossible de mesurer un écart, donc impossible de savoir dans quel sens tourner quoi que ce soit.

Tout ce chapitre part de cette question, et d'elle seule : *et si personne n'écrit les réponses ?*

3.1 La bonne réponse écrite à côté : l'étiquette

En clair

Une étiquette, c'est le petit mot qu'un humain colle sur une donnée pour dire ce qu'elle contient. « Ceci est un chat. » Sans ce petit mot, la méthode du chapitre précédent ne fonctionne pas.

Quand nous avons entraîné la machine à bonbons, nous connaissions les trois réponses : deux bonbons pour Léa, cinq pour Sami, huit pour Nina. Ces trois nombres n'étaient pas tombés du ciel. Trois enfants les avaient comptés, et quelqu'un les avait notés sur un papier. Ce travail de notation a un nom.

Définition 3.1 (L'étiquette)

Une *étiquette* est la bonne réponse que l'on écrit à côté d'une donnée, pour dire à la machine ce qu'elle aurait dû répondre. Sur une photo, ce peut être le mot « chat ». Sur un enregistrement de voix, ce peut être la phrase prononcée. Sur une image médicale, ce peut être « sain » ou « malade ». Une étiquette est toujours écrite par quelqu'un : elle ne se fabrique pas toute seule.

Définition 3.2 (L'apprentissage supervisé)

On parle d'*apprentissage supervisé* quand on entraîne une machine avec des données accompagnées de leurs étiquettes. Le mot « supervisé » veut dire « surveillé » : à chaque essai, quelqu'un — au travers de l'étiquette qu'il a écrite à l'avance — corrige la machine. C'est exactement ce que nous avons fait au chapitre 2.

Exemple 3.3 (Reconnaître un chat d'un chien)

Vous voulez une machine qui, devant une photo, dise « chat » ou « chien ». Vous rassemblez dix mille photos. Puis, une par une, vous les ouvrez, vous regardez, et vous tapez le bon mot à côté. Dix mille fois. Quand c'est fini, vous avez dix mille étiquettes, et vous pouvez lancer la boucle du chapitre précédent : la machine prédit, on compare sa prédiction à

l'étiquette, on mesure l'écart, on corrige les boutons.
Retenez bien le prix payé ici : *dix mille regards humains*. C'est ce prix-là que nous allons regarder de plus près.

Tant qu'on parle de dix mille photos, tout va bien. Une petite équipe y passe quelques semaines et le travail est fait. Voyons ce qui se passe quand on change d'échelle.

3.2 Le mur : ce que coûte vraiment une étiquette

En clair

Étiqueter est un travail humain. Un travail humain prend du temps. Et quand le nombre de données devient très grand, ce temps devient si long qu'aucune entreprise, aucun pays, aucune génération ne pourrait le fournir.

Faisons le calcul ensemble, avec des nombres simples et vérifiables.

Imaginons la tâche la plus facile qui soit : regarder une photo et taper un seul mot. Une personne rapide met environ cinq secondes par photo, en comptant le temps de regarder et le temps de taper. Maintenant, comptons le temps de travail d'une personne dans une année : disons sept heures par jour, deux cent vingt jours par an — les week-ends et les congés en moins. Cela fait $7 \times 220 = 1\,540$ heures de travail par an.

Exemple 3.4 (Étiqueter cent quarante-deux millions d'images)

V-JEPA 2.1 s'entraîne, entre autres, sur un ensemble de 142 000 000 d'images. Étiquetons-les à cinq secondes pièce :

$$142\,000\,000 \times 5 = 710\,000\,000 \text{ secondes.}$$

Convertissons en heures en divisant par 3 600 :

$$\frac{710\,000\,000}{3\,600} \approx 197\,000 \text{ heures.}$$

Puis en années de travail d'une personne, en divisant par 1 540 :

$$\frac{197\,000}{1\,540} \approx 128 \text{ ans.}$$

Cent vingt-huit ans de travail, pour une seule personne, sans jamais s'arrêter. Et c'était la tâche la plus *facile* : un mot par image.

Et les vidéos ? La plus grosse source de V-JEPA 2.1 représente à elle seule 1 600 000 heures de film. Notez bien : il ne s'agit plus du temps qu'il faut pour étiqueter, mais du temps qu'il faudrait rien que pour *regarder* ces vidéos une fois, à vitesse normale.

$$\frac{1\,600\,000}{1\,540} \approx 1\,039 \text{ ans.}$$

Plus de mille ans de visionnage. Il faudrait avoir commencé à regarder à l'époque des premiers châteaux forts pour finir aujourd'hui — et on n'aurait encore rien écrit.

Sur des données réelles

Le mur devient encore bien plus haut dès qu'on demande une étiquette plus fine qu'un simple mot. Pour apprendre à une machine à dire, *pour chaque pixel*, s'il appartient à la route, au trottoir ou à un piéton, il faut colorier l'image à la main, contour par contour. Sur des photos de rue, ce travail prend de l'ordre d'une heure et demie par image.
Sur nos cent quarante-deux millions d'images, cela ferait $142\,000\,000 \times 1{,}5 = 213\,000\,000$ heures, soit environ $213\,000\,000 / 1\,540 \approx 138\,000$ ans. Cent trente-huit mille ans de coloriage. Autant dire : jamais.

La figure 3.1 met ces trois durées côte à côte. Le troisième bâton ne tient pas sur la page, et c'est bien le message du dessin.

Vous voyez le problème. Nous savons faire apprendre une machine, mais notre méthode réclame un carburant — les étiquettes — que nous ne pouvons pas produire en quantité suffisante. Or les données brutes, elles, sont partout : des milliards de photos et de vidéos existent déjà, sans le moindre petit mot écrit à côté.

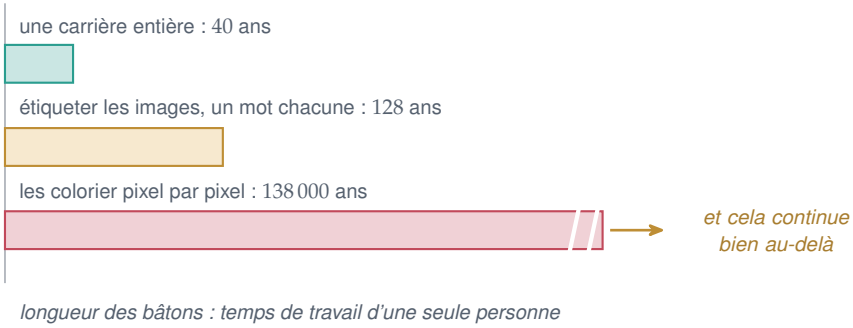


FIGURE 3.1. Le temps de travail humain qu'il faudrait pour étiqueter les données de V-JEPA 2.1, comparé à une carrière entière. Le premier bâton est tout ce qu'une personne peut fournir dans sa vie. Le deuxième est le prix d'un simple mot par image. Le troisième, celui du coloriage pixel par pixel, sort de la page : il faudrait mille fois plus de temps que ce que la feuille peut montrer.

> **À retenir**
>
> Les données brutes sont abondantes et presque gratuites. Les étiquettes sont rares et très chères. Toute la suite du livre découle de ce déséquilibre : il faut trouver un moyen d'apprendre à partir de ce qui est abondant.

La question devient donc : peut-on fabriquer la bonne réponse *sans personne* ? La réponse est oui, et elle est étonnamment simple.

3.3 L'idée qui change tout : cacher un morceau

> **L'intuition**
>
> Si je cache un bout de quelque chose que j'ai sous les yeux, je connais déjà la bonne réponse : c'est le bout que je viens de cacher. Je peux donc me poser une question à moi-même, et me corriger tout seul. Aucun humain n'est nécessaire.

Faisons l'expérience sur vous, tout de suite. Lisez cette phrase :

« Le chat boit du _____. »

Vous avez pensé « lait ». Personne ne vous a donné la réponse. Vous l'avez trouvée parce que vous connaissez les chats, les boissons, et la façon dont les phrases sont construites. Autrement dit : *pour remplir le trou, il a fallu comprendre le reste*. C'est toute l'idée.

Maintenant, regardez ce que cela donne si l'on cache un morceau d'une image. Si je masque le museau d'un chien sur une photo, vous pouvez encore le dessiner de tête, à peu près : vous savez où il va, de quelle couleur il est, quelle forme il a. Pour cela, vous vous êtes servi des oreilles, des yeux, de la fourrure autour. Vous avez compris la photo.

Définition 3.5 (L'apprentissage auto-supervisé)

On parle d'*apprentissage auto-supervisé* quand la machine fabrique elle-même ses questions et ses réponses à partir des données brutes, sans aucune étiquette écrite par un humain. Le préfixe « auto » veut dire « par elle-même » : c'est elle qui se surveille. On l'abrège souvent en *SSL*, initiales de l'anglais *self-supervised learning*.

La figure 3.2 met les deux façons de faire côte à côte. Regardez surtout la différence de rythme entre la colonne de gauche et celle de droite.

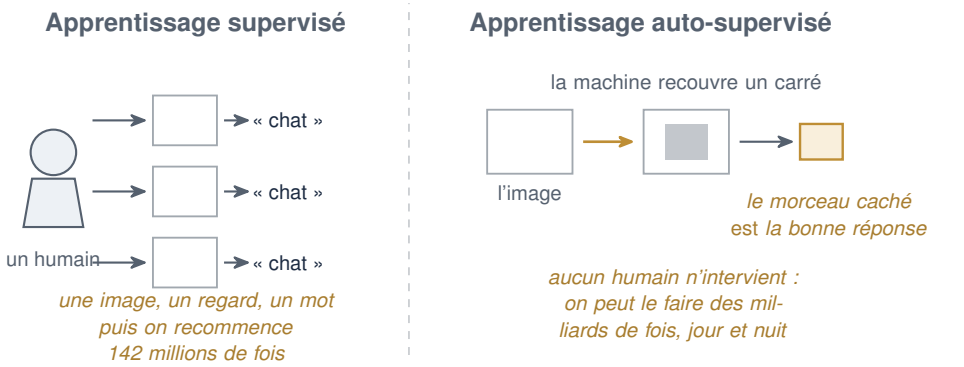


FIGURE 3.2. Deux façons d'obtenir la bonne réponse. À gauche, l'apprentissage supervisé : un humain regarde chaque image et écrit une étiquette, une par une. À droite, l'apprentissage auto-supervisé : la machine recouvre elle-même un morceau de l'image, et le morceau caché devient la bonne réponse. À gauche, il faut un humain à chaque image ; à droite, personne.

Ce petit changement de point de vue a une conséquence énorme, et il faut prendre le temps de la mesurer. Dans la colonne de gauche, le nombre

d'exemples d'entraînement est limité par le nombre d'heures que des humains acceptent d'y passer. Dans la colonne de droite, il est limité par le nombre de vidéos qui existent et par la puissance des machines. Ce ne sont pas du tout les mêmes limites.

Exemple 3.6 (Le même carré, deux rôles)

Prenons une image de 8×8 cases. Je recouvre les seize cases du coin en haut à droite. Ce que je donne à la machine, c'est l'image avec son trou. Ce que je lui demande, c'est de reconstituer les seize cases manquantes. Et comme je viens de les recouvrir moi-même, je les connais parfaitement : je peux donc comparer sa réponse à la vérité et calculer une erreur, exactement comme au chapitre 2.
Le même carré joue donc deux rôles à la fois : il est *la question* quand on le cache, et *la réponse* quand on le compare. Voilà pourquoi on n'a plus besoin de personne.

Ce jeu de cache-cache porte un nom, et il faut le connaître, car nous allons l'employer jusqu'à la fin du livre.

3.4 La tâche prétexte : un jeu que l'on jette après

Définition 3.7 (La tâche prétexte)

Une *tâche prétexte* est un exercice que l'on invente pour faire travailler la machine, alors que la réponse à cet exercice ne nous intéresse pas du tout. Le mot « prétexte » a ici son sens courant : une bonne excuse. Ce qui nous intéresse, ce n'est pas que la machine sache remplir des trous ; c'est tout ce qu'elle a été *obligée* de comprendre pour y arriver.

Cette idée surprend souvent au début, alors prenons une comparaison de la vie de tous les jours.

L'intuition

Un professeur de musique demande à son élève de jouer des gammes pendant des mois. Personne n'a jamais acheté de billet pour aller écou-

ter des gammes. La gamme est un prétexte : ce que le professeur veut vraiment, ce sont des doigts agiles et une oreille juste. Le jour du concert, l'élève ne jouera aucune gamme, mais tout ce qu'il jouera en profitera. Pour notre machine, c'est pareil. Remplir des trous est la gamme. Reconnaître une action, mesurer une distance, suivre un objet : voilà le concert.

Les chercheurs ont inventé beaucoup de gammes différentes. La figure 3.3 en montre quatre parmi les plus connues. Elles ont toutes le même point commun : la bonne réponse se fabrique toute seule, en partant de l'image entière.

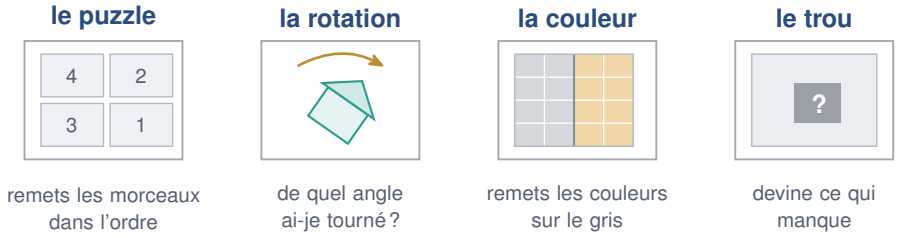


FIGURE 3.3. Quatre tâches prétextes. Dans les quatre cas, on abîme volontairement l'image d'une façon dont on garde le secret, et l'on demande à la machine de retrouver ce qui a été fait. Comme c'est nous qui avons abîmé l'image, nous connaissons toujours la bonne réponse, sans avoir eu besoin de la moindre étiquette.

V-JEPA 2.1 a choisi la quatrième : le trou. On recouvre une partie de la vidéo et l'on demande de deviner ce qui se trouvait dessous. Nous verrons dans la partie II que ce choix n'est pas un hasard : c'est celui qui oblige le plus la machine à comprendre en même temps ce qu'il y a dans la scène et comment cela bouge.

⚠ Piège à éviter

Ne cherchez jamais à juger un modèle auto-supervisé sur sa tâche prétexte. Une machine qui remplit très bien les trous n'est pas forcément une machine utile ; et une machine qui les remplit imparfaitement peut être excellente pour tout le reste. C'est même parfois le contraire de ce qu'on croit : à la fin de ce chapitre, notre petit programme atteindra

un plancher d'erreur qu'il est *impossible* de franchir, et ce sera une très bonne nouvelle.
La seule question qui compte est celle-ci : *ce que la machine a appris en jouant sert-il à autre chose ?*

Ce « quelque chose » que la machine garde après avoir joué, il faut maintenant lui donner un nom. C'est le vrai produit de tout le travail, et c'est le sujet de la section suivante.

3.5 Ce que l'on garde vraiment : la représentation

En clair

Quand la machine regarde une image, elle en fabrique un petit résumé en nombres. Ce résumé s'appelle une représentation. C'est lui, et lui seul, que l'on garde à la fin. Tout le reste — le jeu des trous, les réponses données — part à la poubelle.

Repensez à ce que vous faites quand un ami vous raconte un film. Il ne vous récite pas les deux heures image par image. Il vous dit : « c'est l'histoire d'un pêcheur qui cherche son fils ». En quelques mots, il a gardé ce qui compte et jeté le reste. Ce petit paquet d'informations essentielles, c'est exactement ce que la machine fabrique, sauf qu'elle l'écrit avec des nombres au lieu de mots.

Définition 3.8 (L'encodeur)

Un *encodeur* est la partie du modèle qui prend une image ou une vidéo en entrée, et qui en ressort un résumé beaucoup plus petit. On peut le voir comme un entonnoir : beaucoup de nombres entrent, peu de nombres sortent. C'est la pièce maîtresse de tout le livre : à la fin, c'est l'encodeur, et presque rien d'autre, que l'on met en production.

Définition 3.9 (La représentation)

La *représentation* d'une image est la liste de nombres que l'encodeur produit à partir d'elle. On dit aussi les *caractéristiques*, ou en anglais les *features*. Deux images qui se ressemblent doivent donner deux représentations proches ; deux images très différentes, deux représentations éloignées.

Définition 3.10 (L'espace latent)

L'ensemble de tous les résumés possibles s'appelle l'*espace latent*. « Latent » veut dire « caché », « qui ne se voit pas directement » : ces nombres ne sont ni des pixels ni des mots, ce sont les notes internes de la machine. Dire qu'une représentation « vit dans l'espace latent », c'est simplement dire que c'est une liste de nombres inventée par le modèle, et non quelque chose que l'on peut afficher tel quel à l'écran.

La figure 3.4 montre l'encodeur au travail.

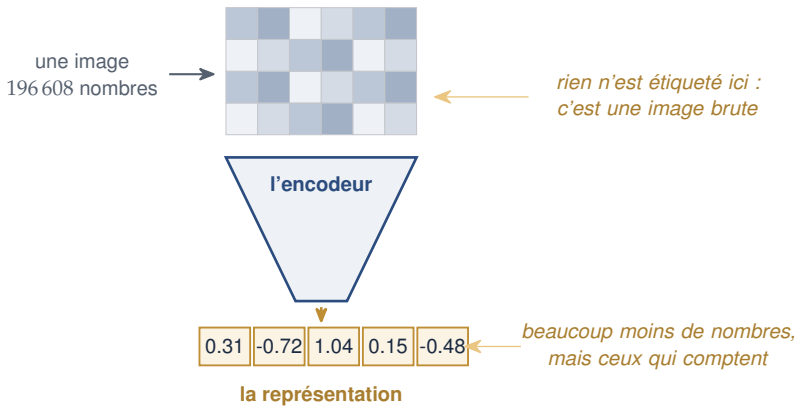


FIGURE 3.4. L'encodeur, dessiné comme un entonnoir. Une image entre par le haut, avec ses centaines de milliers de nombres. Un petit paquet de nombres sort par le bas : c'est la représentation. Ce petit paquet vit dans ce qu'on appelle l'espace latent. Tout l'enjeu du livre est de le rendre aussi riche que possible.

Mettons des nombres sur ce dessin, car c'est plus parlant.

Exemple 3.11 (De deux cent mille nombres à mille)

Une image en couleur de 256×256 pixels contient $256 \times 256 \times 3 = 196\,608$ nombres : nous l'avions calculé au chapitre 1. Supposons que l'encodeur la résume en 1 024 nombres. Le rapport vaut

$$\frac{196\,608}{1\,024} = 192.$$

Le résumé est donc cent quatre-vingt-douze fois plus petit que l'image. Toute la difficulté du métier tient dans cette division : il faut jeter cent quatre-vingt-onze nombres sur cent quatre-vingt-douze, et jeter les bons.

Remarque 3.12

Vous vous demandez peut-être : pourquoi jeter ? Pourquoi ne pas tout garder ? Parce qu'un résumé qui garde tout n'est pas un résumé : c'est une copie, et une copie n'a rien compris. La contrainte de place est précisément ce qui force la machine à distinguer l'important de l'accessoire. Dans le résumé d'un film, « un pêcheur cherche son fils » tient en six mots parce qu'on a renoncé à la couleur des rideaux.

Nous avons donc un jeu (la tâche prétexte), et un produit (la représentation). Il nous manque encore une chose : un moyen de savoir si ce produit est bon.

3.6 Comment savoir si une représentation est bonne

En clair

On bloque l'encodeur pour qu'il ne bouge plus, on branche derrière lui une toute petite machine à apprendre, et on regarde jusqu'où elle arrive. Si même une machine minuscule s'en sort bien, c'est que le résumé était déjà bien rangé.

Voici l'idée, en deux gestes. Premier geste : on décide que l'encodeur ne changera plus. Ses boutons sont fixés une fois pour toutes ; on dit qu'on le *gèle*.

Définition 3.13 (L'encodeur gelé)

Geler un encodeur, c'est décider que ses paramètres ne seront plus modifiés. On s'en sert alors comme d'une machine toute faite : on lui donne une image, il rend une représentation, et rien à l'intérieur ne bouge. C'est ainsi que V-JEPA 2.1 est évalué du début à la fin du papier — et c'est aussi, nous le verrons dans la partie VII, ce qui rend son déploiement, c'est-à-dire sa mise au travail pour de vrai, si peu coûteux.

Exemple 3.14 (Ce que le gel permet)

Imaginons que vous ayez dix mille photos et trois questions différentes à leur poser : à quelle distance est chaque objet, où sont les voitures, et de quelle sorte de scène il s'agit. Comme l'encodeur ne bouge plus, vous lui faites lire vos dix mille photos une seule fois, vous rangez les dix mille résumés dans un fichier, et vous n'y touchez plus. Vos trois questions se règlent ensuite sur ces résumés déjà calculés, sans jamais rouvrir l'encodeur. Chacune vous coûte quelques minutes, au lieu de recommencer un entraînement complet à chaque fois.

Deuxième geste : on branche derrière lui une toute petite machine à apprendre, qui, elle, a le droit d'apprendre. C'est elle qui aura besoin d'étiquettes — mais très peu, puisqu'elle est minuscule.

Définition 3.15 (La sonde)

Une *sonde* est une petite machine à apprendre que l'on branche derrière un encodeur gelé pour mesurer la qualité de ses représentations. Quand elle se réduit à une seule *couche* — c'est-à-dire une seule rangée de boutons qui multiplient les nombres reçus et les additionnent — on parle de *sonde linéaire*. C'est un instrument de mesure, comme un thermomètre : elle est volontairement faible, parce qu'une sonde faible ne peut pas rattraper un mauvais résumé.

L'intuition

Pourquoi prendre une sonde volontairement faible ? Parce qu'on

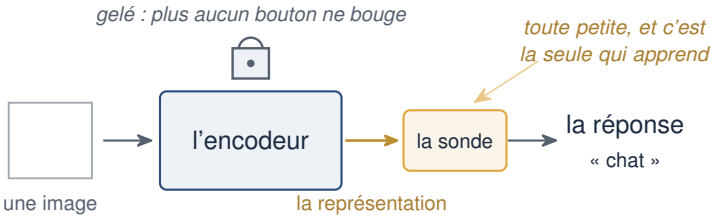


FIGURE 3.5. Le principe de l'évaluation par sonde. L'encodeur porte un cadenas : il est gelé, ses boutons ne bougent plus. Derrière lui, une toute petite sonde apprend, elle, à répondre à une question précise. Si elle y arrive avec très peu d'exemples, c'est que l'information était déjà bien rangée dans la représentation.

cherche à mesurer la qualité du *rangement*, pas la puissance de l'instrument de mesure.
Imaginez deux ateliers. Dans le premier, les outils sont rangés par taille, étiquetés, à portée de main. Dans le second, tout est jeté en vrac dans une caisse. Envoyez un enfant chercher une clé de douze : dans le premier atelier, il la trouve en dix secondes ; dans le second, jamais. Si vous envoyez un mécanicien professionnel, il finira par la trouver dans les deux cas — et vous n'aurez rien appris sur le rangement. La sonde, c'est l'enfant.

Nous reviendrons longuement sur ces mesures dans la partie III, où nous verrons comment on note vraiment une représentation. Pour l'instant, retenez le principe : *encodeur gelé, petite sonde, peu d'étiquettes*.

Ces deux gestes découpent le travail en deux temps. Le premier porte un nom qu'il faut absolument connaître, car il revient dans tous les articles et dans tout le reste du livre.

Définition 3.16 (Le pré-entraînement)

Le *pré-entraînement* est le long entraînement de départ, celui où la machine joue à la tâche prétexte sur des montagnes de données brutes, sans aucune étiquette. Le préfixe « pré » veut dire « avant » : c'est ce qui se passe *avant* qu'on lui demande quoi que ce soit d'utile. C'est de loin la partie la plus coûteuse, et elle ne se fait qu'une seule fois.

Exemple 3.17 (Les deux temps, en chiffres)

Pour V-JEPA 2.1, le pré-entraînement, c'est cent trente-cinq mille pas sur cent soixante-trois millions d'images et de vidéos, sans une seule étiquette. C'est un travail que l'on ne fait qu'une fois, et les auteurs en ont publié le résultat pour que personne n'ait à le recommencer. L'utilisation, c'est tout autre chose : une petite sonde, quelques milliers d'exemples étiquetés, et l'affaire est réglée. C'est ce deuxième temps, et lui seul, que vous ferez tourner sur votre machine — y compris à la fin de ce chapitre.

À retenir

L'auto-supervision se déroule toujours en deux temps, et il ne faut jamais les confondre :

1. **Le pré-entraînement.** La machine joue au jeu des trous sur des montagnes de données brutes, sans la moindre étiquette. C'est long, c'est cher en calcul, et cela ne se fait qu'une fois. C'est ici que naît la représentation.
2. **L'utilisation**, que l'on appelle aussi l'évaluation quand on cherche à noter le modèle. On gèle l'encodeur et l'on branche derrière une petite sonde, entraînée avec un tout petit nombre d'étiquettes, pour la tâche qui nous intéresse vraiment.

Les étiquettes n'ont donc pas complètement disparu : elles ont été repoussées à la toute fin, en quantité minuscule.

Voyons maintenant à quelle échelle tout cela se joue dans le papier que nous expliquons.

3.7 À quelle échelle : les chiffres de V-JEPA 2.1

Jusqu'ici, nous avons parlé de dix mille photos, puis de cent quarante-deux millions. Mettons les vrais chiffres du papier sur la table, pour que vous sachiez de quoi il retourne.

Sur des données réelles

Le jeu de données de V-JEPA 2.1 s'appelle *VisionMix163M*. Son nom vient tout simplement du nombre d'*échantillons* qu'il contient — un échantillon, c'est un élément de la collection, ici une image ou une vidéo :

Source	Échant.	Ce que c'est
LVD-142M	142,0 M	des images choisies avec soin
YT-1B	19,0 M	des vidéos prises sur internet
HowTo100M	1,1 M	des vidéos qui expliquent
Kinetics	0,733 M	des actions filmées de loin
SSv2	0,168 M	des gestes filmés de près
Total	$\approx$ 163 M	d'où le nom

Faites l'addition vous-même : $142 + 19 + 1{,}1 + 0{,}733 + 0{,}168 = 163{,}0$ millions. Mises bout à bout, ces vidéos durent environ 1,7 million d'heures, dont 1 600 000 pour la seule source YT-1B.
Et voici le point qui doit rester : **aucun de ces cent soixante-trois millions d'échantillons n'a été étiqueté à la main pour cet entraînement**. Le calcul de la section 2 nous a montré qu'il aurait fallu, au minimum, cent vingt-huit ans pour les images seules. Ce livre entier raconte comment on s'en passe.

Remarque 3.18

Vous avez peut-être remarqué que certains de ces jeux de données possèdent bel et bien des étiquettes : Kinetics et SSv2 sont même nés pour cela. Elles existent, mais V-JEPA 2.1 ne les regarde pas pendant le pré-entraînement. Il ne prend que les pixels. Les étiquettes ne réapparaissent qu'à la toute fin, pour entraîner les sondes et pour noter les résultats.

Assez parlé. Il est temps de faire tourner tout cela pour de vrai, sur votre machine, avec un programme que vous pouvez lire en entier.

3.8 Passons au code

Nous allons construire le plus petit exemple d'auto-supervision possible, et vérifier de nos yeux qu'il fonctionne. Le programme tient en une page, et il suit exactement les deux temps de l'encadré précédent : d'abord jouer au jeu des trous sans aucune étiquette, ensuite geler et brancher une sonde.

Voici notre monde miniature. Chaque image fait 8×8 cases, et contient une seule barre : soit une barre *couchée*, c'est-à-dire horizontale, soit une barre *debout*, c'est-à-dire verticale. Elle est posée n'importe où. Nous dirons toujours « couchée » et « debout », y compris dans le programme.

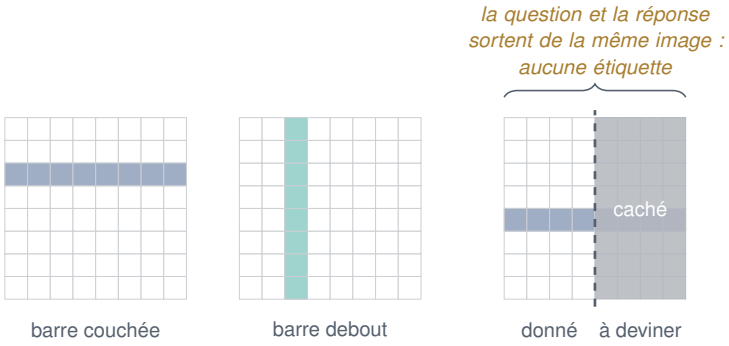


FIGURE 3.6. Notre monde miniature et le jeu que nous allons y jouer. Chaque image contient une barre, couchée ou debout. On coupe l'image en deux : la moitié gauche est donnée à la machine, la moitié droite est cachée et devient la bonne réponse. La machine ne saura jamais qu'il existe des mots comme « couchée » et « debout ».

Commençons par fabriquer ces images. Le programme ci-dessous ne contient rien que nous n'ayons expliqué : des tenseurs, comme au chapitre 1, et des tirages au hasard.

PYTHON • Fabriquer le monde miniature

```python
import torch

torch.manual_seed(0)          # pour que vous obteniez les memes nombres

def fabrique_images(n):
```

```
    """n images de 8x8 : soit une barre couchee, soit une barre debout."""
    images = torch.zeros(n, 8, 8)
    genre = torch.randint(0, 2, (n,))    # 0 = couchee, 1 = debout
    place = torch.randint(0, 8, (n,))    # a quelle ligne, ou a quelle colonne
    for i in range(n):
        if genre[i] == 0:
            images[i, place[i], :] = 1.0     # une ligne entiere
        else:
            images[i, :, place[i]] = 1.0     # une colonne entiere
    return images, genre.float()

images, genre = fabrique_images(2000)
gauche = images[:, :, :4].reshape(-1, 32)   # 8 lignes x 4 colonnes = 32
droite = images[:, :, 4:].reshape(-1, 32)   # l'autre moitie
```

Deux mots sur les deux dernières lignes, car elles contiennent un geste que nous referons souvent. L'écriture `images[:, :, :4]` veut dire : « garde toutes les images, toutes les lignes, mais seulement les quatre premières colonnes » — c'est notre moitié gauche. Quant à `reshape(-1, 32)`, il met chaque petite grille de 8×4 à plat, en une seule rangée de $8 \times 4 = 32$ nombres, comme si l'on découpait les lignes d'un tableau pour les recoller bout à bout. Le -1 dit simplement à la machine : « débrouille-toi pour le nombre d'images, tu sais compter ».

Notez bien que `genre` existe. C'est une *variable*, c'est-à-dire un nom que nous donnons à une valeur pour pouvoir la retrouver plus loin dans le programme ; celle-ci retient, pour chaque image, si la barre était couchée ou debout. Nous allons pourtant l'ignorer complètement pendant tout le pré-entraînement : elle attendra sagement la toute fin, pour la sonde. C'est notre façon d'imiter la réalité : les étiquettes existent quelque part dans le monde, mais on ne s'en sert presque pas.

Construisons maintenant les deux pièces du modèle. La première, nous la connaissons : c'est l'encodeur, qui résume les trente-deux nombres de la moitié gauche en seize nombres. La seconde est nouvelle, et il faut lui donner son nom.

Définition 3.19 (Le prédicteur)

Le *prédicteur* est la pièce qui part du résumé fabriqué par l'encodeur et qui tente de deviner ce qui a été caché. C'est lui qui joue, à proprement parler, au jeu des trous. À la fin de l'entraînement, c'est pourtant l'encodeur que l'on garde précieusement ; le prédicteur, lui, n'a servi qu'à faire travailler son voisin. Nous verrons dans la partie V qu'il sait tout de même rendre un dernier service, quand on veut deviner la suite d'une vidéo.

Chez nous, ce prédicteur part des seize nombres du résumé et tente de reconstituer les trente-deux nombres cachés.

Trois outils de PyTorch apparaissent pour la première fois. Prenons-les un par un, car nous les retrouverons dans tout le livre.

En clair

`torch.nn.Linear(32, 16)` fabrique une *couche* : une rangée de boutons qui prend trente-deux nombres en entrée, les multiplie chacun par un poids, additionne le tout, et rend seize nombres. C'est exactement la machine à bonbons du chapitre 2, en beaucoup plus large : au lieu des deux boutons a et b, celle-ci en possède $32 \times 16 + 16 = 528$.
`torch.nn.ReLU()` est une règle minuscule appliquée après la couche : tout nombre négatif est remplacé par zéro, les autres passent tels quels. Sans elle, empiler deux couches ne servirait à rien, car deux multiplications à la suite se ramènent toujours à une seule. C'est cette petite règle qui permet à un modèle d'apprendre autre chose que des droites. Quant à `Sequential`, il sert simplement à mettre ces deux pièces bout à bout.
`torch.optim.Adam` remplace la ligne `a -= pas * a.grad` que nous écrivions à la main. C'est la même descente de gradient, avec une amélioration : Adam règle tout seul la longueur du pas de chaque bouton, en se souvenant des pentes passées. Le réglage `lr=0.01` est le taux d'apprentissage de départ, celui du chapitre 2. Restent deux lignes de service. `optim.step()` bouge tous les boutons d'un coup, et `optim.zero_grad()` efface les pentes avant le tour suivant. Ce sont les deux gestes que nous écrivions bouton par bouton au chapitre précédent.

Ces trois outils paraissent abstraits tant qu'on ne les a pas vus tourner sur de

vrais nombres. Faisons-le en tout petit.

Exemple 3.20 (Une couche minuscule, calculée à la main)

Prenons une couche qui reçoit deux nombres et n'en rend qu'un seul. Elle possède donc deux poids — disons 0,5 et -2 — plus un décalage de 4, qui est exactement le bouton b de la machine à bonbons. Donnons-lui les nombres 3 et 1 :

$$0{,}5 \times 3 \;+\; (-2) \times 1 \;+\; 4 \;=\; 1{,}5 - 2 + 4 \;=\; 3{,}5.$$

La couche rend donc 3,5, et la règle ReLU le laisse passer tel quel puisqu'il est positif.
Refaisons le calcul avec les nombres 1 et 4 :

$$0{,}5 \times 1 \;+\; (-2) \times 4 \;+\; 4 \;=\; 0{,}5 - 8 + 4 \;=\; -3{,}5.$$

Cette fois le résultat est négatif : ReLU le remplace par 0, et c'est ce zéro qui poursuivra le voyage. Voilà tout le mystère. Notre vrai encodeur fait la même chose, mais avec trente-deux nombres à l'entrée, seize à la sortie, et 528 boutons au lieu de trois.

PYTHON • Le pré-entraînement, sans la moindre étiquette

```python
encodeur = torch.nn.Sequential(torch.nn.Linear(32, 16),
    torch.nn.ReLU())
predicteur = torch.nn.Linear(16, 32)

params = list(encodeur.parameters()) +
    list(predicteur.parameters())
optim = torch.optim.Adam(params, lr=0.01)

for etape in range(1, 601):
    erreur = ((predicteur(encodeur(gauche)) - droite) **
        2).mean()
    optim.zero_grad()
    erreur.backward()
    optim.step()
    if etape in (1, 100, 300, 600):
        print(f"etape {etape:3d} : erreur = {erreur.item():.4f}")
```

Vous reconnaissez la boucle en cinq gestes du chapitre 2, à un détail près : la bonne réponse, `droite`, n'a été écrite par personne. Elle a été découpée dans l'image elle-même.

Voici ce que le programme affiche :

TEXT • Ce que le programme écrit à l'écran

```
etape   1 : erreur = 0.1648
etape 100 : erreur = 0.0487
etape 300 : erreur = 0.0483
etape 600 : erreur = 0.0483
```

L'erreur tombe de 0,1648 à 0,0483, puis ne bouge plus. Cette stagnation n'est pas une panne, et c'est même le passage le plus instructif du chapitre.

Piège à éviter

Quand une barre est debout dans la moitié *droite* de l'image, la moitié gauche est entièrement vide. On demande alors à la machine de deviner une barre dont absolument rien, dans ce qu'elle voit, n'indique la position. Vous-même, devant une moitié gauche toute blanche, ne feriez pas mieux.
Dans notre tirage, cela arrive pour 25,8 % des images. Il existe donc un plancher sous lequel l'erreur ne peut pas descendre, et le modèle l'a simplement atteint. Ne confondez jamais « le modèle a cessé de progresser » avec « le modèle est mauvais » : il faut d'abord se demander si ce qu'on lui demande est seulement possible.

Passons au deuxième temps : geler, brancher une sonde, et compter les points. Nous n'accorderons à la sonde que *vingt* exemples étiquetés — c'est volontairement très peu.

Une dernière nouveauté, et ce sera la seule. Jusqu'ici, notre erreur était la moyenne des carrés des écarts, parce que nous prédisions des nombres. Ici, la sonde doit répondre à une question par oui ou par non : cette barre est-elle debout ? Pour ce genre de question, on emploie une autre mesure d'erreur, `binary_cross_entropy_with_logits`. Son nom fait peur, mais son idée tient en une phrase : elle punit d'autant plus la machine qu'elle s'est trompée *avec assurance*. Répondre « oui » à moitié convaincu quand la réponse était non

coûte peu ; répondre « oui, absolument certain » quand la réponse était non coûte très cher.

La sonde rend un seul nombre par image. On lit ce nombre ainsi : s'il est plus grand que zéro, la machine dit « debout » ; sinon, elle dit « couchée ». C'est le rôle du test `> 0` à l'avant-dernière ligne. Au passage, le mot *logits*, dans le nom de la mesure d'erreur, désigne simplement ce nombre brut, celui qu'on lit avant de le traduire en « oui » ou en « non ». Quant au petit `squeeze(1)`, il ne fait que retirer un axe devenu inutile, pour que les formes des deux tenseurs se correspondent : rien de plus qu'un rangement, au sens du chapitre 1.

PYTHON • Geler, puis brancher une sonde sur vingt étiquettes

```python
def sonde(entrees, cibles, entrees_test, cibles_test):
    """Une seule couche, qui apprend a repondre : couchee ou debout ?"""
    tete = torch.nn.Linear(entrees.shape[1], 1)
    opt = torch.optim.Adam(tete.parameters(), lr=0.05)
    for _ in range(300):
        p = tete(entrees).squeeze(1)
        perte = torch.nn.functional.binary_cross_entropy_with_logits(p, cibles)
        opt.zero_grad()
        perte.backward()
        opt.step()
    with torch.no_grad():
        prediction = (tete(entrees_test).squeeze(1) > 0).float()
        return (prediction == cibles_test).float().mean().item()

test_images, test_genre = fabrique_images(1000)
test_gauche = test_images[:, :, :4].reshape(-1, 32)

with torch.no_grad():                    # l'encodeur est gele : aucun gradient
    h = encodeur(gauche)
    h_test = encodeur(test_gauche)

torch.manual_seed(1)
print("sonde sur la representation :",
      f"{sonde(h[:20], genre[:20], h_test, test_genre)*100:.1f} %")
torch.manual_seed(1)
```

```
print("sonde sur les pixels bruts  :",
      f"{sonde(gauche[:20], genre[:20], test_gauche,
      ↪  test_genre)*100:.1f} %")
```

Et voici le résultat, mesuré sur mille images que la machine n'avait jamais vues :

TEXT • Le verdict

```
sonde sur la representation : 80.4 %
sonde sur les pixels bruts  : 61.8 %
```

Arrêtons-nous, car ces deux lignes résument tout le chapitre.

La même sonde, avec les mêmes vingt étiquettes et le même nombre d'étapes, obtient 80,4 % de bonnes réponses quand on la branche sur la représentation, contre 61,8 % quand on la branche directement sur les pixels. La différence ne vient pas de la sonde, qui est identique dans les deux cas. Elle vient de ce que l'encodeur a appris *en jouant à un jeu qui n'avait rien à voir*. Personne ne lui a jamais montré le mot « debout » ni le mot « couchée ».

Remarque 3.21

Et 80,4 %, est-ce beaucoup ? Refaisons le calcul du piège précédent. Dans 25,8 % des cas, la moitié gauche est vide et personne ne peut répondre autrement qu'en tirant à pile ou face, ce qui rapporte la moitié des points, soit environ 12,9 %. Le meilleur score atteignable est donc de l'ordre de $74{,}2 + 12{,}9 \approx 87$ %. Avec 80,4 %, notre encodeur n'est pas loin du maximum possible, et il l'a obtenu avec vingt étiquettes.

Vous n'avez pas à recopier ces morceaux à la main : le programme entier, tel que nous venons de le construire, est rangé dans le fichier `scripts/03-auto-supervision.py` qui accompagne ce livre. Lancez-le par la commande `python 03-auto-supervision.py` et vous devez retrouver, à la décimale près, les nombres imprimés ci-dessus.

Si vous obtenez autre chose, regardez d'abord la toute première ligne, `torch.manual_seed(0)`. Un ordinateur ne sait pas tirer au sort comme on lance un dé : il fabrique des nombres qui *ressemblent* à du hasard, en partant d'un nombre de départ qu'on appelle la *graine*. Avec la même graine, il refait exactement

les mêmes tirages, dans le même ordre. C'est ce qui vous permet de retrouver nos barres, nos vingt étiquettes et nos résultats à l'identique ; changez le 0 en 1, et tout le monde miniature change avec.

À quoi ça sert en IA ?

Changez maintenant les échelles, et vous avez V-JEPA 2.1. Les images de 8×8 deviennent des vidéos de soixante-quatre images en 384×384. Les seize nombres de la représentation deviennent des milliers. Les six cents étapes deviennent cent trente-cinq mille. Les deux mille images deviennent cent soixante-trois millions d'images et de vidéos. Le prédicteur, lui, devient une machine à calculer bien plus grosse, que nous démonterons pièce par pièce le moment venu.
Mais la structure du programme, elle, ne change pas d'une ligne : cacher, deviner, mesurer, corriger — puis geler et brancher une sonde.

À retenir

Les trois idées du chapitre, en trois phrases.

1. Les étiquettes coûtent trop cher pour être produites à l'échelle des vidéos : il fallait une autre source de bonnes réponses.
2. Cette source, c'est la donnée elle-même : on cache un morceau, et le morceau caché fait office de bonne réponse.
3. Le jeu ne compte pas ; ce qui compte, c'est la représentation que la machine fabrique en y jouant, et que l'on mesure avec une petite sonde derrière un encodeur gelé.

Il reste pourtant une question que nous avons soigneusement contournée, et elle est bien plus profonde qu'elle n'en a l'air. Dans notre petit programme, nous avons demandé à la machine de deviner les *pixels* cachés : les trente-deux nombres de la moitié droite, un par un. Est-ce vraiment ce qu'il faut demander ?

Réfléchissez un instant à ce que cela implique. Pour redessiner exactement les pixels d'une zone cachée dans une vraie vidéo, il faudrait deviner chaque brin d'herbe, chaque grain de la peau, chaque reflet dans une flaque. Or personne au monde ne peut prédire cela, et surtout, personne n'en a besoin.

La machine passerait le plus clair de son temps sur un travail inutile.

Il existe une autre possibilité, beaucoup plus fine : lui demander de deviner non pas les pixels cachés, mais *le résumé* de la zone cachée. Non pas l'herbe brin par brin, mais l'idée « ici, il y a de l'herbe ». C'est-à-dire prédire dans l'espace latent que nous venons de définir.

Cette question — *dans quel espace faut-il deviner : celui des pixels, ou celui des idées ?* — est le cœur de tout le papier que ce livre explique. Sa réponse porte un nom, JEPA, et c'est le sujet de la partie II.

Avant d'y arriver, il nous manque encore une pièce. Nous savons ce que la machine doit apprendre, mais nous n'avons pas encore vu *avec quoi* elle regarde. Quel est l'œil de la machine, celui qui découpe une vidéo en morceaux et les compare entre eux ? C'est le sujet du chapitre suivant.

Exercices

Exercice 1 — Le prix d'une étiquette

Vous devez faire étiqueter 500 000 images. Chaque étiquette prend huit secondes. Vous employez cinq personnes, qui travaillent chacune sept heures par jour. Combien de jours faudra-t-il ?

Solution

Le travail total vaut $500\,000 \times 8 = 4\,000\,000$ secondes, soit $4\,000\,000 / 3\,600 \approx 1\,111$ heures. Chaque jour, les cinq personnes fournissent ensemble $5 \times 7 = 35$ heures. Il faut donc

$$\frac{1\,111}{35} \approx 32 \text{ jours.}$$

Un mois de travail à cinq, pour un demi-million d'images seulement. Multipliez par deux cent quatre-vingt-quatre pour atteindre les cent quarante-deux millions d'images de V-JEPA 2.1, et vous retrouvez l'ordre de grandeur de la section 2.

Exercice 2 — Fabriquer une tâche prétexte

On vous donne un million de vidéos sans aucune étiquette. Proposez deux jeux que la machine pourrait jouer toute seule, et dites à chaque fois d'où sort la bonne réponse.

Solution

Premier jeu : remettre les images dans l'ordre. On mélange l'ordre des images d'un clip et l'on demande de le rétablir ; la bonne réponse est l'ordre d'origine, que l'on connaît puisque c'est nous qui l'avons mélangé.
Deuxième jeu : deviner la suite. On coupe le clip en deux et l'on demande de prédire la seconde moitié à partir de la première ; la bonne réponse est la seconde moitié, que l'on possède déjà.
Dans les deux cas, la règle est la même : on abîme volontairement la donnée d'une façon dont on garde la trace, et cette trace est la bonne réponse.

Exercice 3 — Le taux de compression

Un clip de 16 images en 256×256 pixels couleur est résumé par un encodeur en 512 nombres. Par combien l'information a-t-elle été divisée ?

Solution

Le clip contient $16 \times 256 \times 256 \times 3 = 3\,145\,728$ nombres, comme nous l'avions calculé au chapitre 1. Le rapport vaut

$$\frac{3\,145\,728}{512} = 6\,144.$$

Le résumé est six mille cent quarante-quatre fois plus petit que le clip. Toute la question est de savoir si l'on a jeté les bons nombres — et c'est précisément ce que mesure une sonde.

Exercice 4 — Lire un résultat de sonde

Deux encodeurs sont évalués avec exactement la même sonde et les mêmes étiquettes. Le premier obtient 78 %, le second 62 %. Que peut-on en conclure, et que ne peut-on *pas* en conclure ?

Solution

On peut conclure que le premier encodeur range mieux, dans ses représentations, l'information dont cette tâche précise a besoin : une simple sonde y accède plus facilement.
On ne peut pas conclure que le premier encodeur est meilleur en général. Il pourrait être moins bon sur une autre tâche, par exemple mesurer des distances plutôt que reconnaître des objets. Nous verrons dans la partie V que c'est exactement ce qui arrive à V-JEPA 2 : excellent pour reconnaître une action, faible pour décrire chaque pixel. Un chiffre de sonde ne vaut que pour la question posée.

Exercice 5 — Le plancher d'erreur

Dans le programme de ce chapitre, l'erreur cesse de descendre à 0,0483. Un collègue vous dit : « ton modèle est bloqué, augmente le taux d'apprentissage ». Que lui répondez-vous ?

Solution

Qu'il faut d'abord vérifier si la tâche est possible. Ici, dans 25,8 % des cas, la moitié gauche de l'image est entièrement vide alors qu'une barre se cache à droite : aucune information ne permet de la placer. Une partie de l'erreur est donc irréductible, et le modèle a simplement atteint ce plancher.
Augmenter le taux d'apprentissage ne servirait à rien, et risquerait même de faire exploser l'entraînement, comme nous l'avons vu au chapitre 2. La bonne réaction est de se demander ce que la machine *pourrait* savoir, avant de lui reprocher de ne pas le savoir.

CHAPITRE 4

Le transformeur de vision, l'œil de la machine

Nous savons ce qu'est une image pour un ordinateur, nous savons ce que veut dire apprendre, et nous savons jouer au jeu des trous. Il nous manque une chose, et une seule : l'appareil qui regarde. Dans ce chapitre, nous le construisons pièce par pièce, sans en sauter aucune. À la fin, vous saurez exactement ce qui se passe entre le moment où une vidéo entre dans la machine et le moment où elle en ressort sous forme de résumé.

Ce qu'il faut savoir avant

Les chapitres 1, 2 et 3. Il faut savoir qu'une image est une grille de nombres et connaître sa forme ; il faut savoir qu'un modèle possède des boutons que l'on tourne pour réduire une erreur ; et il faut se souvenir qu'un encodeur transforme une image en un petit paquet de nombres appelé représentation. Nous n'aurons besoin de rien d'autre. Les seules opérations employées seront l'addition, la multiplication et la division.

Au chapitre précédent, nous avons dit qu'une machine cache un morceau de ce qu'elle regarde et essaie de le deviner. Nous avons même écrit un petit programme qui le faisait. Mais nous avons triché sur un point : notre monde tenait dans une grille de huit cases sur huit, et notre modèle était une simple couche de calcul. Une vraie vidéo, elle, contient des millions de nombres. Il faut donc une machine à regarder beaucoup plus solide.

Cette machine porte un nom : le *transformeur de vision*. Tout le monde l'appelle par son surnom anglais, *ViT*, qui se prononce « vi-té » et veut dire *Vision Transformer*. C'est l'œil de V-JEPA 2.1, et c'est aussi l'œil de presque tous les grands modèles d'images d'aujourd'hui. Nous allons le monter devant vous, morceau par morceau.

4.1 Pourquoi on ne peut pas regarder les pixels un par un

> **En clair**
>
> L'idée du transformeur, c'est de laisser chaque morceau de l'image regarder tous les autres morceaux. Le problème, c'est que si les morceaux sont des pixels, il y en a beaucoup trop : le nombre de regards à échanger devient si grand qu'aucune machine au monde ne pourrait le calculer.

Commençons par la question qui commande tout le reste. Pour comprendre une image, il ne suffit pas de regarder chaque endroit tout seul dans son coin. Il faut les mettre en rapport les uns avec les autres.

Prenons un exemple. Sur une photo, vous voyez une oreille pointue en haut à gauche, des moustaches au milieu, et une queue touffue en bas à droite. Aucun de ces trois morceaux ne dit « chat » à lui tout seul : une oreille pointue, cela peut être un renard ; des moustaches, cela peut être un phoque. C'est le *rapprochement* des trois qui donne la réponse. Une bonne machine à regarder doit donc permettre à chaque endroit de l'image de consulter tous les autres endroits.

Comptons maintenant le prix de cette consultation. Imaginons que chaque pixel consulte chaque autre pixel. Dans une image de 256×256, il y a 65 536 pixels. Le nombre de consultations vaut alors

$$65\,536 \times 65\,536 = 4\,294\,967\,296,$$

soit plus de quatre milliards de consultations, et cela pour *une seule* image, à *un seul* endroit du modèle. Un clip de seize images contient seize fois plus de pixels, donc $16 \times 16 = 256$ fois plus de consultations encore. C'est tout

simplement impossible.

L'intuition

Imaginez une salle où chaque personne doit regarder toutes les autres, une par une. À dix personnes, cela fait $10 \times 10 = 100$ regards : on s'en sort en une minute. À mille personnes, cela fait $1\,000 \times 1\,000 = 1\,000\,000$ de regards : la réunion durerait des semaines. Le nombre de personnes a été multiplié par cent, mais le travail a été multiplié par dix mille. C'est ce qu'on appelle un coût *qui grimpe au carré*, et c'est l'ennemi numéro un de tout ce chapitre.

Définition 4.1 (Un coût qui grimpe au carré)

On dit qu'un coût *grimpe au carré* quand il suit le nombre d'éléments multiplié par lui-même. Appelons n le nombre d'éléments — écrire une lettre à la place d'un nombre permet simplement de parler de tous les cas à la fois. Si chacun de ces n éléments doit consulter tous les autres, le nombre de consultations vaut $n \times n$, ce qui s'écrit aussi n^2 et se lit « n au carré ».

Exemple 4.2 (Voir le coût s'emballer)

Mettons quelques nombres côte à côte, et regardons la colonne de droite grandir :

Nombre d'éléments	Travail à faire
10	100
100	10 000
1 000	1 000 000

À chaque fois que la colonne de gauche est multipliée par dix, celle de droite est multipliée par cent. Voilà pourquoi on dit que le coût *s'emballe* : il grandit bien plus vite que ce qu'on lui donne.

La solution est celle que tout le monde emploie quand une réunion est trop nombreuse : au lieu de faire parler chaque personne, on fait parler chaque *table*. On regroupe. C'est exactement ce que fait le transformeur de vision, et

c'est le sujet de la section suivante.

4.2 Le patch : découper l'image en petits carrés

En clair

Plutôt que de laisser dialoguer les pixels un par un, on découpe l'image en petits carrés de taille fixe. Chaque carré devient un porte-parole, et ce sont ces porte-parole qui dialoguent entre eux. Il y en a beaucoup moins, et tout devient calculable.

Définition 4.3 (Le patch)

Un *patch* est un petit carré de pixels découpé dans l'image, toujours de la même taille. Le mot vient de l'anglais et veut dire « pièce » ou « morceau », comme une pièce de tissu cousue sur un pantalon. Dans V-JEPA 2.1, un patch mesure 16×16 pixels. On dit aussi que la *taille de patch* vaut 16.

Le découpage se fait sans laisser de trou et sans que deux morceaux se chevauchent : on pose une grille régulière sur l'image, exactement comme on découpe une plaque de chocolat en carrés le long des rainures. Chaque carré est ensuite mis à la queue leu leu, dans un ordre convenu une fois pour toutes : la première ligne de gauche à droite, puis la deuxième ligne, et ainsi de suite. On obtient une longue file de morceaux.

Combien de morceaux obtient-on ? Le calcul est le plus simple du monde : on divise la largeur par la taille du patch, on divise la hauteur par la taille du patch, et on multiplie les deux résultats.

Exemple 4.4 (Compter les patchs d'une image de V-JEPA 2.1)

Pendant la phase principale de l'entraînement, les images font 256×256 pixels et les patchs mesurent 16×16. Sur la largeur, on peut donc poser

$$\frac{256}{16} = 16 \text{ patchs},$$

et autant sur la hauteur. Le nombre total de patchs vaut

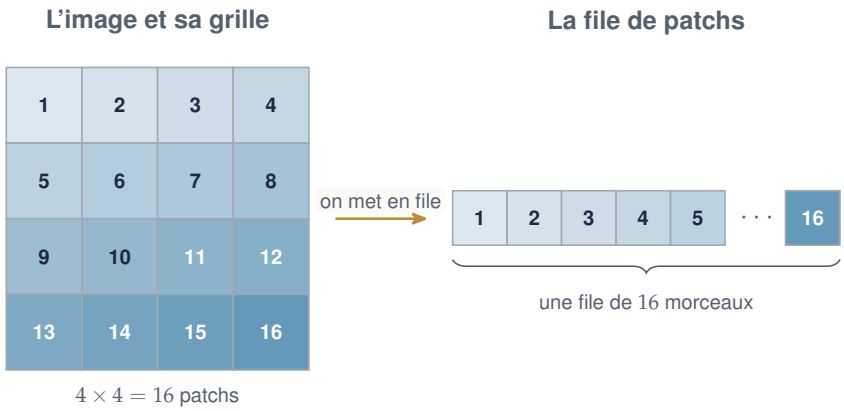


FIGURE 4.1. Le découpage en patchs, puis la mise en file. À gauche, l'image est recouverte d'une grille régulière ; chaque case est un patch et porte son numéro. À droite, les mêmes patchs sont rangés les uns derrière les autres, dans l'ordre de lecture d'un texte : ligne par ligne, de gauche à droite. Cette file est ce que la machine manipulera désormais.

$$16 \times 16 = 256.$$

Retenez bien ce nombre, 256 : nous allons le retrouver sans cesse. Souvenez-vous aussi que cette image contenait 65 536 pixels. Nous venons donc de ramener le nombre d'éléments à consulter de 65 536 à 256, c'est-à-dire de le diviser par 256. Et comme le coût grimpe au carré, le travail, lui, est divisé par 256×256, soit par plus de soixante-cinq mille.

Piège à éviter

La division doit tomber juste. Si vous prenez une image de 250 pixels de côté avec des patchs de 16, la division $250/16$ ne donne pas un nombre entier, et il vous restera une bande de pixels sur le bord que personne ne sait où ranger. C'est pour cela que les tailles d'images choisies dans l'article sont toujours des multiples de 16 : 256, 384, 512. Quand vous changerez de résolution dans vos propres essais, vérifiez toujours ce point avant de lancer quoi que ce soit.

À quoi ça sert en IA ?

Ce découpage est la toute première opération de V-JEPA 2.1, celle qui a lieu avant tout le reste. L'article lui donne un nom que nous retrouverons tout au long de la partie IV : le *tokeniseur*. C'est simplement la pièce qui fabrique les morceaux.

Nous avons donc une file de petits carrés. Mais un petit carré, c'est encore une grille de nombres. Or un transformeur ne sait pas manipuler des grilles : il manipule des listes de nombres. Il faut donc transformer chaque carré en une liste. C'est ce que nous faisons tout de suite.

4.3 Du carré au jeton : le plongement

En clair

Chaque petit carré est d'abord mis à plat, c'est-à-dire écrit comme une simple liste de nombres. Puis on lui applique un calcul qui le remplace par une autre liste, plus courte et mieux rangée. Cette nouvelle liste s'appelle un jeton, et c'est elle que le modèle manipulera du début à la fin.

Reprenons un patch de 16×16 pixels en couleur. Combien de nombres contient-il ? Nous savons le calculer depuis le chapitre 1 : la hauteur, fois la largeur, fois les trois canaux de couleur.

$$16 \times 16 \times 3 = 768 \text{ nombres.}$$

On écrit ces 768 nombres les uns derrière les autres, en une seule ligne. Cette opération s'appelle *mettre à plat*, ou *aplatir*. Rien n'est perdu, rien n'est ajouté : on change seulement la façon de ranger.

Ensuite vient le vrai travail. On ne garde pas ces 768 nombres bruts : on les remplace par une autre liste, obtenue grâce à des *coefficients*. Un coefficient, c'est tout simplement un nombre par lequel on en multiplie un autre. Ceux dont nous parlons ici sont *appris* : ce sont exactement les boutons du chapitre 2. Au début ils valent n'importe quoi, et l'entraînement les règle petit à petit.

Définition 4.5 (Le plongement, ou embedding)

Un *plongement* est une liste de nombres qui représente un morceau de donnée. On dit aussi *embedding*, qui est le mot anglais employé partout. On l'obtient en multipliant les nombres du morceau par des coefficients appris, puis en additionnant les résultats. La longueur de cette liste s'appelle la *dimension* du plongement : c'est un choix que fait celui qui construit le modèle, et elle est la même pour tous les morceaux.

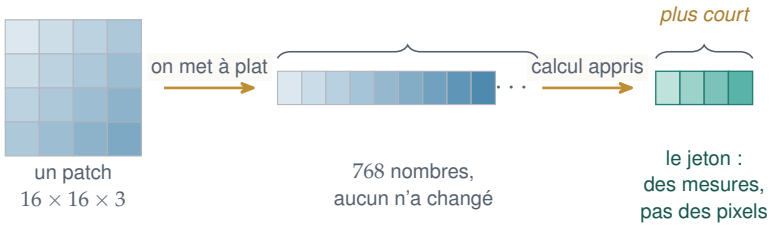


FIGURE 4.2. Du carré au jeton, en deux gestes. On part d'un patch de 16×16 pixels en couleur. On le met d'abord à plat, c'est-à-dire qu'on écrit ses nombres les uns derrière les autres : cela en fait 768, et aucun n'a été touché. On leur applique ensuite un calcul dont les coefficients sont appris, et qui rend une liste bien plus courte. Cette liste est le jeton.

Définition 4.6 (Le jeton, ou token)

Un *jeton* est le plongement d'un patch, c'est-à-dire la liste de nombres qui remplace ce patch dans toute la suite des calculs. On dit aussi *token*, en anglais. Le mot est bien choisi : comme un jeton de casino remplace une somme d'argent, un jeton remplace ici un morceau d'image. Le découpage suivi du plongement s'appelle la *tokenisation*.

Exemple 4.7 (Un plongement en tout petit)

Pour voir le calcul en entier, prenons un patch minuscule de 2×2 pixels en niveaux de gris, donc quatre nombres, et une dimension de plongement de 2 seulement. Le patch vaut

$$\begin{pmatrix} 10 & 20 \\ 30 & 40 \end{pmatrix} \quad \longrightarrow \quad \text{mis à plat : } (10,\ 20,\ 30,\ 40).$$

Le modèle possède, pour cette opération, huit coefficients appris : quatre pour fabriquer le premier nombre du jeton, quatre pour le second. Disons qu'ils valent

$$(0{,}1\ ;\ 0{,}0\ ;\ -0{,}1\ ;\ 0{,}2) \quad \text{et} \quad (0{,}0\ ;\ 0{,}5\ ;\ 0{,}0\ ;\ -0{,}5).$$

Le premier nombre du jeton s'obtient en multipliant terme à terme puis en additionnant :

$$10 \times 0{,}1 + 20 \times 0{,}0 + 30 \times (-0{,}1) + 40 \times 0{,}2 = 1 + 0 - 3 + 8 = 6.$$

Le second nombre, de la même façon :

$$10 \times 0{,}0 + 20 \times 0{,}5 + 30 \times 0{,}0 + 40 \times (-0{,}5) = 0 + 10 + 0 - 20 = -10.$$

Le jeton vaut donc $(6\ ;\ -10)$. Quatre nombres sont devenus deux nombres, et ces deux nombres ne sont plus des intensités de gris : ce sont des mesures fabriquées par le modèle. Regardez la seconde liste de coefficients, $(0\ ;\ 0{,}5\ ;\ 0\ ;\ -0{,}5)$: elle ajoute le pixel en haut à droite et retranche celui d'en bas à droite. Elle mesure donc une différence entre le haut et le bas. Personne ne le lui a demandé : c'est l'entraînement qui a réglé ces quatre nombres ainsi.

L'intuition

Le plongement est un résumé. Là où les 768 nombres bruts disaient « voici l'éclat de chacun de mes pixels », le jeton dit plutôt « je contiens un bord oblique, plutôt sombre, plutôt vert ». Personne n'a écrit ces mots nulle part : ce sont les coefficients appris qui décident, tout seuls, de ce qu'il vaut la peine de mesurer.

Sur des données réelles

Dans V-JEPA 2.1, ce calcul n'est pas fait patch par patch dans une boucle, mais d'un seul coup par une opération appelée *convolution*. Une convolution, c'est une petite fenêtre que l'on fait glisser sur l'image : à chaque arrêt, elle regarde ce qui se trouve dessous et en tire quelques nombres. Ici, la fenêtre mesure 16×16 pixels et elle avance de 16 pixels à chaque pas. Avancer d'exactement la largeur de la fenêtre, c'est précisément découper sans chevauchement. Le résultat est le même que ce que nous venons de calculer à la main, mais il est obtenu bien plus vite. Nous reverrons la convolution de près au chapitre 13 ; pour l'instant, retenez seulement l'image de la fenêtre qui glisse.

Nous savons transformer une image en une file de jetons. Reste une question que nous avons soigneusement mise de côté : et le temps ?

4.4 Le tubelet : découper aussi dans le temps

En clair

Une vidéo, c'est une pile d'images. Si l'on découpait chaque image séparément, on obtiendrait beaucoup trop de jetons. On découpe donc aussi dans l'épaisseur de la pile : chaque morceau prend deux images d'un coup. Ce petit pavé s'appelle un tubelet.

Définition 4.8 (Le tubelet)

Un *tubelet* est un patch qui possède en plus une épaisseur dans le temps. Au lieu d'être un carré plat découpé dans une seule image, c'est un petit pavé qui traverse plusieurs images de suite. Dans V-JEPA 2.1, ce pavé mesure 16×16 pixels et 2 images d'épaisseur : on dit que la *taille de tubelet* vaut 2.

Pourquoi coller deux images ensemble plutôt que de les traiter séparément ? Pour deux raisons, et les deux sont bonnes.

La première est une simple affaire de nombre. Un clip de seize images découpé image par image donnerait $16 \times 256 = 4\,096$ jetons. Avec des tubelets

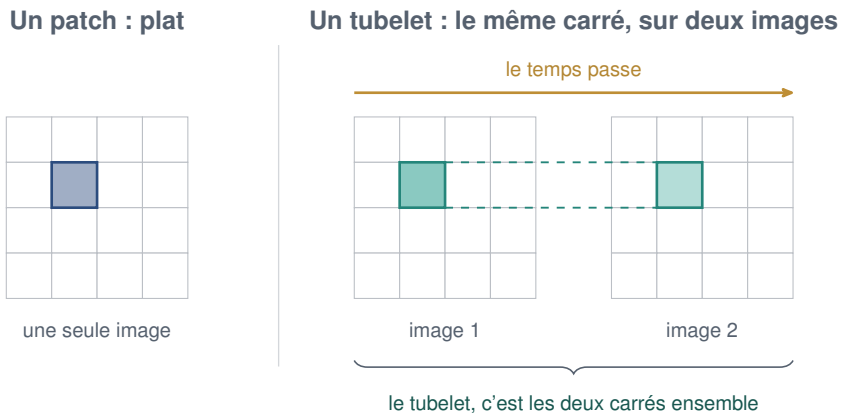


FIGURE 4.3. Le patch et le tubelet. À gauche, un patch : un carré plat, découpé dans une seule image. À droite, un tubelet : le même carré, pris sur deux images qui se suivent. Les traits pointillés montrent qu'il s'agit bien du même endroit. Le tubelet contient donc deux fois plus de pixels, et il porte en lui un petit bout de mouvement, puisqu'il voit cet endroit à deux instants.

de deux images, on en obtient deux fois moins. Comme le coût grimpe au carré, le travail est divisé par quatre.

La seconde raison est plus intéressante. Un tubelet voit le même endroit à deux instants différents. Il contient donc, dès sa fabrication, une petite trace de ce qui a bougé à cet endroit. Le mouvement n'est pas quelque chose que le modèle devra reconstituer plus tard : il est déjà là, dans le morceau.

Exemple 4.9 (Compter les jetons d'un clip de la phase principale)

Pendant la phase principale, un clip contient 16 images de 256×256 pixels. Comptons dans les trois directions.
Dans le temps, avec des tubelets de 2 images :

$$\frac{16}{2} = 8 \text{ tranches.}$$

En hauteur, avec des patchs de 16 pixels :

$$\frac{256}{16} = 16 \text{ rangées.}$$

En largeur, de même : 16 colonnes. Le nombre total de jetons vaut donc le produit des trois :

$$8 \times 16 \times 16 = 2\,048 \text{ jetons.}$$

Voilà le deuxième nombre à retenir par cœur : 2 048 **jetons pour un clip de la phase principale**.

Exemple 4.10 (Compter les jetons d'un clip de refroidissement)

Vous connaissez déjà cette phase. Le chapitre 1 vous a donné la taille de ses clips, 64 images de 384×384 pixels, et le chapitre 2 vous a dit d'où lui vient son nom : sur les douze mille dernières étapes, le taux d'apprentissage redescend jusqu'à presque zéro, et le modèle se pose en douceur. Refaisons pour elle le calcul des jetons :

$$\frac{64}{2} = 32, \qquad \frac{384}{16} = 24, \qquad \frac{384}{16} = 24,$$

$$32 \times 24 \times 24 = 18\,432 \text{ jetons.}$$

C'est le troisième nombre à retenir. Comparons-le au précédent :

$$\frac{18\,432}{2\,048} = 9.$$

Neuf fois plus de jetons : c'est exactement le rapport que nous avions trouvé au chapitre 1 en comptant les pixels bruts, et ce n'est pas un hasard, puisque le découpage garde les proportions.
Mais voici ce que le chapitre 1 ne pouvait pas encore nous dire. Là-bas, neuf fois plus de nombres voulait dire neuf fois plus de mémoire, et nous étions passés de 1,6 à 14,5 gigaoctets par lot. Ici, c'est le *calcul* qui est en jeu, et le calcul grimpe au carré :

$$9 \times 9 = 81.$$

Un clip de refroidissement coûte donc *quatre-vingt-une fois* plus de travail qu'un clip de la phase principale. La mémoire était multipliée par neuf, le calcul l'est par quatre-vingt-un. Voilà pourquoi l'article ne fait durer cette phase que 12 000 pas, alors que la phase principale en compte 135 000.

À retenir

Quatre lignes à garder en tête pour tout le livre :

Entrée	Calcul	Jetons
Image 256×256	16×16	256
Image 512×512	32×32	1 024
Clip $16 \times 256 \times 256$	$8 \times 16 \times 16$	2 048
Clip $64 \times 384 \times 384$	$32 \times 24 \times 24$	18 432

La règle est toujours la même : on divise chaque côté par la taille du découpage, puis on multiplie les résultats entre eux.

Nos morceaux sont prêts et rangés en file. Mais en les mettant en file, nous avons perdu quelque chose d'important, et il faut le récupérer sans tarder.

4.5 Dire où l'on se trouve : l'encodage de position

En clair

Une fois les jetons mis en file, plus rien ne dit lequel venait du coin en haut à gauche et lequel venait du milieu. On doit donc écrire l'adresse de chaque morceau à l'intérieur même de son jeton.

Voici le problème, et il est sérieux. Le calcul de la section suivante, l'attention, traite la file de jetons comme un *sac* : il ne regarde jamais l'ordre. Mélangez la file, il rendra exactement le même résultat. Or une image dont on mélange les morceaux n'est plus la même image du tout.

L'intuition

Prenez un puzzle terminé, photographiez-le, puis jetez toutes les pièces dans une boîte. Les pièces sont toujours là, aucune n'a été perdue, et pourtant le dessin a disparu. Ce qui manque, ce n'est pas la matière, c'est la *place* de chaque pièce. Voilà exactement ce que nous devons rendre au modèle.

Définition 4.11 (L'encodage de position)

L'*encodage de position* est la façon dont on inscrit, dans le jeton lui-même, l'endroit d'où venait son morceau. Sans lui, le modèle ne saurait pas distinguer le haut du bas, la gauche de la droite, ni le début de la fin d'une vidéo.

V-JEPA 2.1 emploie une méthode particulière, appelée *RoPE*. Ce nom est une abréviation anglaise de *Rotary Position Embedding*, ce qui veut dire « encodage de position par rotation ». Le mot rotation dit tout : au lieu d'ajouter un numéro au jeton, on *tourne* le jeton d'un angle qui dépend de sa position.

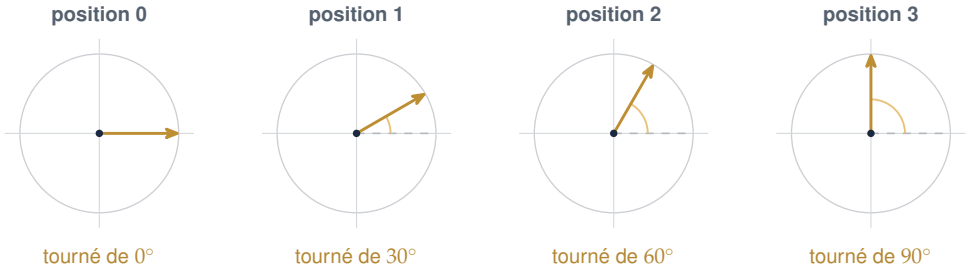


FIGURE 4.4. L'idée de RoPE, expliquée avec une aiguille d'horloge. On prend les nombres du jeton deux par deux et on les voit comme une flèche. Cette flèche est ensuite tournée d'un angle proportionnel à la position du morceau : rien à la position 0, un cran à la position 1, deux crans à la position 2, et ainsi de suite. La longueur de la flèche ne change jamais : seule sa direction porte l'information de place.

Exemple 4.12 (Tourner une flèche, avec les nombres)

Prenons deux nombres d'un jeton, disons $(1\ ;\ 0)$, et voyons-les comme une flèche qui pointe vers la droite. Convenons que chaque position fait tourner la flèche de trente degrés. Mesurons à la règle où arrive la pointe de la flèche après chaque rotation. Voici ce que l'on relève :

Position	Angle	Les deux nombres
0	0°	(1,000 ; 0,000)
1	30°	(0,866 ; 0,500)
2	60°	(0,500 ; 0,866)
3	90°	(0,000 ; 1,000)

Les deux nombres changent, alors que le contenu du morceau, lui, n'a

pas bougé. C'est la position, et elle seule, qui a fait tourner la flèche.

Pourquoi tourner plutôt qu'ajouter un numéro ? Parce que la rotation a un avantage qui tombe à pic. Quand deux jetons se comparent, ce qui compte n'est pas leur angle à chacun, mais l'*écart* entre leurs deux angles. Or cet écart ne dépend que de la *distance* entre leurs positions.

Exemple 4.13 (Ce qui compte, c'est l'écart)

Reprenons nos trente degrés par position. Deux jetons placés aux positions 1 et 3 sont séparés de deux crans, donc de 60 degrés. Deux autres jetons placés aux positions 4 et 6 sont eux aussi séparés de deux crans, donc des mêmes 60 degrés. Pour le modèle, les deux paires sont dans la même situation.
C'est très exactement ce que l'on veut. Un bord vertical reste un bord vertical, qu'il se trouve à gauche ou à droite de la photo. Ce qui importe, c'est la place des choses *les unes par rapport aux autres*, et non l'endroit exact où elles se trouvent dans l'image.

Dans une vidéo, il y a trois directions à repérer : la ligne, la colonne et l'instant. On fait donc tourner une partie des nombres du jeton selon la hauteur, une autre selon la largeur, et une troisième selon le temps. Le principe ne change pas d'un pouce : il est simplement appliqué trois fois, une fois par direction.

Les auteurs appellent cela un *RoPE en trois dimensions*. Attention à ne pas confondre ce mot avec la « dimension du plongement » vue plus haut : ici, « dimension » veut dire « direction dans laquelle on se repère », et rien d'autre.

À quoi ça sert en IA ?

Voici maintenant le point qui vous servira le jour où vous mettrez ce modèle en service, et l'article y insiste. Comme la position se traduit par un angle, on peut décider de *resserrer* ou d'*écarter* les crans.
Supposons que le modèle ait été entraîné sur des images de 16 patchs de côté, avec trente degrés par cran. On veut maintenant lui donner des images de 32 patchs de côté. Si l'on garde trente degrés par cran, la

flèche fera plusieurs tours complets et le modèle sera perdu. Mais si l'on passe à quinze degrés par cran, la traversée complète de l'image correspondra toujours au même angle total : le modèle retrouve ses repères.
Cette opération porte un nom, l'*interpolation des fréquences*, et l'article s'en sert vraiment, pour deux travaux que nous découvrirons au chapitre 7 : la *segmentation*, qui consiste à dire à quel objet appartient chaque pixel, et l'*estimation de profondeur*, qui consiste à dire à quelle distance se trouve chaque pixel.
Ce que cela permet est énorme : **on peut donner au modèle des images d'une autre taille que celles de son entraînement, sans le réentraîner**. Nous y reviendrons au chapitre 26, quand nous choisirons nos réglages de production.

Piège à éviter

« Sans le réentraîner » ne veut pas dire « sans rien vérifier ». Changer de résolution modifie toujours un peu la qualité des résultats. La règle de métier est simple : vous avez le droit de changer la résolution, mais vous avez le devoir de mesurer ce que cela change sur vos propres données. Nous verrons dans la partie VII que l'article lui-même a mesuré cet effet, et qu'il n'est pas nul.

Nos jetons contiennent maintenant ce qu'ils voient et l'endroit d'où ils viennent. Il est temps de les faire dialoguer.

4.6 L'attention : chaque jeton écoute les autres

En clair

L'attention est le cœur du transformeur. Chaque jeton pose une question, tous les autres présentent une étiquette, et celui qui pose la question écoute surtout ceux dont l'étiquette répond le mieux. Il repart alors avec un contenu enrichi de tout ce qu'il vient d'entendre.

Nous arrivons à la pièce maîtresse. Prenons le temps qu'il faut, car tout le

reste du livre s'appuie dessus.

Reprenons notre photo de chat. Le jeton qui contient les moustaches aimerait bien savoir s'il y a des oreilles pointues quelque part dans l'image. Il ne peut pas aller les chercher lui-même : il ne voit que son propre morceau. Il va donc lancer un appel à toute la file, et voir qui répond.

Pour organiser cet appel, chaque jeton fabrique trois listes de nombres à partir de son contenu, exactement comme il fabriquait son plongement : en multipliant ses nombres par des coefficients appris. Ces trois listes ont chacune un rôle bien précis.

Définition 4.14 (La question, la clé et le contenu)

À partir de son propre jeton, chacun fabrique trois listes de nombres :

- la *question*, que l'on appelle *query* en anglais : elle dit « voici ce que je cherche » ;
- la *clé*, ou *key* : elle dit « voici de quoi je parle » ;
- le *contenu*, ou *value* : c'est ce que le jeton donnera réellement à ceux qui l'auront écouté.

Les coefficients qui fabriquent ces trois listes sont appris pendant l'entraînement. Personne ne les écrit à la main.

L'intuition

Imaginez une salle de classe. Un élève lève la main et demande : « qui sait quelque chose sur les oreilles pointues ? » — c'est sa question. Chaque autre élève porte une pancarte autour du cou où il a écrit son sujet : « je parle de fourrure », « je parle d'oreilles », « je parle de flaques d'eau » — ce sont les clés. L'élève qui a posé la question regarde toutes les pancartes, repère celles qui collent le mieux à sa demande, et va écouter ces camarades-là en priorité. Ce qu'ils lui racontent alors, c'est leur contenu.

Comment mesure-t-on qu'une clé « répond bien » à une question ? On multiplie les deux listes terme à terme et on additionne le tout. Cette opération porte un nom en mathématiques : le *produit scalaire*. Elle donne un grand nombre quand les deux listes se ressemblent, et un petit nombre quand elles

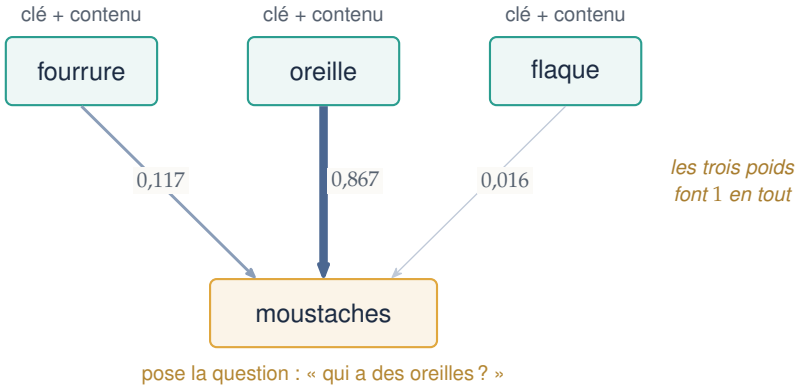


FIGURE 4.5. L'attention vue comme une salle de classe. Le jeton du bas pose sa question. Chaque autre jeton présente sa clé. Plus la clé répond à la question, plus la flèche est épaisse et plus le jeton sera écouté. Le résultat est un mélange des contenus, dosé par l'épaisseur des flèches. Ici, le jeton « oreille » est écouté fort, tandis que le jeton « flaque » est presque ignoré.

n'ont rien à voir.

Le nombre obtenu s'appelle un *score*. On en obtient un par jeton écouté. Il faut ensuite transformer ces scores en *poids*, c'est-à-dire en parts qui s'additionnent pour faire un, comme des parts d'un gâteau. L'opération qui fait cela s'appelle la *fonction softmax*.

Définition 4.15 (La fonction softmax)

La *softmax* transforme une liste de nombres quelconques en une liste de parts, toutes positives, dont la somme fait exactement 1. Elle procède en deux temps : on calcule d'abord l'*exponentielle* de chaque nombre, puis on divise chacun par la somme de tous.

L'exponentielle est une machine à agrandir. On l'écrit e^x, ce qui se lit « e puissance x ». Elle prend un nombre et en rend un autre, toujours positif, et qui grossit très vite : $e^0 = 1$, $e^1 \approx 2{,}7$, $e^2 \approx 7{,}4$, $e^4 \approx 54{,}6$. Vous n'avez pas besoin de savoir la calculer — toute calculatrice le fait. Retenez ses deux effets, qui sont exactement ceux dont la softmax a besoin : elle rend tous les nombres positifs, et elle écarte fortement les grands des petits.

Le nom « softmax » veut dire « maximum en douceur » : elle donne la

plus grosse part au plus grand score, sans pour autant écraser complètement les autres.

Exemple 4.16 (Une attention calculée du début à la fin)

Faisons le calcul en entier, avec de tout petits nombres. Reprenons les trois jetons de la figure 4.5 : « fourrure », « oreille » et « flaque ». Toutes les listes auront quatre nombres, et nous conviendrons que le premier de ces quatre nombres veut dire « je parle d'oreilles ».

Premier temps : la question. Le jeton « moustaches » cherche des oreilles. Il met donc du poids sur le premier nombre :

$$q = (2\,;\,0\,;\,0\,;\,0).$$

Deuxième temps : les clés. Les trois autres présentent les leurs. « Oreille » parle beaucoup d'oreilles, « fourrure » un peu, « flaque » pas du tout :

$$k_{\text{fourrure}} = (2\,;\,0\,;\,0\,;\,0),$$
$$k_{\text{oreille}} = (4\,;\,0\,;\,0\,;\,0),$$
$$k_{\text{flaque}} = (0\,;\,3\,;\,0\,;\,0).$$

Troisième temps : les scores. On fait le produit scalaire de la question avec chaque clé :

$$2 \times 2 = 4, \qquad 2 \times 4 = 8, \qquad 0.$$

Quatrième temps : on remet à l'échelle. On divise chaque score par la racine carrée de la longueur des listes, ici $\sqrt{4} = 2$. La racine carrée d'un nombre, c'est celui qui, multiplié par lui-même, redonne ce nombre : $\sqrt{4} = 2$ parce que $2 \times 2 = 4$, et $\sqrt{9} = 3$ parce que $3 \times 3 = 9$. Cette division empêche les scores de devenir énormes quand les listes sont longues :

$$\frac{4}{2} = 2, \qquad \frac{8}{2} = 4, \qquad \frac{0}{2} = 0.$$

Cinquième temps : la softmax. On prend l'exponentielle de chacun :

$$e^2 \approx 7{,}389, \qquad e^4 \approx 54{,}598, \qquad e^0 = 1.$$

On les additionne : $7{,}389 + 54{,}598 + 1 = 62{,}987$. Puis on divise chacun par cette somme :

$$\frac{7{,}389}{62{,}987} \approx 0{,}117, \quad \frac{54{,}598}{62{,}987} \approx 0{,}867, \quad \frac{1}{62{,}987} \approx 0{,}016.$$

Vérifions : $0{,}117 + 0{,}867 + 0{,}016 = 1{,}000$. Ce sont bien des parts de gâteau, et ce sont exactement les trois nombres écrits sur les flèches de la figure 4.5.

Sixième temps : le mélange des contenus. Disons que les trois contenus valent

$$\begin{aligned} v_{\text{fourrure}} &= (10\,;\,0\,;\,0\,;\,0), \\ v_{\text{oreille}} &= (0\,;\,10\,;\,0\,;\,0), \\ v_{\text{flaque}} &= (0\,;\,0\,;\,10\,;\,0). \end{aligned}$$

Le résultat est la somme des contenus, chacun multiplié par sa part :

$$\begin{aligned} &0{,}117 \times v_{\text{fourrure}} + 0{,}867 \times v_{\text{oreille}} + 0{,}016 \times v_{\text{flaque}} \\ &\quad = (1{,}17\,;\,8{,}67\,;\,0{,}16\,;\,0). \end{aligned}$$

Voilà le nouveau jeton « moustaches ». Il ne contient plus seulement ce qu'il voyait tout seul : il contient surtout ce que lui a dit « oreille », un peu de ce que lui a dit « fourrure », et presque rien de « flaque ». Il a posé une question sur les oreilles, et c'est bien le jeton « oreille » qu'il a le plus écouté.

Piège à éviter

N'oubliez jamais la division par la racine carrée. Sans elle, avec des listes de mille nombres, les scores atteignent des valeurs si grandes que la softmax donne une part de 1 à un seul jeton et 0 à tous les autres. Le modèle n'écoute plus qu'un seul camarade, l'entraînement se bloque, et rien dans les messages d'erreur ne vous dira pourquoi.

À retenir

L'attention se résume à six gestes, toujours les mêmes : je fabrique ma

question, chacun fabrique sa clé et son contenu, je compare ma question à chaque clé, je remets à l'échelle, je transforme les scores en parts, je mélange les contenus selon ces parts. Tout ce qui suit dans ce chapitre n'est que de l'organisation autour de ces six gestes.

Cette façon de faire est puissante, mais il lui manque quelque chose. Voyons quoi, et comment on y remédie.

4.7 Plusieurs têtes valent mieux qu'une

En clair

Avec une seule attention, chaque jeton ne peut poser qu'une seule question à la fois. On en fait donc tourner plusieurs en même temps, chacune avec ses propres réglages, et on met leurs réponses bout à bout. Chacune de ces attentions qui travaillent côte à côte s'appelle une tête.

Notre jeton « moustaches » cherchait des oreilles. Mais il aurait aussi aimé savoir quelle est la couleur dominante autour de lui, et si quelque chose bouge à sa gauche. Avec une seule question, il doit choisir. C'est dommage.

Définition 4.17 (La tête d'attention)

Une *tête d'attention* est un exemplaire complet de tout ce que nous venons de décrire, avec ses propres coefficients pour fabriquer les questions, les clés et les contenus. Plusieurs têtes travaillent en même temps sur les mêmes jetons, mais chacune pose sa propre question. On parle alors d'*attention multi-têtes*.

Le partage se fait ainsi : on ne donne pas la totalité des nombres du jeton à chaque tête, on les répartit entre elles. Chaque tête travaille sur sa part, produit sa réponse, et l'on recolle toutes les réponses bout à bout à la fin. La liste finale a donc la même longueur qu'au départ.

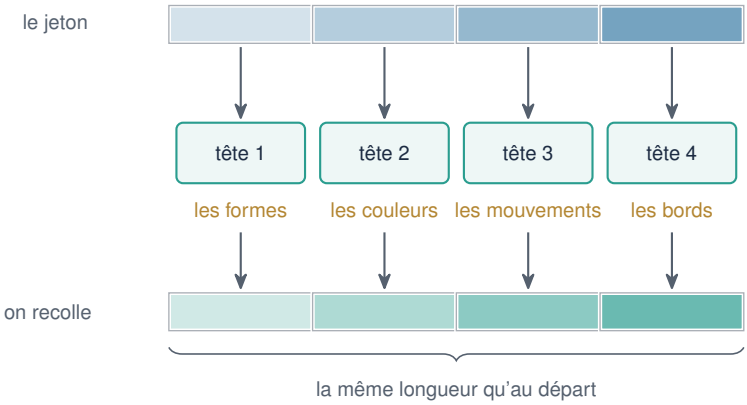


FIGURE 4.6. Le partage entre les têtes. Les nombres du jeton sont répartis en parts égales, une par tête. Chaque tête pose sa propre question et ne travaille que sur sa part. On recolle ensuite les réponses bout à bout, et l'on retrouve une liste de la même longueur qu'au départ. Quatre têtes sont dessinées ici ; le modèle en emploie seize.

Exemple 4.18 (Répartir mille vingt-quatre nombres entre seize têtes)

Prenons un jeton de 1 024 nombres et 16 têtes d'attention. Chaque tête reçoit

$$\frac{1\,024}{16} = 64 \text{ nombres.}$$

Chacune produit une réponse de 64 nombres. En recollant les seize réponses, on retrouve

$$16 \times 64 = 1\,024 \text{ nombres.}$$

La taille du jeton n'a pas changé, mais seize questions différentes ont été posées au lieu d'une seule, et pour à peu près le même prix.

L'intuition

Retournons dans la salle de classe. Avec une seule tête, l'élève ne peut poser qu'une question et doit attendre la réponse avant d'en poser une autre. Avec seize têtes, c'est comme s'il avait seize paires d'oreilles : l'une écoute les formes, l'autre les couleurs, une troisième

les mouvements. Il rassemble ensuite tout ce qu'il a entendu en une seule fiche.

Nous tenons la pièce centrale. Il faut maintenant l'emballer proprement, et c'est le rôle du bloc de transformeur.

4.8 Le bloc de transformeur, ouvert en coupe

En clair

Un bloc de transformeur, c'est l'attention plus trois pièces qui la rendent utilisable : une remise au propre des nombres avant chaque étage, un petit réseau qui réfléchit sur chaque jeton séparément, et deux raccourcis qui laissent passer l'information sans la déformer.

L'attention toute seule ne suffit pas. Elle mélange les jetons entre eux, mais elle ne fait aucun raisonnement à l'intérieur d'un jeton. Et surtout, si l'on empile beaucoup d'attentions à la suite, les nombres partent dans tous les sens et l'entraînement échoue. Trois pièces règlent ces problèmes.

Première pièce : la normalisation de couche

Définition 4.19 (La normalisation de couche, ou LayerNorm)

La *normalisation de couche* remet les nombres d'un jeton à une échelle raisonnable. Pour chaque jeton pris séparément, on calcule la moyenne de ses nombres, on la retire de chacun, puis on divise le tout par un nombre qui mesure à quel point ils s'écartaient de cette moyenne. Après cette opération, les nombres du jeton sont centrés autour de zéro et ne sont ni minuscules ni gigantesques. On l'appelle presque toujours par son nom anglais, *LayerNorm*.

Exemple 4.20 (Normaliser quatre nombres)

Un jeton contient les nombres $(2\,;\,4\,;\,6\,;\,8)$.
Sa moyenne vaut $(2+4+6+8)/4 = 20/4 = 5$. On la retire de chacun :

$$(-3\,;\,-1\,;\,1\,;\,3).$$

On mesure ensuite de combien les nombres s'écartent en moyenne. On multiplie chaque écart par lui-même — c'est ce qu'on appelle l'élever au carré — ce qui donne 9, 1, 1, 9. Multiplier par soi-même sert ici à effacer les signes moins, car un nombre négatif multiplié par lui-même devient positif. La moyenne de ces carrés vaut $(9+1+1+9)/4 = 5$, et sa racine carrée vaut $\sqrt{5} \approx 2{,}236$. On divise :

$$\left(\frac{-3}{2{,}236}\,;\,\frac{-1}{2{,}236}\,;\,\frac{1}{2{,}236}\,;\,\frac{3}{2{,}236}\right) \approx (-1{,}342\,;\,-0{,}447\,;\,0{,}447\,;\,1{,}342).$$

Les nombres tiennent maintenant à peu près entre $-1{,}5$ et $1{,}5$. Le jeton dit toujours la même chose — les rapports entre ses nombres n'ont pas changé — mais il le dit à voix normale au lieu de crier ou de chuchoter.

Voilà nos nombres remis d'aplomb. Passons à la pièce qui va s'en servir.

Deuxième pièce : le petit réseau de réflexion

Après avoir écouté les autres, chaque jeton a besoin d'un moment pour digérer ce qu'il vient d'entendre. C'est le rôle du second étage.

Définition 4.21 (Le MLP)

Un *MLP* est un petit réseau appliqué à chaque jeton séparément, sans aucun regard sur les autres. Ces trois lettres viennent de l'anglais *Multi-Layer Perceptron*. Ce nom ne vous servira jamais : retenez seulement « petit réseau de réflexion ». Il procède en trois temps : il agrandit d'abord la liste de nombres, en général en la multipliant par quatre ; il applique ensuite à chaque nombre une petite opération de tri appelée *fonction d'activation* ; enfin il ramène la liste à sa taille de départ.

Définition 4.22 (La fonction d'activation)

Vous connaissez déjà cette pièce, sans connaître son nom. Dans le programme du chapitre 3, nous avions glissé une petite règle appelée `ReLU`, qui remplaçait par zéro tout nombre négatif et laissait passer les autres. Nous avions dit alors que, sans elle, empiler deux couches ne servirait à rien.

Cette règle porte un nom de famille : on l'appelle une *fonction d'activation*. C'est une opération appliquée à chaque nombre séparément, et qui ne se contente pas de le multiplier ou de lui ajouter quelque chose. La `ReLU` est la plus simple de la famille ; celle qu'emploient les transformeurs, la `GELU`, arrondit un peu les angles mais rend le même service. Sans l'une d'elles, toute la pile de blocs se ramènerait à une seule multiplication suivie d'une seule addition.

Exemple 4.23 (Un MLP en tout petit)

Prenons un jeton de deux nombres, $(3\,;\,-2)$, et un MLP qui agrandit la liste en la multipliant par deux, donc qui passe de deux à quatre nombres.

On agrandit. Après le premier calcul, disons que l'on obtient

$$(4\,;\,-1\,;\,6\,;\,-5).$$

On trie. La fonction d'activation remplace les négatifs par zéro :

$$(4\,;\,0\,;\,6\,;\,0).$$

Regardez ce qui vient de se passer : deux des quatre mesures ont été éteintes, deux sont restées allumées. Le jeton ne réagit donc qu'à certaines situations et reste muet devant les autres. C'est exactement ce qu'une suite de multiplications ne saurait pas faire.

On revient à la taille de départ. Un dernier calcul ramène ces quatre nombres à deux, et le résultat est ajouté au jeton d'origine — nous allons voir tout de suite pourquoi.

L'intuition

L'attention, c'est le moment où l'on discute avec ses voisins. Le MLP, c'est le moment où l'on retourne à sa place pour réfléchir tout seul à ce qu'on vient d'apprendre. Un bloc de transformeur fait donc alterner, encore et encore, un temps de discussion et un temps de réflexion personnelle.

Il nous reste une pièce, la plus discrète des trois, et pourtant celle sans laquelle rien ne tiendrait debout.

Troisième pièce : la connexion résiduelle

Définition 4.24 (La connexion résiduelle)

Une *connexion résiduelle* est un raccourci : au lieu de remplacer le jeton par le résultat d'un étage, on *ajoute* ce résultat au jeton d'origine. L'ancienne valeur n'est donc jamais perdue ; l'étage ne fait qu'apporter une correction. On dit aussi *connexion saute-mouton*, parce que l'information peut sauter par-dessus l'étage.

Exemple 4.25 (Ce que le raccourci change, en chiffres)

Un jeton vaut $(3\,;\,-2)$ et l'étage qu'il traverse produit $(0{,}4\,;\,0{,}1)$.
Sans raccourci, le jeton devient $(0{,}4\,;\,0{,}1)$: tout ce qu'il contenait a disparu, remplacé par le travail de l'étage.
Avec raccourci, on additionne :

$$(3\,;\,-2) + (0{,}4\,;\,0{,}1) = (3{,}4\,;\,-1{,}9).$$

Le jeton est toujours reconnaissable, simplement corrigé un peu. Imaginez maintenant quarante-huit étages à la suite : dans le premier cas, il ne reste plus rien du jeton de départ après quelques étages ; dans le second, il est encore là, augmenté de quarante-huit petites corrections.

L'intuition

Vous relisez le devoir d'un élève. Vous pouvez soit tout réécrire de votre main — et si vous vous trompez, le travail de l'élève est perdu — soit écrire vos remarques en marge, à côté de son texte. La connexion résiduelle, c'est la deuxième façon de faire. Elle explique pourquoi l'on peut empiler quarante-huit blocs les uns sur les autres sans que le message du départ se dissolve en chemin.

Assemblons maintenant les trois pièces autour de l'attention.

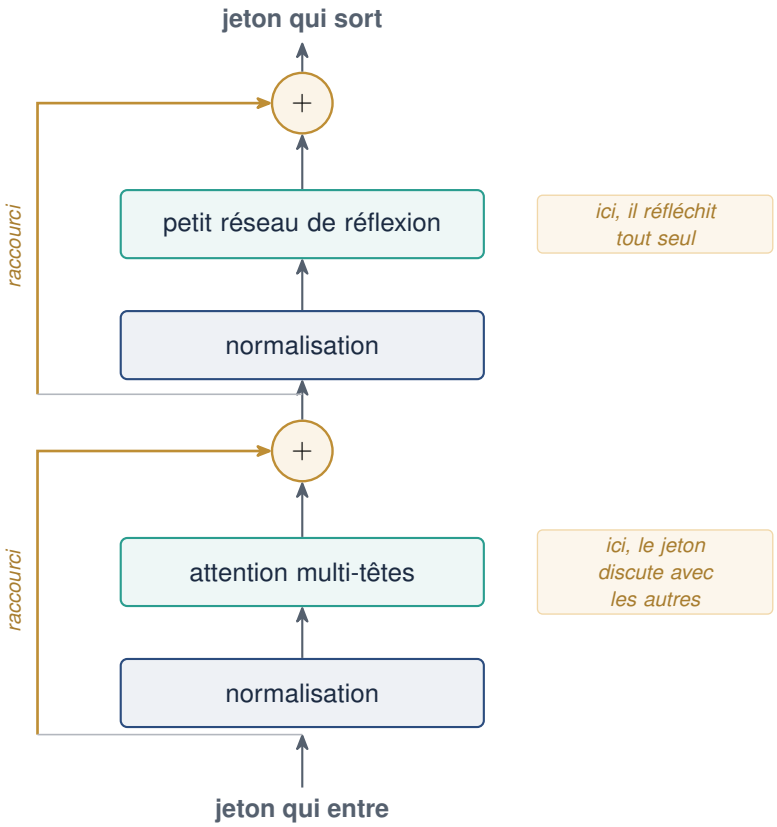


FIGURE 4.7. Un bloc de transformeur ouvert en coupe. Le jeton entre par le bas. Il passe dans deux étages successifs. Chaque étage commence par une normalisation, puis fait son travail — écouter les autres pour le premier, réfléchir tout seul pour le second — et son résultat est *ajouté* à ce qui entrait dans l'étage, grâce au raccourci dessiné sur le côté. Le jeton ressort par le haut, de la même taille qu'à l'entrée.

Définition 4.26 (Le bloc de transformeur)

Un *bloc de transformeur* est l'ensemble que nous venons d'assembler : une normalisation puis une attention multi-têtes, le tout ajouté à l'entrée ; puis une normalisation puis un petit réseau de réflexion, le tout ajouté de nouveau. Un bloc reçoit une file de jetons et rend une file de jetons de la même taille. C'est pour cela qu'on peut en empiler autant qu'on veut.

Exemple 4.27 (Un bloc, en chiffres)

Donnons à un bloc les 2 048 jetons d'un clip de la phase principale, chacun fait de 1 024 nombres. À la sortie, nous récupérons encore 2 048 jetons de 1 024 nombres.
Rien n'a changé en taille, tout a changé en contenu : chaque jeton a écouté les 2 047 autres, puis a réfléchi de son côté. Comme l'entrée et la sortie ont exactement la même forme, on peut brancher la sortie de ce bloc sur l'entrée du suivant, sans le moindre ajustement. C'est ce détail, et lui seul, qui rend les empilements possibles.

Et c'est justement ce que l'on fait dans la section suivante.

4.9 Empiler les blocs : la profondeur et les quatre tailles

En clair

On met les blocs les uns au-dessus des autres, et la file de jetons les traverse tous. Le nombre de blocs empilés s'appelle la profondeur. Plus on monte, plus ce que voient les jetons devient général.

Définition 4.28 (La profondeur)

La *profondeur* d'un transformeur est le nombre de blocs empilés les uns sur les autres. Les jetons traversent le premier bloc, puis le deuxième, et ainsi de suite jusqu'au dernier. Chaque bloc reçoit le travail du précédent.

Ce qui se passe le long de cette pile vaut la peine qu'on s'y arrête, et nous nous en servirons énormément dans la partie IV. Les premiers blocs repèrent des choses très simples : un bord, un changement de couleur, un léger déplacement. Les blocs du milieu assemblent ces petites choses en morceaux reconnaissables : un coin, une roue, une main. Les derniers blocs assemblent ces morceaux en idées complètes : une voiture, quelqu'un qui verse de l'eau.

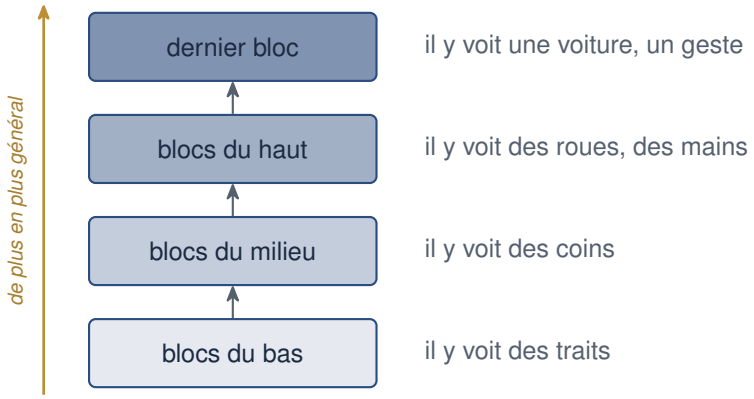


FIGURE 4.8. Ce que voient les blocs, du bas vers le haut. En bas, on repère des traits. Un peu plus haut, des coins. Plus haut encore, des morceaux d'objets. Tout en haut, l'objet entier et l'action. C'est cette montée en généralité qui explique pourquoi, plus tard, nous aurons parfois envie d'aller chercher des informations à mi-hauteur plutôt qu'au sommet.

⚠ Piège à éviter

Cette montée en généralité a un revers, et c'est exactement le problème que V-JEPA 2.1 vient résoudre. Si le sommet ne parle plus que d'idées générales, il a pu oublier *où* les choses se trouvaient. Or pour dire à quelle distance est chaque pixel, on a justement besoin du « où ». Gardez cette tension en tête : elle est le sujet de toute la partie III.

⚠ Piège à éviter

Il faut maintenant corriger une image que nous vous avions donnée au chapitre 3. Nous y avions dessiné l'encodeur comme un *entonnoir* : beaucoup de nombres entrent, un petit paquet sort. Cette image était

commode pour comprendre l'idée de résumé, mais elle est trop simple. La vérité, vous venez de la voir : l'encodeur ne rend pas *un* paquet pour toute l'image. Il rend *un paquet par jeton*. Un clip de la phase principale entre avec 2 048 jetons, et il ressort avec 2 048 jetons, un pour chaque endroit du clip. L'entonnoir n'a pas rétréci la file : il a rétréci chaque morceau, chacun de son côté.

Retenez bien cette différence, car tout le livre repose dessus. S'il vous faut une seule réponse pour toute la vidéo — « que se passe-t-il ici ? » — il faudra mélanger ces 2 048 paquets en un seul. Mais s'il vous faut une réponse *par endroit* — « à quelle distance est ce pixel-là ? » — il faudra que chaque paquet, pris tout seul, décrive bien son propre morceau. Ces deux besoins tirent le modèle dans des directions différentes, et c'est précisément le conflit que raconte la partie III.

Combien de blocs empile-t-on, et de quelle taille ? La communauté s'est mise d'accord sur quelques tailles standards, désignées par une lettre. V-JEPA 2.1 en emploie quatre.

Sur des données réelles

Les quatre tailles employées dans l'article, de la plus petite à la plus grande :

Nom	Se lit	Boutons	Rôle dans l'article
ViT-B	« bé », pour *Base*	80 M	le petit modèle de service
ViT-L	« èl », pour *Large*	300 M	le modèle des essais
ViT-g	« gé » minuscule, *giant*	1 milliard	la taille de V-JEPA 2
ViT-G	« gé » majuscule, *Giant*	2 milliards	le grand modèle final

Les deux plus petits ne sont pas entraînés depuis le début : ils apprennent en copiant le grand, par un procédé appelé *distillation* que nous verrons au chapitre 15. Retenez seulement, pour l'instant, qu'un petit modèle peut hériter de presque toute la qualité d'un grand.

Attention à la lettre **g** : la minuscule et la majuscule désignent deux modèles différents, et l'article emploie les deux à quelques lignes d'écart. Le grand modèle final, celui qui obtient tous les meilleurs résultats, est le ViT-G majuscule et compte 48 blocs empilés. Nous le savons

parce que l'article indique qu'il va chercher des informations aux blocs numéro 12, 24, 36 et 48 — nous verrons au chapitre 11 pourquoi il fait cela.

Exemple 4.29 (Ce que « deux milliards de boutons » veut dire)

Reprenez le chapitre 2 : un bouton, c'est un nombre que l'entraînement règle. Le ViT-G en possède deux milliards. Si vous les imprimiez à raison de cinquante nombres par ligne et cinquante lignes par page, il vous faudrait huit cent mille pages, soit une pile de papier de quatre-vingts mètres de haut. Et l'entraînement les règle *tous*, 147 000 fois de suite.
C'est aussi pour cela que nous consacrerons tout le chapitre 15 à rétrécir ce modèle : personne ne veut faire tourner quatre-vingts mètres de papier dans un serveur, et l'article montre comment ramener presque toute cette qualité dans un modèle six fois plus petit.

Il nous reste une dernière chose à regarder en face avant de coder : le prix de tout cela.

4.10 Ce que l'attention coûte vraiment

En clair

Dans un transformeur, le nombre de jetons est le bouton le plus important pour la vitesse. Le doubler ne double pas le travail : il le multiplie par quatre. C'est la première chose à regarder quand on choisit une résolution.

Nous l'avons dit au début du chapitre, mais nous pouvons maintenant le dire avec de vrais chiffres. Dans l'attention, chaque jeton compare sa question à la clé de tous les autres. Avec n jetons, cela fait $n \times n$ comparaisons.

Exemple 4.30 (Le prix des deux configurations de V-JEPA 2.1)

Phase principale, 2 048 jetons :

$$2\,048 \times 2\,048 = 4\,194\,304 \text{ comparaisons.}$$

Phase de refroidissement, 18 432 jetons :

$$18\,432 \times 18\,432 = 339\,738\,624 \text{ comparaisons.}$$

Le rapport vaut

$$\frac{339\,738\,624}{4\,194\,304} = 81.$$

Quatre-vingt-une fois plus de travail, et cela dans *chacun* des quarante-huit blocs. Voilà le chiffre qui explique tout le calendrier de l'entraînement décrit dans l'article.

À retenir

Retenez la règle du praticien : **ce que vous paierez suit à peu près le carré du nombre de jetons**. Quand votre modèle sera trop lent en service, le premier bouton à tourner ne sera ni le processeur ni la bibliothèque : ce sera la résolution d'entrée, parce que c'est elle qui décide du nombre de jetons. Nous referons ce calcul, pour de vrai, au chapitre 29.

Nous avons monté l'œil de la machine en entier. Vérifions maintenant, avec un programme, que tout ce que nous avons calculé à la main est bien ce qui se passe pour de vrai.

4.11 Passons au code

Nous allons écrire quatre petits programmes. Les deux premiers découpent une image et un clip en jetons, et retrouvent les quatre nombres du tableau. Le troisième refait, ligne par ligne, l'exemple d'attention calculé à la main. Le quatrième assemble un bloc de transformeur complet. Vous pouvez les taper tels quels : ils tournent sur n'importe quel ordinateur, même sans le processeur spécialisé dont les gros entraînements ont besoin.

Commençons par le découpage. Comme annoncé, on l'obtient avec une convolution dont la fenêtre et le pas ont la même taille.

PYTHON • Découper une image et un clip en jetons

```python
import torch
import torch.nn as nn

# --- La voie image : une convolution 2D de 16 x 16, qui avance de 16 ---
tokeniseur_image = nn.Conv2d(
    in_channels=3,       # les trois couleurs
    out_channels=1024,   # la taille d'un jeton
    kernel_size=16,      # la fenetre : un patch de 16 x 16
    stride=16,           # elle avance de 16 : aucun chevauchement
)

image = torch.zeros(1, 3, 256, 256)          # (lot, couleurs, hauteur, largeur)
sortie = tokeniseur_image(image)             # -> (1, 1024, 16, 16)
jetons_image = sortie.flatten(2).transpose(1, 2)

print(jetons_image.shape)   # -> torch.Size([1, 256, 1024])
# 256 jetons : c'est bien le nombre calcule a la main.

# --- La voie video : une convolution 3D, avec 2 images d'epaisseur ---
tokeniseur_video = nn.Conv3d(
    in_channels=3,
    out_channels=1024,
    kernel_size=(2, 16, 16),   # (tubelet, hauteur, largeur)
    stride=(2, 16, 16),
)

clip = torch.zeros(1, 3, 16, 256, 256)       # (lot, couleurs, temps, H, W)
sortie = tokeniseur_video(clip)              # -> (1, 1024, 8, 16, 16)
jetons_video = sortie.flatten(2).transpose(1, 2)

print(jetons_video.shape)   # -> torch.Size([1, 2048, 1024])
```

```
# 2048 jetons : 8 x 16 x 16, exactement notre calcul.
```

Piège à éviter

Regardez bien l'ordre des axes : PyTorch attend les couleurs en deuxième position, pas en dernière. C'est le changement d'ordre que nous avions déjà rencontré au chapitre 1. Si vous lui donnez un tenseur rangé dans l'ordre de l'affichage, il ne se plaindra pas toujours : il calculera quelque chose de faux, en silence. Prenez l'habitude d'afficher la forme avant et après chaque étape.

Vérifions maintenant les quatre nombres du tableau, tous d'un coup.

PYTHON • Vérifier les quatre nombres de jetons

```python
def compte_jetons(images, hauteur, largeur, patch=16, tubelet=2):
    """Rend le nombre de jetons. Mettre images=None pour une
    ↪ image seule."""
    n = (hauteur // patch) * (largeur // patch)
    if images is not None:
        n = n * (images // tubelet)
    return n

print(compte_jetons(None, 256, 256))   # -> 256
print(compte_jetons(None, 512, 512))   # -> 1024
print(compte_jetons(16,  256, 256))    # -> 2048
print(compte_jetons(64,  384, 384))    # -> 18432

# Et le prix de l'attention, qui grimpe au carre :
print(18432**2 / 2048**2)              # -> 81.0
```

Passons à l'attention. Nous reprenons exactement les nombres de l'exemple calculé à la main : vous devez retrouver les mêmes parts, à la troisième décimale.

PYTHON • L'attention, refaite avec nos nombres

```python
import torch

```

```python
# Notre question, puis les cles et les contenus de fourrure, oreille, flaque.
q = torch.tensor([2.0, 0.0, 0.0, 0.0])
k = torch.tensor([[2.0, 0.0, 0.0, 0.0],    # fourrure
                  [4.0, 0.0, 0.0, 0.0],    # oreille
                  [0.0, 3.0, 0.0, 0.0]])   # flaque
v = torch.tensor([[10.0,  0.0,  0.0, 0.0],
                  [ 0.0, 10.0,  0.0, 0.0],
                  [ 0.0,  0.0, 10.0, 0.0]])

scores = k @ q                       # les produits scalaires
print(scores)                        # -> tensor([4., 8., 0.])

scores = scores / (q.shape[0] ** 0.5)   # on divise par racine de 4, donc par 2
print(scores)                        # -> tensor([2., 4., 0.])

poids = torch.softmax(scores, dim=0)
print(poids)                         # -> tensor([0.1173, 0.8668, 0.0159])
print(poids.sum())                   # -> tensor(1.)

resultat = poids @ v
print(resultat)                      # -> tensor([1.1731, 8.6681, 0.1588, 0.])
```

Les trois parts sont bien 0,117, 0,867 et 0,016, dans cet ordre, et le résultat est bien celui que nous avions trouvé à la main. Le programme ne fait rien d'autre que ce que nous avons écrit sur le papier.

Terminons par un bloc de transformeur complet. Comptez les lignes : il n'y en a pas vingt. Tout ce chapitre tient là-dedans.

</> PYTHON • Un bloc de transformeur, en entier

```python
import torch
import torch.nn as nn

class BlocTransformeur(nn.Module):
    def __init__(self, dimension=1024, nb_tetes=16):
        super().__init__()
        self.norme1 = nn.LayerNorm(dimension)
```

```
        self.attention = nn.MultiheadAttention(
            dimension, nb_tetes, batch_first=True)
        self.norme2 = nn.LayerNorm(dimension)
        self.reflexion = nn.Sequential(
            nn.Linear(dimension, 4 * dimension),   # on agrandit
            nn.GELU(),                             # la fonction
            ↪ d'activation
            nn.Linear(4 * dimension, dimension),   # on revient a
            ↪ la taille de depart
        )

    def forward(self, jetons):
        # Premier etage : on ecoute les autres, et on AJOUTE le
        ↪ resultat.
        propre = self.norme1(jetons)
        ecoute, _ = self.attention(propre, propre, propre)
        jetons = jetons + ecoute

        # Second etage : on reflechit tout seul, et on AJOUTE
        ↪ encore.
        jetons = jetons + self.reflexion(self.norme2(jetons))
        return jetons

bloc = BlocTransformeur()
jetons = torch.zeros(1, 2048, 1024)     # un clip de la phase
↪ principale
print(bloc(jetons).shape)               # -> torch.Size([1, 2048,
↪ 1024])

# La taille ne change pas : c'est ce qui permet d'empiler 48
↪ blocs.
encodeur = nn.Sequential(*[BlocTransformeur() for _ in
↪ range(48)])
```

Piège à éviter

Repérez les deux `jetons = jetons + ...` : ce sont les connexions résiduelles. Si vous écrivez `jetons = ecoute` au lieu de `jetons = jetons + ecoute`, le programme tournera sans erreur, mais l'entraînement d'un modèle profond ne convergera jamais. C'est l'une des erreurs les plus

fréquentes, et l'une des plus difficiles à repérer, parce que rien ne casse : les résultats sont simplement mauvais.

À retenir

Un transformeur de vision, c'est quatre gestes et rien de plus. On découpe l'image ou la vidéo en petits morceaux. On remplace chaque morceau par une liste de nombres, dans laquelle on inscrit aussi sa position. On empile des blocs, où les jetons discutent entre eux puis réfléchissent chacun de leur côté. Et l'on récupère à la sortie une file de jetons de la même taille qu'à l'entrée, mais dont chacun a été enrichi par tous les autres.

Nous avons maintenant tout ce qu'il faut. Nous savons ce qu'est une image pour une machine, ce que veut dire apprendre, comment jouer au jeu des trous sans étiquettes, et de quoi est fait l'œil qui regarde. Ces quatre chapitres formaient les fondations, et elles sont posées.

Une seule question reste en suspens, celle que nous avions laissée à la fin du chapitre 3 : quand la machine devine ce qui est caché, doit-elle deviner les pixels, ou l'idée ? La réponse à cette question porte un nom, JEPA, et c'est par elle que s'ouvre la partie II.

Exercices

Exercice 1 — Compter des jetons

Vous donnez au modèle une image de 768×768 pixels, avec des patchs de 16. Combien de jetons obtenez-vous ? Et un clip de 32 images en 256×256, avec des tubelets de 2 ?

Solution

Pour l'image : $768/16 = 48$ sur chaque côté, donc $48 \times 48 = 2\,304$ jetons. Pour le clip : $32/2 = 16$ tranches de temps, et $256/16 = 16$ patchs sur chaque côté, donc $16 \times 16 \times 16 = 4\,096$ jetons. Remarquez que ce clip

coûte $(4\,096/2\,048)^2 = 4$ fois plus cher qu'un clip de la phase principale, alors qu'il ne contient que deux fois plus d'images.

Exercice 2 — Choisir une résolution sous contrainte

Votre serveur ne peut pas dépasser 5 000 jetons par clip. Vous voulez des clips de 16 images. Quelle est la plus grande résolution carrée, multiple de 16, que vous pouvez vous permettre ?

Solution

Avec 16 images et des tubelets de 2, il y a 8 tranches de temps. Le budget par tranche vaut donc $5\,000/8 = 625$ patchs. Comme l'image est carrée, il faut au plus $\sqrt{625} = 25$ patchs de côté, ce qui donne $25 \times 16 = 400$ pixels. Et 400 est bien un multiple de 16 : la réponse est donc 400×400, qui consomme $8 \times 25 \times 25 = 5\,000$ jetons, exactement le budget.
Un mot de prudence, quand même. Tomber pile sur la limite ne laisse aucune marge, et l'article, lui, a retenu 384×384 pour ses vidéos, ce qui donne $8 \times 24 \times 24 = 4\,608$ jetons. Choisir la résolution juste en dessous de son plafond est une habitude qui évite bien des mauvaises surprises le jour où le serveur est chargé.

Exercice 3 — Répartir entre les têtes

Un modèle a des jetons de 1 408 nombres et 22 têtes d'attention. Combien de nombres chaque tête reçoit-elle ? Que se passerait-il avec 20 têtes ?

Solution

Avec 22 têtes : $1\,408/22 = 64$ nombres chacune. Le compte tombe juste. Avec 20 têtes : $1\,408/20 = 70{,}4$, ce qui n'est pas un nombre entier. On ne peut pas donner 70,4 nombres à une tête. La bibliothèque refusera de construire le modèle. C'est pour cela que le nombre de têtes doit toujours diviser exactement la taille du jeton.

Exercice 4 — Une softmax à la main

Trois scores valent 1, 1 et 3. Calculez les trois parts. Que remarquez-vous sur les deux premières ?

Solution

Les exponentielles valent $e^1 \approx 2{,}718$, $e^1 \approx 2{,}718$ et $e^3 \approx 20{,}086$. Leur somme vaut 25,522. Les parts valent donc

$$\frac{2{,}718}{25{,}522} \approx 0{,}106, \qquad \frac{2{,}718}{25{,}522} \approx 0{,}106, \qquad \frac{20{,}086}{25{,}522} \approx 0{,}787.$$

Les deux premières sont égales, ce qui est normal : elles avaient le même score. Et l'on voit l'effet de l'exponentielle : le score 3 n'est que trois fois plus grand que le score 1, mais il récolte sept fois et demie plus de part, puisque $0{,}787/0{,}106 \approx 7{,}4$. La softmax écarte bel et bien les grands des petits.

Exercice 5 — Le mélange des positions

Un collègue vous dit : « j'ai mélangé l'ordre de mes jetons avant de les donner au modèle, mais comme ils contiennent tous la même information qu'avant, cela ne change rien ». A-t-il raison ?

Solution

Il a raison sur un point et tort sur l'essentiel. L'attention traite bien la file comme un sac : à elle seule, elle rendrait le même résultat.
Mais les jetons ne contiennent pas que leur contenu : l'encodage de position y a inscrit l'endroit d'où ils viennent. En mélangeant la file, il a donc associé chaque morceau à une mauvaise adresse. Pour le modèle, c'est comme si l'on avait recollé les pièces du puzzle au hasard : les pièces sont justes, le dessin ne l'est plus. Le résultat sera faux, et sans le moindre message d'erreur.

Exercice 6 — Le prix de la finesse

Un ingénieur propose de passer les patchs de 16 à 8 pixels, « pour voir plus de détails », en gardant des images de 256×256. Par combien le travail de l'attention est-il multiplié ?

Solution

Avec des patchs de 16, on avait $16 \times 16 = 256$ jetons. Avec des patchs de 8, on obtient $256/8 = 32$ patchs de côté, donc $32 \times 32 = 1\,024$ jetons : quatre fois plus.
Comme le coût grimpe au carré, le travail est multiplié par $4 \times 4 = 16$. Sa proposition améliorera sans doute la finesse, mais elle coûtera seize fois plus cher. C'est un choix parfaitement défendable — à condition de le faire les yeux ouverts, et de vérifier que le gain de qualité vaut la dépense.

Deuxième partie

Comprendre JEPA et V-JEPA 2, l’ancêtre direct

Troisième partie

Le problème que V-JEPA 2.1 vient résoudre

Quatrième partie

V-JEPA 2.1, ingrédient par ingrédient

Cinquième partie

Les résultats, lus avec honnêteté

Sixième partie

Construire le modèle et le programme d'entraînement

Septième partie

De l'entraînement à la production

Bibliographie

[1] M. ASSRAN et al. « V-JEPA 2 : Self-Supervised Video Models Enable Understanding, Prediction and Planning ». In : *arXiv preprint arXiv :2506.09985* (2025). URL : https://arxiv.org/abs/2506.09985.

[2] L. MUR-LABADIA et al. « V-JEPA 2.1 : Unlocking Dense Features in Video Self-Supervised Learning ». In : *arXiv preprint* (2026). FAIR at Meta et Universidad de Zaragoza. URL : https://github.com/facebookresearch/vjepa2.